COMP0411 중간고사 · 10월 21일(수) 10:30

컴퓨터구조 벼락치기

Ch1~Ch4를 **5~10분짜리 단원 36개**로 나눴습니다. 단원마다 순서가 똑같습니다: **① 읽기 → ② 핵심 → ③ 풀이 예 → ④ 떠올리기**. 한 번에 단원 하나만 하면 됩니다.

지금 할 일은 딱 세 가지

1. 오른쪽 위 **[집중 15분]**을 누른다.
2. 목차에서 아직 안 한 단원 하나를 열고 ①부터 ④까지 차례로 한다. 단원 맨 위에 **예상 시간**이 있다.
3. 끝나면 맨 아래 **"이 단원 다 봤어요"**를 체크하고 [다음]. 15분이 끝나면 알림이 뜨니 **5분 쉬기**.

단원 안에서

- **① 읽기**: 처음 보는 사람용 설명. 위에서부터 그냥 읽으면 됩니다.
- **② 핵심**: 외울 말에 노란 밑줄. 다 읽었으면 **[가리고 떠올리기]**를 눌러 노란 칸을 먼저 떠올리고, 칸을 눌러 확인합니다.
- **③ 풀이 예**: 계산 단원만. 따라 읽기.
- **④ 떠올리기**: 빈칸 풀이(노란 칸을 채우고 눌러 확인)와 카드. 카드는 답을 떠올린 뒤 펼치고 **[맞음]/[틀림]**. 틀린 카드는 **오답 노트**에 자동으로 모입니다.

남은 계획

날짜	할 것
10/19 (월)	Ch1 (9단원) → Ch2 · 1 ~ 5. 집중 15분 × 6~8번.
10/20 (화)	Ch2 · 6 ~ 9 → Ch3 (9단원) → Ch4 (9단원). 저녁에 오답 노트 다시 풀기.
10/21 (수) 아침	**시험 전날 한 장**(노란 칸 떠올리기) + 오답 노트. 시험 10:30.
틈날 때	폰으로 암기 퀴즈 한 판(20문제, 약 3분).

중간고사 공지	
시간	10월 21일(수) 오전 10:30 ~ 오후 12:00 (90분)
장소	IT5호관(융복합관) B101호
형식	Closed Book 필기시험, 단답형 위주(T/F, 빈칸, 계산, 코드), 한글 시험
범위	Ch1 ~ Ch3 전체 + Ch4 단일 사이클(p.1–29)

단원별 진도

필수 기초 1~4는 이미 끝냈으니 넘어가도 됩니다. 기초가 흔들리면 기초 · 5(문제 100개)를 풀어 보세요. 더 자세한 설명은 이론 노트에 있습니다.

기초 · 1 슬라이드 Ch2 p.17–18 (쓰이는 곳: Ch2 · 4, Ch3 · 2)

진법 변환과 16진수

시험에 나오는 모양: 10진 ↔ 2진 ↔ 16진 바꾸기, 32비트 값을 16진 8자리로 쓰기, 10진 소수를 2진으로(IEEE 754의 첫 단계). 이 단원이 막히면 인코딩·IEEE 754 문제를 하나도 못 푼다.

쉽게 말하면

2진수는 자릿값이 **오른쪽 끝 자리부터** 1, 2, 4, 8, … 로 두 배씩 커지는 수다(10진수가 1, 10, 100, … 로 열 배씩 커지는 것과 같은 원리). 2진수 한 자리를 **비트(bit)**라 부른다. 1011_2는 4비트다. 16진수는 2진 4자리를 한 글자로 줄여 쓴 것이라 2진과 16진 사이는 계산 없이 표만 보고 바꾼다.

외울 것

n	0	1	2	3	4	5	6	7	8
2^n	1	2	4	8	16	32	64	128	256

n	9	10	11	12	13	14	15	16	32
2^n	512	1024	2048	4096	8192	16384	32768	65536	4,294,967,296 (약 43억)

n	−1	−2	−3	−4	−5
2^n	0.5	0.25	0.125	0.0625	0.03125

16진	0	1	2	3	4	5	6	7
2진	0000	0001	0010	0011	0100	0101	0110	0111

16진	8	9	A(10)	B(11)	C(12)	D(13)	E(14)	F(15)
2진	1000	1001	1010	1011	1100	1101	1110	1111

- **2진 → 10진**: 1이 있는 자리의 자릿값을 더한다. 1011_2 = 8 + 2 + 1 = 11. [Ch2 p.17]
- **10진 → 2진(정수)**: 2로 나누기를 **몫이 0이 될 때까지** 반복한다(마지막은 1 ÷ 2 = 몫 0, 나머지 1). 나머지를 **아래에서 위로** 읽는다. 작은 수는 **자릿값 빼기(분해)**도 된다: 13 = 8 + 4 + 1 → 8·4·1 자리에 1 → 1101_2(Ch3에서 이 방법을 씀).

  왜 아래 → 위인가: 2로 나눈 나머지는 그 수가 홀수(1)인지 짝수(0)인지, 곧 **1의 자리 비트**다. 몫은 1의 자리를 떼어 내고 남은 비트들을 한 칸 오른쪽으로 당긴 수라, 몫을 다시 나누면 2의 자리, 그다음은 4의 자리 비트가 나온다. 먼저 나온 나머지일수록 오른쪽 자리이므로 쓸 때는 마지막(맨 아래) 나머지를 맨 왼쪽에 둔다. 예 6: 6 ÷ 2 = 3 … 0(1의 자리 0), 3 ÷ 2 = 1 … 1(2의 자리 1), 1 ÷ 2 = 0 … 1(4의 자리 1) → 110_2 = 4 + 2 = 6.

  왜 몫 0까지인가: 몫이 1이면 아직 쓰지 않은 자리(값 1)가 하나 남아 있다는 뜻이다. 1 ÷ 2 = 0 … 1을 해야 그 1이 맨 왼쪽 비트로 나온다. 몫이 0이면 남은 자리가 없으니 멈춘다.
- **10진 → 2진(소수)**: 소수 부분에 2를 곱하고 나온 정수 부분(0 또는 1)을 떼어 적는다. 소수 부분이 0이 될 때까지 반복하고 **위에서 아래로** 읽는다.

  왜 위 → 아래인가: 2진 소수의 자릿값은 소수점 바로 오른쪽부터 0.5, 0.25, 0.125, …다. 2를 곱하면 0.5 자리가 1의 자리로 올라와 정수 부분에 드러난다. 그래서 처음 나온 정수 부분이 소수점 바로 오른쪽(0.5 자리), 다음이 0.25 자리다. 예 0.75: 0.75 × 2 = 1.5 → 1(0.5 자리), 0.5 × 2 = 1.0 → 1(0.25 자리) → 0.11_2 = 0.5 + 0.25.

- **2진 ↔ 16진**: 소수점(정수면 오른쪽 끝)에서부터 4비트씩 묶어 표로 바꾼다. 모자란 왼쪽은 0으로 채운다.
  4비트씩인 이유: 4비트로 만들 수 있는 패턴은 0000 ~ 1111, 곧 0 ~ 15의 16가지($2^4 = 16$)라 16진 한 자리와 하나씩 짝이 맞는다.
- **16진 자릿값**: 오른쪽부터 1, 16, 256, 4096(16배씩). 0x2BC = 2 × 256 + 11 × 16 + 12 = 700.
- **10진 → 16진**: ÷16 반복(나머지 아래 → 위), 또는 256·16 단위로 빼기. 700 = 2 × 256 + 188, 188 = 11 × 16 + 12 → 2, B, C → 0x2BC.
- **32비트 = 16진 8자리**. 16진 한 자리가 4비트라 32 ÷ 4 = 8자리다. 앞의 0도 써서 8자리를 채운다: 42 = 0x2A → 0x0000002A. 16진은 0x를 붙이고 A~F는 대문자.
- **2진 덧셈**: 0 + 1 = 1, 1 + 1 = 10_2(0 쓰고 1 올림), 1 + 1 + 1 = 11_2(1 쓰고 1 올림). 예 $0110_2 + 0111_2$: 오른쪽부터 1, 0(올림 1), 1(올림 1), 1 → 1101_2(6 + 7 = 13).
- **16진 덧셈**: 자리 합이 16 이상이면 **16을 빼고** 왼쪽 자리에 1을 올린다(15가 아님). **16진 뺄셈**: 모자라면 왼쪽에서 1을 빌려 **16을 더한** 뒤 뺀다.
  왜 16인가: 16진은 한 자리 왼쪽으로 갈 때마다 자릿값이 16배다(1, 16, 256, …). 그래서 어떤 자리에 16이 모이면 그것은 바로 왼쪽 자리의 1과 같은 양이다. 10진에서 7 + 5 = 12일 때 10을 왼쪽 1로 올리고 2를 쓰는 것과 똑같은 일이다(10진은 10배씩이라 10을 뺀다). 15(F)는 한 자리에 쓸 수 있는 가장 큰 숫자일 뿐 올림 단위가 아니다. 뺄셈도 같다: 왼쪽 자리에서 빌려 온 1은 이 자리로 오면 16이다.

푸는 순서

1. 정수 부분과 소수 부분을 나눈다.
2. 정수는 ÷2 반복 또는 자릿값 빼기, 소수는 ×2 반복(위 규칙대로 읽는다).
3. 둘을 소수점으로 잇는다. 16진이 필요하면 4비트씩 묶는다.
4. 검산: 자릿값을 더해 원래 10진이 나오는지 본다.

함정

- 나누기를 몫 1에서 멈추면 맨 앞 1이 빠진다. 1 ÷ 2 = 몫 0, 나머지 1까지 해야 한다.
- 소수 ×2의 결과를 아래 → 위로 읽으면 틀린다. 0.375 = 0.011_2이지 0.110_2가 아니다.
- 16진 덧셈에서 0xA + 0x8 = 18이면 18 − 16 = 2, 올림 1 → 0x12. 15를 빼서 3으로 쓰는 실수가 잦다.

예제 (시험 수준)

37.375를 2진수로 쓰시오.

풀이 보기

회독 점검

1. 89를 2진수와 16진수로 쓰시오.

   답 보기

---

2. 1101 0110_2를 10진수와 16진수로 쓰시오.

   답 보기

---

3. 10.625를 2진수로 쓰시오.

   답 보기

---

4. 16진 덧셈 0x3A7 + 0x2C9를 계산하시오.

   답 보기

---

5. 16진 뺄셈 0x5A2 − 0x1B7을 계산하시오.

   답 보기

---

6. 빈칸: 2^{12} = ( ), 2^{16} = ( ), 300을 32비트 값으로 보고 16진 8자리로 쓰면 ( ).

   답 보기

더 자세히: 이론 노트 기초 · 1, 2, 3

기초 · 2 슬라이드 Ch2 p.18–20, 31 / Ch4 p.13

2의 보수와 부호 확장

시험에 나오는 모양: 음수를 2진·16진으로 쓰기, 0xFFFFFFFC 같은 값을 10진으로 읽기, 12비트 즉시값을 32비트로 부호 확장하기, 같은 비트를 부호 있는 수·무부호 수로 읽기.

먼저 알아 둘 말

- **명령어(instruction)**: CPU에게 시키는 일 하나. 예 "더해라", "메모리에서 읽어라". 하나가 32비트 기계어로 저장된다. [Ch2 p.2]
- **레지스터(register)**: CPU 안에 있는 32개의 저장 칸 x0~x31. 칸 하나에 32비트 값이 들어간다. 계산은 이 칸들 사이에서 한다. [Ch2 p.10]
- **즉시값(immediate)**: 명령어 안에 직접 적힌 상수. 기본 12비트라서 계산 전에 32비트로 늘려야 한다. 그 일을 하는 장치가 ImmGen(즉시값 생성기)이다. [Ch2 p.16, Ch4 p.13]

쉽게 말하면

2의 보수(two's complement)에서는 맨 왼쪽 비트(MSB, Most Significant Bit)의 자릿값만 음수다. 맨 오른쪽 비트는 LSB(Least Significant Bit)라 부른다. 8비트라면 맨 왼쪽 자리가 +128이 아니라 −128이다. 나머지 자리는 무부호와 똑같다.

왜 이렇게 되나

- **"보수(complement)"라는 말**: 어떤 수에 더해서 정해진 기준 수를 채워 주는 짝이다. 10진 예: 10을 기준으로 하면 7의 보수는 3이다(7 + 3 = 10). **2의 보수**는 n비트 수 x에 더해서 2^n(2진으로 1 뒤에 0이 n개)을 채우는 짝, 곧 **$2^n - x$**다. 기준이 2의 거듭제곱이라 "2의" 보수라 부른다. PDF 외 보충
- **왜 그 짝이 −x 역할을 하나**: 4비트 예. 3 = 0011의 2의 보수는 16 − 3 = 13 = 1101. 0011 + 1101 = 10000인데 칸이 4개뿐이라 맨 앞 1은 칸 밖으로 버려지고 0000이 남는다. 3과 더해 0이 되니 1101이 −3이다. 그래서 "n비트에서 −x는 $2^n - x$의 비트 패턴으로 저장한다"고 약속한 방식이 **2의 보수 표현**이다. 음수를 나타내는 방법은 부호-크기(sign and magnitude), 1의 보수, 2의 보수 세 가지가 있었고 최종적으로 2의 보수를 쓴다. [Ch2 p.19]
- **MSB가 -2^{n-1}이라는 규칙과 같은 말이다**: 1101을 MSB 규칙으로 읽으면 −8 + 4 + 1 = −3. 무부호로 읽으면 8 + 4 + 1 = 13. MSB 자리를 +8 대신 −8로 읽으면 값이 16(= 2^4) 작아지므로 13 − 16 = −3, 곧 $2^n - x$에서 2^n을 빼는 것과 같다.
- **왜 "뒤집고 1을 더하면" −x인가**: x와 ~x(모든 비트를 뒤집은 것)를 더하면 자리마다 0 + 1이나 1 + 0이라 올림 없이 전부 1, 곧 $2^n - 1$이 된다(1111_2 = 16 − 1 = 15처럼 1이 n개면 $2^n - 1$). 그래서 ~x = $2^n - 1 - x$이고, 1을 더하면 $2^n - x = -x$의 패턴이 된다. 4비트 3: ~0011 = 1100(= 12 = 15 − 3), 1을 더하면 1101. 뒤집기만 한 ~x는 $2^n - 1$(모두 1)에 대한 짝이라 1의 보수라고 부르고, 16진 한 자리씩 뒤집으면 F(1111)에서 빼는 것이라 **15 − (그 자리 숫자)**다. 16은 1을 더한 뒤에야 나오는 수다. [Ch2 p.20]
- **범위가 왜 $-2^{n-1} \sim 2^{n-1} - 1$인가**: 가장 작은 값은 음수 자리(MSB)만 1이고 양수 자리는 모두 0인 패턴 = -2^{n-1}. 가장 큰 값은 MSB가 0이고 나머지 n − 1개가 모두 1인 패턴 = $2^{n-1} - 1$. 4비트: 1000 = −8, 0111 = 7. 무부호는 n개가 모두 1인 $2^n - 1$이 최대다.
- **같은 비트, 두 가지 읽기(약속)**: 레지스터·메모리에 들어 있는 것은 0과 1뿐이고 "이 값은 음수"라는 표시는 따로 없다. 같은 비트를 무부호(모든 자리 양수)로 읽을지 부호 있는 수(MSB 자리만 음수)로 읽을지는 **그 값을 쓰는 명령어가 정한다**. 두 읽기의 차이는 MSB 자리 하나뿐이다: 무부호로는 $+2^{n-1}$, 부호 있는 수로는 -2^{n-1}이라 차이는 2^n이다. 그래서 MSB가 1이면 무부호 값 = 부호 있는 값 + 2^n, MSB가 0이면 두 값이 같다.
- **32비트에서 "2^{32}에서 얼마 모자라나" 구하기**: 2^{32} = 0x1 0000 0000(1 뒤에 0이 8개, 9자리)이고 0xFFFFFFFF는 그보다 딱 1 작다. 그래서 앞이 F로 채워진 값은 **(0xFFFFFFFF까지 남은 양) + 1**만큼 모자란다. 예 0xFFFFFFF0: 0xFFFFFFFF − 0xFFFFFFF0 = 0xF = 15, 여기에 1을 더해 16 → −16. 15에서 멈추면 −15로 틀린다(뒤집기만 하고 +1을 빼먹은 것과 같은 실수).

- **부호 확장이 왜 값을 지키나**: 4비트 1110 = −8 + 4 + 2 = −2. 8비트로 늘려 1111 1110이 되면 −128 + 64 + 32 + 16 + 8 + 4 + 2 = −2로 같다. 새 MSB(−128)와 1로 채운 자리(64, 32, 16)와 옛 MSB 자리(이제 +8)를 합하면 −128 + 64 + 32 + 16 + 8 = −8이라 옛 MSB의 −8과 똑같기 때문이다. 0으로 채웠다면 0000 1110 = 14가 되어 값이 바뀐다. 원래 4비트(1110)는 손대지 않는다. 슬라이드 예: −2 = 1111 1110 → 1111 1111 1111 1110. [Ch2 p.20]

외울 것

- **n비트 부호 있는 수 범위**: $-2^{n-1} \sim 2^{n-1} - 1$. 무부호 수(unsigned)는 $0 \sim 2^n - 1$. [Ch2 p.18–19]
- MSB가 부호 비트(sign bit): 1이면 음수, 0이면 양수. [Ch2 p.19]
- **부정(= 부호 반전, x → −x)**: ~x + 1 = −x (모든 비트를 뒤집고 1을 더함). [Ch2 p.20]
- **음수 읽기**: 부호 비트가 1이면 -2^{n-1} + (나머지 비트의 값). 또는 뒤집고 1을 더해 크기를 구한 뒤 − 를 붙인다.
- **부호 확장(sign extension)**: 비트 수를 늘릴 때 부호 비트를 왼쪽 빈칸에 복사한다. 값이 그대로 유지된다. [Ch2 p.20]
- ImmGen(즉시값 생성기)은 명령어의 12비트 즉시값을 32비트로 부호 확장한다. [Ch4 p.13]

비트 수	부호 있는 범위	무부호 범위
4	−8 ~ 7	0 ~ 15
8	−128 ~ 127	0 ~ 255
12	−2048 ~ 2047	0 ~ 4095
32	$-2^{31} \sim 2^{31} - 1$	$0 \sim 2^{32} - 1$

32비트 16진	부호 있는 값	무부호 값
0xFFFFFFFF	−1	$2^{32} - 1$
0xFFFFFFFE	−2	$2^{32} - 2$
0xFFFFFFFC	−4	$2^{32} - 4$
0x80000000	-2^{31}	2^{31}
0x7FFFFFFF	$2^{31} - 1$	$2^{31} - 1$

일반형: n비트에서 −a의 비트 패턴을 무부호로 읽으면 $2^n - a$다. 8비트 −37 = 256 − 37 = 219 = 0xDB.

거꾸로 32비트 0xFFFFFFFC는 2^{32}에서 4 모자라므로 −4.

푸는 순서

1. 비트 수 n을 확인한다(8, 12, 32 ...). 16진이면 2진으로 펼친다.
2. MSB가 0이면 무부호처럼 읽고, 1이면 위 "음수 읽기"대로 읽는다.
3. 부호 확장이면 늘어난 왼쪽 칸을 전부 MSB 값으로 채운다(16진으로는 F 또는 0).

함정

- 뒤집기만 하고 +1을 빼먹는다. ~x는 −x − 1이다. 0xFFFFFFF0을 −15로 읽는 것이 이 실수다: 뒤집으면 0x0000000F = 15, 여기에 1을 더한 16이 크기라 −16이다.
- 자리마다 16 − (숫자)를 한다. 16은 덧셈 넘침·뺄셈 빌림에만 쓴다. 뒤집기는 15 − (숫자) 후 끝에 +1 한 번이고, 헷갈리면 크기는 **다음 큰 수에서 한 번에 빼기**로 구한다: 0xFE8의 크기 = 0x1000 − 0xFE8 = 0x18 = 24(자리마다 빼면 0x128 = 296이 되어 틀림).
- 부호 확장하면서 원래 숫자를 바꾼다. 부호 확장은 **옮겨 적기**라 원래 자리는 그대로 두고 왼쪽 빈칸만 MSB로 채운다: 0xFE8 → 0xFFFFFFE8. MSB가 1인데 0으로 채우면 큰 양수가 된다.
- 같은 비트도 읽는 방식에 따라 값이 다르다. 두 레지스터를 비교해 "작으면 다른 줄로 건너뛰는" 명령어가 둘 있는데, blt는 부호 있는 수로, bltu는 무부호 수로 읽고 비교한다. 그래서 0xFFFFFFFF는 blt에서 −1, bltu에서 2^{32} − 1이다(자세히는 Ch2 · 2). [Ch2 p.31]

예제 (시험 수준)

−6을 32비트 2의 보수로 나타내 16진 8자리로 쓰고, 결과를 다시 10진으로 읽어 확인하시오.

풀이 보기

회독 점검

1. −37을 8비트 2의 보수로 2진과 16진으로 쓰시오.

   답 보기

2. 8비트 2의 보수 1010 0110을 10진으로 쓰시오.

   답 보기

3. 32비트 값 0xFFFFFFF0을 부호 있는 수로 읽으면? 무부호 수로 읽으면?

   답 보기

4. 12비트 즉시값 1111 1110 1000을 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

   답 보기

5. 8비트 패턴 1100 1000을 부호 있는 수(2의 보수)로 읽으면? 무부호 수로 읽으면?

답 보기

---

6. 빈칸: 12비트 2의 보수가 나타낼 수 있는 범위는 ( ) ~ ( )이고, 32비트 부호 있는 수의 최솟값은 16진으로 ( )이다.

답 보기

더 자세히: 이론 노트 기초 · 3 / Ch2 · 4, 7 / Ch4 · 3

기초 · 3 슬라이드 Ch2 p.25–27, 30

비트 연산과 시프트

시험에 나오는 모양: and·or·xor 결과를 16진으로 쓰기, sll·srl·sra 결과 쓰기, [19:15] 같은 비트 범위 읽기, slli 2가 왜 ×4인지(배열 인덱스).

먼저: 어셈블리 한 줄 읽는 법

이 단원 예제와 점검에는 `and x9, x10, x11` 같은 줄이 나온다. 읽는 규칙은 세 가지다(자세한 표는 기초 · 4).

1. **맨 앞 단어 = 할 일(명령어 이름)**. `and` = AND 하라, `srli` = 오른쪽으로 밀어라(shift right logical immediate).
2. **쉼표 뒤 첫 번째 = 결과를 넣을 칸**, 그 뒤 = 계산에 쓸 값. `and x9, x10, x11` = "x10과 x11을 AND해서 x9에 넣어라", 즉 x9 ← x10 and x11. x10과 x11은 읽기만 하므로 값이 그대로다. [Ch2 p.9]
3. **이름 끝에 i가 붙으면 마지막 값이 숫자 상수**(즉시값)다. `srli x9, x9, 8` = "x9를 오른쪽으로 8칸 밀어서 다시 x9에 넣어라", 즉 x9 ← x9 >> 8. 결과 칸과 읽는 칸이 같으면 자기 값을 바꾼 것이다.

"실행했다"는 맨 앞 칸(결과 칸)의 값이 바뀌었다는 뜻이다. 문제가 묻는 것은 보통 그 칸의 새 값이다.

외울 것

a	b	and	or	xor
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

세 연산의 뜻(위 표는 이 뜻을 네 경우에 적용한 것):

- **and(그리고)**: 두 비트가 **둘 다 1일 때만** 1. 하나라도 0이면 0이다. 그래서 0 and 0 = 0이다(1이 하나도 없으니 "둘 다 1"이 아님). 스위치 두 개를 일렬로 이은 전등과 같다: 둘 다 켜야 불이 들어온다.
- **or(또는)**: 두 비트 중 **하나라도 1이면** 1. 둘 다 0일 때만 0이다.
- **xor(배타적 또는, exclusive or)**: 두 비트가 **서로 다를 때만** 1. 같으면(0과 0, 1과 1) 0이다.
- 여러 비트짜리 값끼리 할 때는 **같은 번호의 비트끼리 따로따로** 위 규칙을 적용한다. 예 1100 and 1010: 자리마다 1·1 → 1, 1·0 → 0, 0·1 → 0, 0·0 → 0 → 1000.
- **and = 마스크(mask)**: 마스크 1인 자리만 남기고 나머지는 0으로 지운다. [Ch2 p.26]
- **or = 1로 설정**: 마스크 1인 자리를 1로 켜고 나머지는 그대로. [Ch2 p.26]
- **xor = 뒤집기**: 같으면 0, 다르면 1. 마스크 1인 자리만 뒤집힌다. 모두 1인 값과 xor하면 NOT이 된다(RISC-V에는 NOT 명령이 없음). [Ch2 p.27]
- 비트 연산은 자리끼리만 계산하고 올림이 없다. 그래서 **16진 자리 단위**로 계산해도 된다. 16진 F = 1111(네 비트 모두 1), 0 = 0000(모두 0)이라 아래 표가 나온다. x는 16진 한 자리(네 비트)다.

16진 자리 계산	결과	비트 단위 이유 (b = x의 비트 하나)	예
x and F	x 그대로	b and 1 = b (b가 1이면 둘 다 1이라 1, b가 0이면 0). 네 비트 모두 1과 만나니 x가 그대로 남는다	B and F = 1011 and 1111 = 1011 = B
x and 0	0	b and 0 = 0 (한쪽이 0이면 "둘 다 1"이 될 수 없음)	B and 0 = 0000 = 0
x or F	F	b or 1 = 1 (한쪽이 이미 1)	B or F = 1111 = F
x or 0	x 그대로	b or 0 = b (0은 1을 보태지 않음)	B or 0 = 1011 = B
x xor F	15 − x	b xor 1 = b의 반대(다르면 1). 네 비트가 모두 뒤집히고, x와 뒤집힌 x를 더하면 1111 = 15이므로 뒤집힌 값 = 15 − x	B xor F = 0100 = 4 = 15 − 11
x xor 0	x 그대로	b xor 0 = b (b가 0이면 같아서 0, 1이면 달라서 1)	B xor 0 = 1011 = B

마스크 자리가 0도 F도 아니면(예 C and 6) 두 숫자를 2진으로 펼쳐 비트마다 계산한다: 1100 and 0110 = 0100 = 4.

- **비트 번호**: 맨 오른쪽(LSB)이 0번, 32비트면 맨 왼쪽(MSB)이 31번. **[hi:lo]** = hi번~lo번 비트(슬라이드의 [19-15]와 같은 뜻). [19:15]는 19, 18, 17, 16, 15번의 5개. imm[12:1]은 즉시값의 12번~1번 비트(0번은 칸에 없음).
- **레지스터 번호**: 레지스터 32개(기초 · 2)는 x0, x1, ..., x31처럼 **x 뒤의 숫자가 번호**다. `add x9, x20, x21`은 9번, 20번, 21번 레지스터를 쓴다는 뜻이다. [Ch2 p.10]
- CPU가 실제로 읽는 32비트 기계어에는 "x20" 같은 글자를 쓸 수 없어서 **번호만 2진수로** 적는다. 번호는 0~31이고 2^5 = 32이므로 레지스터 하나에 **5비트 칸**이면 충분하다. 계산에 읽는 첫 번째 레지스터 칸을 **rs1**, 두 번째를 **rs2**, 결과를 넣을 칸을 **rd**라 부른다(형식은 Ch2 · 4). [Ch2 p.22]

예: `add x9, x20, x21`의 기계어 0x015A04B3을 칸으로 나누면 아래와 같다. 각 레지스터 칸의 5비트를 10진으로 바꾸면 그대로 레지스터 번호가 나온다. [Ch2 p.22]

칸	비트 위치	들어 있는 2진	10진	뜻
rs2	[24:20]	10101_2	16 + 4 + 1 = 21	두 번째로 읽는 레지스터 = x21
rs1	**[19:15]**	10100_2	16 + 4 = 20	첫 번째로 읽는 레지스터 = **x20**
rd	[11:7]	01001_2	8 + 1 = 9	결과를 넣는 레지스터 = x9

그래서 "[19:15]가 rs1 칸이라면 몇 번 레지스터인가"는 **그 5비트를 10진으로 바꿔 앞에 x를 붙이라**는 뜻이다.

시프트	방향	빈칸 채우기	값으로 보면
sll, slli	왼쪽	오른쪽에 0	k칸 = × 2^k(넘친 비트는 버림)
srl, srli	오른쪽	왼쪽에 0	무부호 ÷ 2^k(몫)
sra, srai	오른쪽	왼쪽에 부호 비트(MSB) 복사	부호 있는 ÷ 2^k(내림)

슬라이드 예: 1100 0011을 sra 1칸 → 1110 0001. [Ch2 p.25] 4칸 시프트 = 16진 한 자리 이동(16진 한 자리가 4비트이므로).

왜 왼쪽 k칸이 × 2^k인가: 왼쪽으로 한 칸 밀면 모든 1이 자릿값이 두 배인 자리로 옮겨 가므로 합도 두 배가 된다. 10진에서 오른쪽에 0을 하나 붙이면(왼쪽으로 한 칸) × 10이 되는 것과 같다. 예 0011(3) → 0110(6) → 1100(12). 두 칸이면 × 2 × 2 = × 4. 오른쪽으로 밀면 반대로 자릿값이 절반이 되고, 칸 밖으로 버려지는 오른쪽 비트가 나머지라 ÷ 2^k의 몫만 남는다. 예 1101(13) srl 1 → 0110(6) = 13 ÷ 2의 몫. sra가 빈칸을 MSB로 채우는 것은 부호 확장(기초 · 2)과 같은 이유다: 음수를 밀어도 음수로 남게 하려는 것이다.

시프트가 가장 많이 쓰이는 곳은 배열 주소 계산이다. 메모리(memory)는 바이트마다 0부터 번호가 붙은 긴 칸 줄이고, 그 번호가 **주소(address)**다. [Ch2 p.12] 배열(array)은 같은 종류의 값을 메모리에 차례로 붙여 놓은 것이고, A[i]는 배열 A의 i번 원소(0번부터 셈)다. int(정수) 하나는 4바이트라 A[i]는 배열 시작 주소에서 i × 4바이트 떨어져 있다. 이 거리(바이트 수)를 **오프셋(offset)**이라 한다. i × 4는 `slli x10, x22, 2` (x22 = i를 왼쪽으로 2칸)로 구한다. [Ch2 p.30]

푸는 순서

1. 두 값을 같은 자리 수(32비트면 8자리)로 맞춰 위아래로 놓는다.
2. 마스크 자리가 0이나 F면 위 요령으로 바로 쓴다. 섞인 자리만 2진으로 펼쳐 진리표를 적용한다.
3. 시프트는 2진으로 펼쳐 옮기고, 빈칸을 0(sll, srl) 또는 MSB(sra)로 채운 뒤 다시 16진으로 묶는다.

유형별 풀이 예 (점검 문제와 같은 모양, 숫자만 다름)

① NOT 만들기 (xor로 모든 비트 뒤집기)

NOT은 "모든 비트를 반대로 바꾸기"(0 → 1, 1 → 0)다. RISC-V에는 not 명령이 없어서, **모든 비트가 1인 값과 xor**해서 만든다. 이유: b xor 1은 "b와 1이 다르면 1"이라 b가 0이면 1, b가 1이면 0, 즉 b의 반대가 된다. [Ch2 p.27]

예: 8비트 0xA5를 0xFF와 xor. 0xA5 = 1010 0101, 0xFF = 1111 1111. 자리마다 뒤집으면 0101 1010 = **0x5A**. 16진 자리로 해도 같다: 15 − A(10) = 5, 15 − 5 = 10 = A → 0x5A. 이 결과를 "NOT(모든 비트 반전)과 같다"고 답한다.

② 여러 칸 시프트 (8비트, 2진)

k칸 시프트는 세 단계로 한다. (가) 미는 방향의 끝에서 k개 비트를 버린다. (나) 남은 비트를 순서 그대로 반대쪽 끝까지 붙인다. (다) 비어 있는 k칸을 **모두 같은 값**으로 채운다: sll·srl은 0, sra는 원래 MSB.

1001 0110을 2칸	(가) 버리는 비트	(나) 남은 비트	(다) 빈칸 2개	결과
sll 2 (왼쪽)	왼쪽 끝 10	01 0110 → 왼쪽으로 붙임	오른쪽에 00	**0101 1000**
srl 2 (오른쪽)	오른쪽 끝 10	1001 01 → 오른쪽으로 붙임	왼쪽에 00	**0010 0101**
sra 2 (오른쪽)	오른쪽 끝 10	1001 01 → 오른쪽으로 붙임	원래 MSB가 1 → 왼쪽에 11	**1110 0101**

srl과 sra는 (다)만 다르다. 원래 MSB가 0이면 둘의 결과가 똑같다.

③ 32비트 16진 값을 4칸 시프트

16진 한 자리 = 4비트이므로 **4칸 시프트 = 16진 숫자 한 자리 이동**이다. 2진으로 펼칠 필요가 없다.

- srli 4: 맨 오른쪽 16진 숫자 하나를 버리고 나머지를 한 자리 오른쪽으로, 맨 왼쪽에 **0**. 0x80000120 → **0x08000012**.
- srai 4: 같지만 맨 왼쪽 새 자리를 MSB 4개로 채운다. MSB는 맨 앞 16진 숫자의 첫 비트다(8 = **1**000). 1이 4개면 1111 = **F**, 0이 4개면 0. 0x80000120 → **0xF8000012**. 부호 확장(기초 · 2)에서 F로 채우던 것과 같은 원리다.
- slli 4: 맨 왼쪽 숫자 하나를 버리고 한 자리 왼쪽으로, 맨 오른쪽에 0. 0x00000120 → 0x00001200.

④ slli로 배열 오프셋 구하기

x5 = 7(인덱스 i)일 때 `slli x6, x5, 2`. 먼저 읽기: "x5를 왼쪽으로 2칸 밀어 x6에 넣어라". 왼쪽 2칸 = × 2^2 = × 4이므로 x6 = 7 × 4 = **28**(2진으로 0111 → 1 1100 = 16 + 8 + 4 = 28). 뜻: int는 4바이트라 A[7]은 배열 시작 주소에서 28바이트 떨어져 있다. 즉 28은 **A[7]의 오프셋(시작 주소에서 A[7]까지의 바이트 수)**이고, 시작 주소 + 28이 A[7]의 주소다. [Ch2 p.30]

함정

- 0 and 0 = 1이 아니다. and는 "둘 다 1일 때만 1"이라 0 and 0 = 0이다. "둘이 같으면 1"이라는 규칙은 위 세 연산 어디에도 없다(xor는 반대로 "다르면 1").
- and를 "작은 쪽 고르기"로 외우면 틀린다. F and 3 = 3인 이유는 F = 1111이 3 = 0011의 비트를 하나도 지우지 않기 때문이다. 6 and 3 = 0110 and 0011 = 0010 = 2(6도 3도 아님), 5 and A = 0101 and 1010 = 0000 = 0이다. 항상 비트로 펼쳐 확인한다.
- 음수(MSB 1)를 srl하면 왼쪽에 0이 들어와 큰 양수가 된다. 부호를 지키는 것은 sra뿐이다.
- 왼쪽 시프트로 1이 칸 밖으로 넘치면 $\times 2^k$가 성립하지 않는다(8비트 1100 0000을 sll 1 → 1000 0000).
- sra는 음수 홀수에서 내림이다: −7 sra 1 = −4(1111 1001 → 1111 1100). 부호 있는 나눗셈 몫 −7 ÷ 2 = −3과 다르다. PDF 외 보충

예제 (시험 수준)

x10 = 0x12345678이다. x11 = 0x0000FF00일 때 `and x9, x10, x11`을 실행한 뒤 `srli x9, x9, 8`을 실행했다. 각 단계의 x9를 16진 8자리로 쓰시오.

풀이 보기

회독 점검

1. 0x0000ABCD AND 0x00000FF0을 16진 8자리로 쓰시오.

   답 보기

   ---

2. 32비트 값 0x000A4000의 [19:15] 비트 5개를 2진으로 쓰고, 이 5비트가 rs1 칸이라면 몇 번 레지스터인지 쓰시오.

   답 보기

   ---

3. 8비트 값 0xC3을 0xFF와 xor한 결과를 16진과 2진으로 쓰고, 이 연산이 무엇과 같은지 쓰시오.

   답 보기

   ---

4. 8비트 1011 0100을 2칸 sll, srl, sra 한 결과를 각각 2진으로 쓰시오.

   답 보기

   ---

5. 32비트 0xF0000040을 4칸 srai 한 결과와 4칸 srli 한 결과를 16진 8자리로 쓰시오.

   답 보기

---

6. x22 = 9(인덱스 i)일 때 `slli x10, x22, 2` 실행 뒤 x10은? 이 값은 int 배열에서 무엇인가?

   답 보기

---

7. (다시 풀기, 3번 유형) 8비트 값 0x6E를 0xFF와 xor한 결과를 16진과 2진으로 쓰고, 이 연산이 무엇과 같은지 쓰시오.

   답 보기

---

8. (다시 풀기, 4번 유형) 8비트 1010 1101을 3칸 sll, srl, sra 한 결과를 각각 2진으로 쓰시오.

   답 보기

---

9. (다시 풀기, 5번 유형) 32비트 0xC00000F0을 8칸 srai 한 결과와 8칸 srli 한 결과를 16진 8자리로 쓰시오.

   답 보기

---

10. (다시 풀기, 6번 유형) x7 = 12(인덱스 i)일 때 `slli x8, x7, 2` 실행 뒤 x8은? int 배열 A의 시작 주소가 0x1000이면 A[12]의 주소는?

    답 보기

더 자세히: 이론 노트 기초 · 4 / Ch2 · 6, 7

기초 · 4 슬라이드 Ch1 p.26 / Ch2 p.9–14, 30, 35

단위와 어셈블리 읽는 법

시험에 나오는 모양: GHz ↔ ps 환산과 10의 거듭제곱 계산(성능 공식), 어셈블리 한 줄을 C 문장으로 읽기, 배열 원소 주소 계산.

먼저 알아 둘 말

- **클럭(clock)**: CPU가 일정한 박자에 맞춰 일하게 하는 신호. 박자 한 번이 **클럭 사이클(clock cycle)**, 그 한 번의 길이(시간)가 **클럭 주기(clock period)**, 1 s에 박자가 몇 번인지가 **클럭 속도(clock rate)**다. 속도 단위 Hz = 1 s에 1번. [Ch1 p.26]
- **C 문장**: 시험의 "C로 쓰시오"는 `A[7] = A[5] - 1;` 같은 한 줄을 말한다. `=` 는 "오른쪽을 계산해 왼쪽에 넣어라", `A[5]` 는 배열 A의 5번 원소(기초 · 3), 끝의 `;` 는 문장의 끝 표시다. PDF 외 보충

외울 것

큰 쪽	값	작은 쪽	값
k(킬로)	10^3	m(밀리)	10^{-3}
M(메가)	10^6	μ(마이크로)	10^{-6}
G(기가)	10^9	n(나노)	10^{-9}
		p(피코)	10^{-12}

- **음수 지수와 0 지수**: $10^{-k} = 1 / 10^k$(예 $10^{-3} = 1 / 1000 = 0.001$), $10^0 = 1$. 기초 · 1의 $2^{-1} = 0.5$와 같은 규칙이다. 그래서 1 ns = 10^{-9} s = 1000 × 10^{-12} s = **1000 ps**, 1 s = 1000 ms다.
- **클럭 주기 = 1 / 클럭 속도**. 4 GHz → 1 / (4 × 10^9) s = 0.25 ns = 250 ps. [Ch1 p.26] 이유: 1 s에 박자가 f번이면 1 s를 f조각으로 나눈 것이 박자 하나의 길이다(1 s에 4번이면 한 번에 1/4 s).
- **지나간 클럭 사이클 수 = 시간 × 클럭 속도**. 1 s에 f번 박자가 오면 t s 동안에는 f × t번 온다(1 s에 10번이면 3 s에 30번). 같은 말로 시간 ÷ 클럭 주기다.
- 빠른 환산: 주기(ps) = 1000 / 속도(GHz 숫자). 2 GHz → 500 ps, 500 ps → 1000 / 500 = 2 GHz. 이유: 속도가 f GHz면 주기 = 1 / (f × 10^9) s = (1 / f) × 10^{-9} s = (1 / f) ns이고, 1 ns = 1000 ps라 (1000 / f) ps다.
- 지수 계산: $10^a \times 10^b = 10^{a+b}$, $10^a / 10^b = 10^{a-b}$. 앞의 숫자끼리 따로, 지수끼리 따로 계산한다. a, b가 음수여도 같다: $10^{-3} \times 10^9 = 10^6$, $1 / 10^{-12} = 10^{0-(-12)} = 10^{12}$.
- 한국어 큰 수: 1만 = 10^4, 1억 = 10^8. 예 6억 = 6 × 10^8, 600만 = 6 × 10^6.
- 바이트 = 8비트, **word = 32비트 = 4바이트**, doubleword = 64비트 = 8바이트. [Ch2 p.10] KiB(2^{10} = 1024)와 KB(10^3)는 섞지 않는다.

어셈블리	뜻
`add x5, x6, x7`	x5 ← x6 + x7. 꼴 `add rd, rs1, rs2`에서 **맨 앞 rd = 결과를 받는 레지스터(destination)**, rs1·rs2 = 값을 읽는 레지스터(source). [Ch2 p.9]
`sub x5, x6, x7`	x5 ← x6 − x7 (순서 그대로 뺀다)
`addi x5, x6, -3`	x5 ← x6 + (−3). 즉시값(immediate)은 상수
`lw x9, 32(x22)`	주소 x22 + 32에서 word를 읽어 x9에 넣는다. [Ch2 p.14]
`sw x9, 48(x22)`	x9를 주소 x22 + 48에 쓴다(x9는 그대로)
`beq x5, x6, L`	x5 == x6이면 L로 건너뛰고, 아니면 다음 줄. 이렇게 조건에 따라 다른 줄로 건너뛰는 것을 **분기(branch)**라 한다. [Ch2 p.28] bne = 같지 않으면, blt = 작으면, bge = 크거나 같으면 L로
`Loop:`	라벨(label). 명령어 위치의 이름, 분기 목적지로 쓴다
`// …`	주석. 실행되지 않는다

- **PC(Program Counter)**: 실행할 명령어의 주소를 담는 레지스터. 명령어 하나가 4바이트라 보통 다음은 PC + 4. [Ch2 p.35]
- **배열 주소**: int 하나 = word = 4바이트라 int A[i]의 주소 = 시작 주소 + i × 4. `lw`·`sw`의 괄호 앞 숫자(32, 48)가 이 i × 4, 즉 오프셋이다. 연속된 word의 주소는 4씩 차이 난다. [Ch2 p.12, 14]

푸는 순서

1. 단위 문제: 모든 값을 10의 거듭제곱 꼴로 바꾼 뒤 숫자와 지수를 따로 계산하고, 마지막에 원하는 단위로 바꾼다.
2. 어셈블리: rd(맨 앞)를 먼저 표시하고 오른쪽을 계산한다.
3. lw·sw: 괄호 안 레지스터 값 + 앞의 숫자 = 주소. 배열이면 (앞의 숫자) ÷ 4 = 인덱스.

함정

- `lw x9, 32(x22)`는 A[32]가 아니라 A[8]이다. 32는 바이트 수라 4로 나눠야 인덱스가 된다.
- sub의 소스 순서를 바꾸면 부호가 바뀐다. `sub x5, x7, x6`은 x7 − x6이다.
- GHz는 1 s에 10^9번, 1 ns = 10^{-9} s. 주기와 속도는 역수라 한쪽이 2배면 다른 쪽은 절반이다.

예제 (시험 수준)

int 배열 A의 시작 주소는 x23에 있고 x23 = 0x00000200이다. 아래 코드가 읽고 쓰는 주소를 쓰고, C 문장 하나로 쓰시오.

```
lw   x9, 20(x23)     // ?
addi x9, x9, -1
sw   x9, 28(x23)
```

풀이 보기

회독 점검

1. 클럭 속도 2.5 GHz인 CPU의 클럭 주기는 몇 ps인가?

   답 보기

---

2. 클럭 주기가 800 ps인 CPU의 클럭 속도는 몇 GHz인가?

   답 보기

---

3. 클럭 속도 2 GHz인 CPU에서 3 ms 동안 지나가는 클럭 사이클 수를 10의 거듭제곱 꼴과 "만" 단위로 쓰시오.

   답 보기

---

4. x6 = 10, x7 = 4일 때 `sub x5, x7, x6` 실행 뒤 x5는? 이 명령을 C 문장 꼴로 쓰시오.

   답 보기

---

5. x22 = 0x00001000이고 x22가 int 배열 A의 시작 주소다. `lw x9, 24(x22)`는 몇 번지를 읽고, C로는 무엇인가?

   답 보기

---

6. int 배열 A의 시작 주소가 0x00002000이다. A[10]의 주소를 16진 8자리로 쓰시오. 주소 0x00002034에 있는 원소는 A의 몇 번 원소인가?

   답 보기

더 자세히: 이론 노트 기초 · 5, 7 / Ch1 · 8 / Ch2 · 3

기초 문제 100

기초 1~4만 알면 풀리는 연습 문제 100개입니다. 파트마다 25문제이고, 앞쪽은 규칙 하나를 바로 쓰는 문제, 뒤쪽은 시험 수준입니다. 답을 펼치기 전에 연습장에 먼저 풀고 [맞음]/[틀림]을 누르세요. 틀린 문제는 오답 노트에 모입니다.

푸는 법

- 한 번에 다 풀지 말고 파트 하나(25문제, 약 1시간)씩 끊어서 풉니다.
- 틀리면 해설의 단계 중 **어느 번호에서** 어긋났는지 확인하고, 그 규칙을 기초 1~4에서 다시 읽습니다.
- 같은 유형이 숫자를 바꿔 여러 번 나옵니다. 앞에서 틀린 유형이 뒤에서 다시 나오면 그때 맞히는 것이 목표입니다.

파트 1 · 진법 변환과 16진수 (1~25번)

근거: 기초 · 1. 막히면 기초 · 1의 외울 것과 유형별 예를 다시 보고 오세요.

문제 1~25

1. 10진수 45를 2진수로 쓰시오. 2로 나누기를 쓰시오.

   답 보기

2. 1100101_2를 10진수로 쓰시오.

   답 보기

3. $1011\ 1110\ 0101_2$(12비트)를 16진수로 쓰시오.

   답 보기

4. 0x7C를 8비트 2진수로 쓰시오.

   답 보기

5. 10진 소수 0.6875를 2진수로 쓰시오.

   답 보기

6. 16진 덧셈 0x4B + 0x37을 계산하시오.

   답 보기

---

7. 빈칸: 2^{11} = ( ), 2^{14} = ( ), 2^{-3} = ( ).

   답 보기

---

8. 0x1F4를 10진수로 쓰시오.

   답 보기

---

9. 10진수 150을 2로 나누기로 2진수로 바꾸고, 그 결과를 16진수로 쓰시오.

   답 보기

---

10. 16진 뺄셈 0x72B − 0x3E5를 계산하시오.

    답 보기

---

11. 22.8125를 2진수로 쓰시오.

    답 보기

---

12. 10011.101_2를 10진수로 쓰시오.

    답 보기

---

13. 2진 덧셈 0101 1011_2 + 0011 0110_2를 8비트 2진수로 쓰시오.

    답 보기

---

14. 10진수 1000을 32비트 값으로 보고 16진 8자리로 쓰시오. 16으로 나누기를 쓰시오.

    답 보기

---

15. 16진 덧셈 0x5E8 + 0x2B9를 계산하시오.

    답 보기

---

16. 16비트 값 0x0A50에서 1인 비트의 번호를 모두 쓰시오(맨 오른쪽 비트를 0번으로 센다). 그 값을 10진수로도 쓰시오.

답 보기

---

17. 9.5625를 2진수로 쓰시오.

답 보기

---

18. 빈칸: 2^{20} = ( ), 2^{24} = ( ). 외운 2^{10}, 2^{8}, 2^{16}을 써서 계산하시오.

답 보기

---

19. 43.40625를 2진수로 쓰시오.

답 보기

---

20. 16진 뺄셈 0xC14 − 0x6A7을 계산하시오.

답 보기

---

21. 32비트 값 0x00012F40을 부호 없는 10진수로 쓰시오.

답 보기

---

22. 2진 덧셈 1110 1101_2 + 0011 0111_2의 결과를 2진수와 16진수로 쓰시오(자리 수 제한 없음).

답 보기

---

23. 32비트 16진 덧셈 0x0000FFF8 + 0x00000019를 16진 8자리로 쓰시오.

답 보기

---

24. 10진수 1234를 32비트 값으로 보고 16진 8자리로 쓰시오. 그 값의 2진수에서 1인 가장 높은 비트 번호(맨 오른쪽 = 0번)도 쓰시오.

답 보기

---

25. 세 수 0x9C, 1001 1011.1_2, 155.75를 10진수로 바꾸어 큰 것부터 순서대로 쓰시오.

답 보기

파트 2 · 2의 보수와 부호 확장 (26~50번)

근거: 기초 · 2. 막히면 기초 · 2의 외울 것과 회독 점검을 다시 보고 오세요.

문제 26~50

26. −12를 8비트 2의 보수로 나타내 2진과 16진 2자리로 쓰시오.

    답 보기

---

27. 8비트 2의 보수 1011 0101을 10진으로 쓰시오. (MSB 자릿값을 음수로 읽는 방법으로)

    답 보기

---

28. 빈칸: 16비트 부호 있는 수(2의 보수)의 범위는 ( ) ~ ( )이고, 16비트 무부호 수의 범위는 0 ~ ( )이다.

    답 보기

---

29. 8비트 2의 보수 0xE9를 10진으로 쓰시오. (뒤집고 1 더하는 방법으로)

    답 보기

---

30. 32비트 값 0xFFFFFFFD를 부호 있는 수로 읽으면 10진으로 얼마인가?

    답 보기

---

31. 4비트 2의 보수 1011을 8비트로 부호 확장하시오. 확장 전과 후의 10진 값도 쓰시오.

    답 보기

---

32. 8비트 패턴 0x9C를 무부호 수로 읽으면? 부호 있는 수(2의 보수)로 읽으면?

    답 보기

---

33. 12비트 즉시값 0x07B를 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

    답 보기

---

34. −45를 8비트 2의 보수로 나타내 16진 2자리로 쓰고, 같은 값을 32비트 16진 8자리로도 쓰시오. (45는 2로 나누기로 2진 변환)

답 보기

---

35. 32비트 값 0xFFFFFF80을 부호 있는 수로 읽으면 10진으로 얼마인가?

답 보기

---

36. 12비트 즉시값 0xF3A를 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

답 보기

---

37. −200을 32비트 2의 보수로 나타내 16진 8자리로 쓰시오. (200은 ÷16으로 16진 변환)

답 보기

---

38. 32비트 값 0xFFFFFF00을 부호 있는 수로 읽으면? 무부호 수로 읽으면?

답 보기

---

39. 8비트에서 x = 0x3A의 2의 보수를 $2^8 - x$로 구해 16진 2자리로 쓰고, x와 더해 8비트 결과가 0이 됨을 16진 덧셈으로 보이시오.

답 보기

---

40. 12비트 즉시값 0x800을 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

답 보기

---

41. 8비트 2의 보수 1100 0011을 10진으로 쓰시오. (뒤집고 1 더하는 방법으로)

답 보기

---

42. 12비트 즉시값 0xFFF와 0x7FF를 각각 32비트로 부호 확장해 16진 8자리와 10진 값을 쓰시오.

답 보기

---

43. 32비트 값 0xFFFFFFE0을 부호 있는 수로 읽으면 10진으로 얼마인가?

답 보기

---

44. −1000을 16비트 2의 보수로 나타내 16진 4자리로 쓰고, 그 값을 32비트로 부호 확장해 16진 8자리로 쓰시오.

    답 보기

---

45. 32비트 값 0xFFFFF000을 부호 있는 수로 읽으면? 무부호 수로 읽으면?

    답 보기

---

46. 12비트 즉시값 1001 0110 0100을 32비트로 부호 확장해 16진 8자리로 쓰고, 10진 값을 쓰시오.

    답 보기

---

47. 다음 중 8비트 부호 있는 수(2의 보수)로 나타낼 수 없는 것을 모두 고르고, 그중 8비트 무부호 수로는 나타낼 수 있는 것을 고르시오: −128, 128, −129, 127, 200

    답 보기

---

48. 8비트 값 0xB4에 대해 (가) 무부호로 읽은 값, (나) 부호 있는 수로 읽은 값, (다) 32비트로 부호 확장한 16진 8자리, (라) 32비트로 왼쪽을 0으로 채운 16진 8자리와 그 10진 값을 쓰시오.

    답 보기

---

49. 32비트 x = 500일 때 −x를 16진 8자리로 쓰고, x + (−x)를 16진 덧셈으로 계산해 32비트 결과가 0이 됨을 보이시오.

    답 보기

---

50. 12비트 즉시값 0xFA0에 대해 학생이 "32비트 부호 확장 = 0x00000FA0, 10진 값 = −95"라고 답했다. 틀린 곳 두 군데와 이유를 쓰고 바른 답을 쓰시오.

    답 보기

파트 3 · 비트 연산과 시프트 (51~75번)

근거: 기초 · 3. 막히면 기초 · 3의 "먼저: 어셈블리 한 줄 읽는 법", 외울 것, 유형별 풀이 예를 다시 보고 오세요.

문제 51~75

51. 4비트 값 1001_2와 0101_2의 and, or, xor 결과를 각각 4비트 2진으로 쓰시오.

    답 보기

---

52. 32비트 값 0x00007E3B AND 0x000000FF를 16진 8자리로 쓰시오.

    답 보기

---

53. 8비트 값 0x9C를 0xFF와 xor한 결과를 16진과 2진으로 쓰고, 이 연산이 무엇과 같은지 쓰시오.

    답 보기

---

54. 8비트 무부호 값 0110 0101을 왼쪽으로 1칸 시프트(sll 1)한 결과를 2진으로 쓰고, 시프트 전후 값을 10진으로 쓰시오.

    답 보기

---

55. 16진 숫자 한 자리끼리의 and: A and 7, 9 and 6을 각각 16진 한 자리로 쓰시오.

    답 보기

---

56. 8비트 값 0xB6의 [5:2] 비트 4개를 2진으로 쓰시오.

    답 보기

---

57. 8비트 무부호 값 1110 0100을 오른쪽으로 2칸 논리 시프트(srl 2)한 결과를 2진으로 쓰고, 결과를 10진으로 쓰시오.

    답 보기

---

58. x5 = 0x00000A3C, x6 = 0x0000F000이다. `or x7, x5, x6`을 실행한 뒤 x7을 16진 8자리로 쓰고, 이 줄에서 값이 바뀌는 레지스터를 모두 쓰시오.

    답 보기

---

59. 8비트 1100 1110을 3칸 sll, srl, sra 한 결과를 각각 2진으로 쓰시오.

    답 보기

---

60. 32비트 값 0x9000A0B4를 4칸 srli, 4칸 srai, 4칸 slli 한 결과를 각각 16진 8자리로 쓰시오.

    답 보기

---

61. `add` 명령어의 32비트 기계어가 0x00C58433이다. rs2 칸은 [24:20], rs1 칸은 [19:15], rd 칸은 [11:7]이다. 세 칸의 레지스터 번호를 구하고 이 명령어를 어셈블리 한 줄로 쓰시오.

    답 보기

---

62. x10 = 0x00003B7D이다. 아래 두 줄을 차례로 실행한 뒤 각 줄의 x11을 16진 8자리로 쓰시오.

```
andi x11, x10, 0xF0
srli x11, x11, 4
```

    답 보기

---

63. 4비트 값 세 개 1011_2 xor 0110_2 xor 0011_2을 계산해 4비트 2진으로 쓰시오.

    답 보기

---

64. x5 = 21이다. `slli x6, x5, 3`을 실행한 뒤 `srli x7, x6, 4`를 실행했다. x6과 x7을 10진과 16진 8자리로 쓰시오.

    답 보기

---

65. x9 = 0xFFFFFFB0이다. `srai x10, x9, 4`와 `srli x11, x9, 4`를 실행한 뒤 x10, x11을 16진 8자리로 쓰고, x9와 x10을 부호 있는 10진으로 쓰시오.

    답 보기

---

66. x5 = 0x000000C6이다. 아래 두 줄을 차례로 실행한 뒤 x6, x7을 16진 8자리로 쓰시오.

```
ori  x6, x5, 0x0F
xori x7, x6, 0xFF
```

    답 보기

---

67. int 배열 A의 시작 주소가 x25 = 0x00002000, 인덱스 i가 x22 = 15이다. 아래 두 줄을 실행한 뒤 x10을 16진 8자리로 쓰고, 이 값이 무엇인지 쓰시오.

```
slli x10, x22, 2
add  x10, x10, x25
```

답 보기

---

68. 32비트 기계어 0x00068580에서 [19:15]와 [11:7]의 5비트를 각각 2진으로 쓰고, 레지스터 칸이라면 몇 번 레지스터인지 쓰시오.

답 보기

---

69. x10 = 0x7E2C9D41이다. 아래 두 줄을 실행한 뒤 각 줄의 x11을 16진 8자리로 쓰고, 최종 x11을 10진으로 쓰시오. 최종 x11은 x10의 몇 번~몇 번 비트를 꺼낸 것인가?

```
srli x11, x10, 12
andi x11, x11, 0xFF
```

답 보기

---

70. x5 = 0x0000013A이다. 아래 두 줄을 실행한 뒤 x6, x7을 16진 8자리로 쓰고, 각각 부호 있는 10진으로 쓰시오. (즉시값 −1은 12비트로 저장된 뒤 32비트로 부호 확장되어 계산된다.)

```
xori x6, x5, -1
add  x7, x6, x5
```

답 보기

---

71. 8비트 값 1000 1100을 2칸 sra 한 결과와 2칸 srl 한 결과를 2진으로 쓰시오. sra 결과는 부호 있는 수로, srl 결과는 무부호 수로 읽어 10진으로 쓰고, 원래 값과 비교해 확인하시오.

답 보기

---

72. `xor` 명령어의 32비트 기계어가 0x01334733이다(rs2 칸 [24:20], rs1 칸 [19:15], rd 칸 [11:7]). 레지스터 번호를 구해 어셈블리 한 줄로 쓰고, x6 = 0x00FF1234, x19 = 0x0F0F00FF일 때 실행 후 결과 칸의 값을 16진 8자리로 쓰시오.

답 보기

---

73. x22 = 33(인덱스 i), x25 = 0x00001F00(배열 시작 주소)이다. (가) int 배열에서 아래 두 줄을 실행한 뒤 x9를 16진 8자리로 쓰시오. (나) 원소 하나가 8바이트인 배열이라면 첫 줄의 2를 몇으로 바꿔야 하며, 그때 x9는?

```
slli x9, x22, 2
add  x9, x9, x25
```

답 보기

---

74. x5 = 0x0000A7D3이다. x5의 [11:8] 비트 4개를 (가) 0으로 지우는 and 마스크, (나) 1로 켜는 or 마스크, (다) 뒤집는 xor 마스크를 각각 32비트 16진 8자리로 쓰고, 각 연산 결과를 16진 8자리로 쓰시오.

답 보기

---

75. x10 = 0x0003BE50이다. 아래 세 줄을 차례로 실행한 뒤 x11, x12, x13을 16진 8자리로 쓰고, x12는 부호 있는 10진, x13은 무부호 10진으로 쓰시오.

```
slli x11, x10, 20
srai x12, x11, 24
srli x13, x11, 24
```

답 보기

파트 4 · 단위와 어셈블리 읽는 법 (76~100번)

근거: 기초 · 4. 막히면 기초 · 4의 외울 것(단위 표, 어셈블리 표, 배열 주소)과 예제를 다시 보고 오세요.

문제 76~100

76. 클럭 속도가 5 GHz인 CPU의 클럭 주기는 몇 ps인가?

답 보기

---

77. x10 = 0x00003000일 때 `lw x5, 12(x10)`이 읽는 메모리 주소를 16진 8자리로 쓰시오.

답 보기

---

78. 750 ns는 몇 μs인가? 또 몇 ps인가?

답 보기

---

79. x20에 int 배열 A의 시작 주소 0x00000800이 들어 있다. `lw x7, 44(x20)`이 읽는 주소를 16진 8자리로 쓰고, 이 명령을 C 문장으로 쓰시오.

답 보기

---

80. x12 = 5, x13 = 20일 때 `sub x11, x12, x13` 실행 뒤 x11은 몇인가(10진)? 이 명령을 C 문장 꼴로 쓰시오.

답 보기

---

81. 클럭 속도 3 GHz인 CPU에서 2 s 동안 지나가는 클럭 사이클 수를 10의 거듭제곱 꼴과 "억" 단위로 쓰시오.

답 보기

---

82. 메모리에 아래 값(10진)이 들어 있고 x8 = 0x00004000이다. `lw x9, 4(x8)` 실행 뒤 x9와 `lw x10, 8(x8)` 실행 뒤 x10을 10진으로 쓰시오.

주소	0x00004000	0x00004004	0x00004008	0x0000400C
들어 있는 값	25	7	-3	90

답 보기

---

83. int 배열 A의 시작 주소가 0x00005000이다. 주소 0x00005024에 있는 원소는 A의 몇 번 원소인가?

답 보기

---

84. 클럭 주기가 625 ps인 CPU의 클럭 속도는 몇 GHz인가?

답 보기

---

85. x21에 int 배열 A의 시작 주소 0x000010F4가 있다. `sw x6, 16(x21)`이 쓰는 주소를 16진 8자리로 쓰고, C 문장으로 쓰시오. 실행 뒤 x6은 바뀌는가?

답 보기

---

86. 클럭 속도 1.5 GHz인 CPU에서 40 μs 동안 지나가는 클럭 사이클 수를 10의 거듭제곱 꼴과 "만" 단위로 쓰시오.

답 보기

---

87. x22에 int 배열 A의 시작 주소 0x00000300이 있고, 변수 g는 x21에 있다. 아래 코드가 읽는 주소와 쓰는 주소를 16진 8자리로 쓰고, 세 줄을 C 문장 하나로 쓰시오.

```
lw   x9, 8(x22)
add  x9, x9, x21
sw   x9, 32(x22)
```

답 보기

---

88. x5 = 13(인덱스 i), x20 = 0x00002400(int 배열 A의 시작 주소)이다. 아래 두 줄 실행 뒤 x6을 쓰고(첫 줄 뒤는 10진, 둘째 줄 뒤는 16진 8자리), x6이 무엇의 주소인지 쓰시오.

```
slli x6, x5, 2
add  x6, x6, x20
```

답 보기

---

89. 메모리에 아래 값(10진)이 있고 x10 = 0x00006000이다. 아래 코드 실행 뒤 x5, x6, x7을 10진으로 쓰시오.

주소	0x00006000	0x00006004	0x00006008	0x0000600C
들어 있는 값	31	2	14	−9

```
lw   x5, 8(x10)
lw   x6, 12(x10)
add  x7, x5, x6
```

답 보기

---

90. 클럭 주기가 1.25 ns인 CPU의 클럭 속도를 MHz로 쓰고, 0.5 s 동안 지나가는 클럭 사이클 수를 "억" 단위로 쓰시오.

답 보기

---

91. int 배열 A의 시작 주소가 0x00007F68이다. 주소 0x00007FA0에 있는 원소는 A의 몇 번 원소인가?

답 보기

---

92. x5 = 7, x6 = 7, x7 = 10일 때 아래 코드를 실행하면 x7은 몇인가? x6 = 8이었다면 x7은 몇인가?

```
      beq  x5, x6, Skip
      addi x7, x7, 1
Skip: addi x7, x7, 4
```

답 보기

---

93. 클럭 속도 3.2 GHz인 CPU에서 1 s 동안, 그리고 5 ms 동안 지나가는 클럭 사이클 수를 각각 "억"·"만" 단위로 쓰시오.

답 보기

---

94. x23에 int 배열 A의 시작 주소 0x000013F0이 있고, 메모리는 아래와 같다(10진). 코드가 읽는 주소와 쓰는 주소를 16진 8자리로, 실행 뒤 x9와 주소 0x0000141C의 값을 10진으로 쓰고, C 문장 하나로 쓰시오.

주소	0x00001404	0x0000141C
실행 전 값	30	77

```
lw   x9, 20(x23)
addi x9, x9, -6
sw   x9, 44(x23)
```

답 보기

---

95. x20에 int 배열 A의 시작 주소 0x00009A00이 있고, 인덱스 i는 x21 = 3이다. 메모리는 아래와 같다(10진). 코드 실행 뒤 x5(16진 8자리)와 x6(10진)을 쓰고, 세 줄을 C 문장 하나로 쓰시오.

주소	0x00009A00	0x00009A04	0x00009A08	0x00009A0C	0x00009A10
들어 있는 값	11	-4	27	8	19

```
slli x5, x21, 2
add  x5, x5, x20
lw   x6, 0(x5)
```

답 보기

---

96. 원소 하나가 8바이트(doubleword)인 배열 B의 시작 주소가 x22 = 0x00000500이고, 인덱스 i는 x9 = 5이다. 오프셋을 시프트 한 번으로 만들려면 `slli x10, x9, ?`의 ?는 몇이어야 하는가? 그때 x10(10진)과 B[5]의 주소(16진 8자리)를 쓰시오.

답 보기

---

97. x5 = 12, x6 = 15, x10 = 40일 때 아래 코드 실행 뒤 x10과 x11을 쓰시오. x6 = 12였다면 x10과 x11은?

```
      bne  x5, x6, Diff
      addi x10, x10, 100
Diff: sub  x11, x10, x5
```

답 보기

---

98. 클럭 속도 1.6 GHz인 CPU에서 어떤 프로그램이 8억 클럭 사이클 동안 실행되었다. 클럭 주기(ps)와 실행 시간(ms)을 쓰시오.

답 보기

---

99. x8에 int 배열 A의 시작 주소 0x00000FF8이 있고, 메모리는 아래와 같다(10진). 코드가 읽고 쓰는 세 주소를 16진 8자리로, 실행 뒤 x7과 주소 0x00001000의 값을 10진으로 쓰고, C 문장 하나로 쓰시오.

주소	0x00000FF8	0x00000FFC	0x00001000	0x00001004
실행 전 값	6	13	50	−20

```
lw   x5, 4(x8)
lw   x6, 12(x8)
add  x7, x5, x6
sw   x7, 8(x8)
```

답 보기

---

100. x22에 int 배열 A의 시작 주소 0x00003A9C가 있고, 변수 h는 x10에 있다. 아래 코드를 실행했더니 lw가 읽은 주소는 0x00003AC0, sw가 쓴 주소는 0x00003AA4였다. 빈칸 (가), (나)의 숫자(10진)를 쓰고, 세 줄을 C 문장 하나로 쓰시오.

```
lw   x9, (가)(x22)
sub  x9, x9, x10
sw   x9, (나)(x22)
```

답 보기

Ch1 · 1 약 7분

컴퓨터 분류와 ISA

기출 21-2, 26-1: 임베디드 정의 T/F. ISA·ABI 정의는 단답 단골

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 컴퓨터는 쓰임새에 따라 PC·서버·임베디드로 나뉜다. 소프트웨어와 하드웨어 사이에는 ISA라는 약속이 있다.

세 가지 분류: 개인용 컴퓨터(PC)는 범용(general purpose), 즉 다양한 소프트웨어를 돌리는 컴퓨터라서 비용과 성능 사이의 균형이 중요하다(예 데스크톱, 노트북). 서버는 보통 네트워크를 통해 접근하며 대용량·고성능·고신뢰성(잘 멈추지 않음)이 필요하다. 임베디드 컴퓨터는 다른 기기 안에 들어가(슬라이드 예 스마트폰·태블릿, 그 밖에 자동차·세탁기 속 컴퓨터) 미리 정해진 응용만 실행하고, 전력·성능·비용 제약이 엄격하다. [Ch1 p.3]

ISA와 컴퓨터 구조: ISA(Instruction Set Architecture, 명령어 집합 구조)는 하드웨어와 가장 낮은 수준의 소프트웨어 사이의 추상 인터페이스로, 줄여서 아키텍처(architecture)라고도 한다. 어떤 명령어가 있고 레지스터가 몇 개인지 같은 약속이다. 추상화(abstraction)는 아래 계층의 세부를 감추고 위에는 사용법만 보여 주는 것이다. 컴퓨터 구조 = ISA + 조직(Machine Organization)이고, 조직은 그 약속을 실제로 만드는 쪽(프로세서·입출력 구성, 논리·회로·물리 설계)이다. [Ch1 p.9, 17]

ABI와 구현: ABI(Application Binary Interface)는 ISA + OS(운영체제) 인터페이스다. ISA 중 사용자 프로그램이 쓰는 부분에 운영체제를 부르는 방식을 더한, 응용 프로그램이 기계어 수준에서 지킬 약속 전체다. 구현(Implementation)은 아키텍처를 따르는 실제 하드웨어이며, 같은 ISA에 구현은 여러 개일 수 있다. 그래서 칩이 달라도 ISA가 같으면 같은 프로그램이 돈다. [Ch1 p.17]

프로그램 아래 계층: 응용 소프트웨어 → 시스템 소프트웨어(대표 = OS, 컴파일러) → 하드웨어 순서로 쌓여 있다. OS가 하는 일은 셋: 기본 입출력 처리, 저장 공간·메모리 할당, 여러 프로그램이 컴퓨터를 나눠 쓰도록 순서와 몫을 정하는 자원 공유·작업 스케줄링. 컴파일러는 C 같은 고급 언어 프로그램을 하드웨어가 실행하는 명령어로 번역한다. 고급 언어의 장점은 생산성(productivity)과 이식성(portability, 다른 컴퓨터에서도 다시 컴파일만 하면 씀)이다. [Ch1 p.13–14]

비유: 운전자는 핸들·페달만 알면 차를 몬다. 핸들·페달이 ISA, 엔진·브레이크 장치가 그 아래 구현이다. 안 맞는 점: 컴퓨터에서는 아래 계층을 알면 더 빠른 프로그램을 쓸 수 있어서, 이 과목은 추상화 아래를 들여다본다. [Ch1 p.10]

주의: 임베디드는 성능이 낮은 컴퓨터라는 뜻이 아니다. 「범용, 다양한 SW, 비용과 성능의 균형」은 PC 정의라 임베디드라고 하면 X. ABI를 ISA와 같다고 쓰면 X, 구현은 소프트웨어가 아닌 하드웨어다.

2 · 핵심 정리

- PC = 범용, 다양한 SW, 비용·성능 균형 / 서버 = 보통 네트워크로 접근, 대용량·고성능·고신뢰 [Ch1 p.3]
- 임베디드 = 다른 기기 안에서 미리 정해진 응용 실행, 전력·성능·비용 제약이 엄격 [Ch1 p.3]
- 컴퓨터 구조 = ISA + 조직(Machine Organization) [Ch1 p.9]
- ISA = 하드웨어와 최하위 소프트웨어 사이의 추상 인터페이스 = 아키텍처 / ABI = ISA + OS 인터페이스 / 구현 = 아키텍처를 따르는 하드웨어 [Ch1 p.17]
- 응용 SW → 시스템 SW(OS, 컴파일러) → HW. OS 역할 = 입출력 처리, 저장공간·메모리 할당, 자원 공유·작업 스케줄링 [Ch1 p.13]

4 떠올리기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

O/X: 임베디드 컴퓨터는 범용으로 다양한 소프트웨어를 실행하며 비용과 성능의 균형이 가장 중요하다.

답 보기

다른 기기 안에 들어가 미리 정해진 응용을 실행하고 제약이 엄격한 컴퓨터는?

답 보기

빈칸: 컴퓨터 구조 = ( ) + 조직(Machine Organization)

답 보기

ABI = ?

답 보기

아키텍처(ISA)를 따르는 하드웨어를 무엇이라 하는가?

답 보기

고급 언어의 장점 두 가지는?

답 보기

Ch1 · 2 약 6분

8대 위대한 아이디어

map 상: 나열·짝짓기 단답, 영어 이름도 함께

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 컴퓨터 설계자들이 수십 년 동안 되풀이해 써 온 설계 원리 8개다. 시험은 한국어·영어 이름 나열과 설명 짝짓기로 낸다. [Ch1 p.12]

설계 2개: ① 무어의 법칙을 고려한 설계(Design for Moore's Law). 집적회로 자원(칩에 넣는 트랜지스터 수)이 18~24개월마다 2배가 되므로, 몇 년 걸리는 설계를 완성 시점의 기술에 맞춘다. ② 추상화로 설계 단순화(Use abstraction to simplify design). 아래 세부를 감춰 설계를 여러 수준으로 나눈다(앞 덩어리의 ISA·소프트웨어 계층이 예).

빠르게 4개: ③ 자주 생기는 경우를 빠르게(Make the common case fast). 드문 경우를 최적화하는 것보다 효과가 크며, 뒤에 나올 암달의 법칙의 교훈이다. ④~⑥은 영어가 모두 Performance via ~ 꼴이다. 병렬성(parallelism) = 여러 연산을 동시에 계산, 파이프라이닝(pipelining) = 일을 단계로 나눠 단계마다 다른 일을 겹쳐 처리하는 병렬성의 특정 형태, 예측(prediction) = 결과를 기다리기보다 추측해 먼저 시작하는 것이 평균적으로 빠름.

나머지 2개: ⑦ 메모리 계층(Hierarchy of memories). 빠른 메모리는 비싸서 작게만 만들고, 크고 싼 메모리는 느리다. 이 크기와 속도의 충돌을 작고 빠른 층을 위에, 크고 느린 층을 아래에 쌓아 해결한다(캐시 → 주기억 → 디스크). ⑧ 중복을 통한 신뢰성(Dependability via redundancy). 여분 부품을 넣어 하나가 고장 나도 시스템을 믿을 만하게 만든다.

비유: 요리로 치면 여러 명이 동시에 썰기 = 병렬성, 물 끓는 동안 채소 손질 = 파이프라이닝, 자주 쓰는 재료를 손 닿는 곳에 = 메모리 계층이다. 안 맞는 점: 요리 요령은 경험칙이지만 이 8개는 뒤 단원 하드웨어에 실제로 들어 있는 원리다.

주의: 「드문 경우를 빠르게」로 바꾼 O/X는 X. 8번은 중복으로 「성능」이 아니라 「신뢰성」이다. 슬라이드 순서 그대로 앞 글자 무·추·자·병·파·예·메·중으로 외운다.

2 · 핵심 정리

- 설계 2: 무어의 법칙을 고려한 설계(Design for Moore's Law, 18~24개월마다 2배) · 추상화로 설계 단순화(Use abstraction to simplify design) [Ch1 p.12]
- 빠르게 4: 자주 생기는 경우를 빠르게(Make the common case fast) · 병렬성 · 파이프라이닝 · 예측을 통한 성능(Performance via parallelism / pipelining / prediction) [Ch1 p.12]
- 나머지 2: 메모리 계층(Hierarchy of memories) · 중복을 통한 신뢰성(Dependability via redundancy) [Ch1 p.12]
- 파이프라이닝 = 병렬성의 특정 형태 / 예측 = 기다리기보다 추측해 시작하는 게 평균적으로 빠름 / 메모리 계층 = 크기와 속도의 충돌 해결 [Ch1 p.12]

외우는 고리

앞 글자 무·추 | 자·병·파·예 | 메·중 = 설계 2 · 빠르게 4 · 기억·신뢰 2. 「무추(무를 추수하고) 자병파예(자, 병 깨고 파 예약) 메중(메모 중)」. 영어 Performance via ~ 는 병·파·예 세 개

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

8대 아이디어 중 빠진 하나: 무어의 법칙, 추상화, 자주 생기는 경우를 빠르게, 병렬성, 파이프라이닝, 메모리 계층, 중복을 통한 신뢰성

답 보기

영어 이름이 Performance via ~ 인 아이디어 세 개(한국어)와, 그중 병렬성의 특정 형태는?

답 보기

Make the common case fast 의 한국어 이름은?

답 보기

메모리 계층(Hierarchy of memories)은 무엇과 무엇의 충돌을 해결하는가?

답 보기

Dependability via redundancy 의 한국어 이름은?

답 보기

Ch1 · 3 약 6분

5대 구성 요소와 메모리

기출 21-2: 프로세서를 구성하는 두 가지. 26-1 빈칸: 캐시 메모리

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 모든 컴퓨터는 입력·출력·메모리·데이터패스·제어 다섯 부분으로 되어 있고, 그중 데이터패스와 제어가 프로세서다. [Ch1 p.15]

다섯 부분: 데스크톱·서버·임베디드 모두 같은 구성 요소를 갖는다. 데이터패스(datapath)는 산술 연산을 실제로 수행하는 부분이다. 제어(control)는 데이터패스·메모리·입출력에 언제 무엇을 할지 동작 신호를 보낸다. 이 둘을 합친 것이 프로세서(CPU)이고, 메모리·입력·출력은 프로세서 밖에 있다. [Ch1 p.15–16]

데이터 흐름: 입력 장치는 받은 데이터를 메모리에 쓰고, 출력 장치는 메모리에서 데이터를 읽어 내보낸다. 프로세서는 메모리에서 명령어와 데이터를 가져온다. 즉 모든 흐름이 메모리를 거친다. [Ch1 p.15]

DRAM·SRAM·캐시: 주기억장치(메모리)는 DRAM(Dynamic RAM) 칩으로 만든다. 캐시 메모리는 DRAM 앞에서 버퍼 역할을 하는 작고 빠른 메모리이고, SRAM(Static RAM)으로 만든다. SRAM은 DRAM보다 빠르지만 집적도가 낮아(같은 면적에 담는 비트가 적어) 더 비싸다. 그래서 SRAM은 작은 캐시에만 쓰고 큰 주기억은 DRAM으로 만든다. [Ch1 p.17]

휘발성과 비휘발성: 휘발성 메모리는 전원이 꺼지면 내용을 잃는다. 주기억장치(1차 기억장치)인 DRAM과 SRAM이 휘발성이다. 비휘발성 메모리는 전원이 꺼져도 내용이 남는 2차 기억장치로, 디스크·플래시가 예이며 주기억보다 크고 느리다. [Ch1 p.18]

비유: 식당 주방에서 재료 반입구 = 입력, 서빙 창구 = 출력, 냉장고 = 메모리, 요리사 = 데이터패스, 주방장 = 제어다. 주방장은 직접 요리하지 않고 신호만 보낸다. 안 맞는 점: 이 모델에서 재료는 반입구에서 요리사에게 바로 가지 않고 반드시 냉장고(메모리)에 먼저 들어가며, 냉장고에는 요리법(명령어)도 함께 있다.

주의: 「입력은 메모리에서 읽는다」, 「SRAM이 더 싸다」, 「메모리도 프로세서에 포함」은 모두 X.

2 · 핵심 정리

- 5대 구성 요소 = 입력, 출력, 메모리, 데이터패스, 제어 (모든 컴퓨터에 같다) [Ch1 p.15]
- 프로세서 = 데이터패스(산술 연산 수행) + 제어(데이터패스·메모리·입출력에 동작 신호) [Ch1 p.15–16]
- 입력은 메모리에 쓰고, 출력은 메모리에서 읽는다 [Ch1 p.15]
- DRAM = 주기억 / 캐시 메모리 = DRAM의 버퍼인 작고 빠른 메모리, SRAM으로 만듦(빠르지만 집적도 낮아 비쌈) [Ch1 p.17]
- 휘발성 = 주(1차) 기억(DRAM·SRAM) / 비휘발성 = 2차 기억(디스크·플래시), 더 크고 느림 [Ch1 p.18]

외우는 고리

입·출·메 + 데·제. 앞 셋은 프로세서 밖, 뒤 둘(데·제)이 프로세서 안

---

4 떠올리기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

컴퓨터의 5대 구성 요소 중 프로세서를 이루는 두 가지는?

답 보기

5대 구성 요소 중 빠진 하나: 입력, 출력, 데이터패스, 제어

답 보기

데이터패스와 제어가 각각 하는 일은?

답 보기

빈칸: (　　)는 DRAM의 버퍼 역할을 하는 작고 빠른 메모리이며, (　　)으로 만든다.

답 보기

O/X: SRAM은 DRAM보다 싸서 주기억장치로 쓴다.

답 보기

O/X: 입력 장치는 메모리에서 데이터를 읽는다.

답 보기

Ch1 · 4 약 5분

성능의 정의와 시간

map 상: 응답 시간 vs 처리량 O/X, CPU 시간 NOT 문장

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 성능은 시간으로 잰다. 작업 하나에 걸린 시간(응답 시간)과 정해진 시간에 해낸 양(처리량)을 구별하고, 「몇 배 빠르다」는 느린 쪽 시간 ÷ 빠른 쪽 시간으로 구한다.

응답 시간과 처리량: 응답 시간(response time)은 작업 하나의 시작부터 끝까지 걸린 시간으로, 실행 시간(execution time)이라고도 한다. 처리량(throughput)은 정해진 시간 동안 해낸 일의 총량으로, 대역폭(bandwidth)이라고도 한다. 프로세서를 더 빠른 것으로 교체하면 둘 다 좋아지지만, 프로세서를 하나 더 추가하면 동시에 하는 일만 늘어 처리량만 좋아진다. [Ch1 p.24]

비유: 택배 회사가 더 빠른 트럭으로 바꾸면 내 택배도 빨리 오고 하루 배달량도 는다. 트럭을 한 대 더 사면 하루 배달량은 늘지만 내 택배 한 개가 오는 시간은 그대로다. 안 맞는 점: 택배가 밀려 줄 서 있다면 트럭을 늘려 기다림이 줄 수도 있지만, 시험은 슬라이드대로 「추가 = 처리량만」으로 답한다.

몇 배 빠른가: 시간이 짧을수록 좋으므로 성능 = 1 / 실행 시간으로 정한다. 그러면 $성능_X$ / $성능_Y$ = $시간_Y$ / $시간_X$가 되어 느린 쪽 시간을 빠른 쪽 시간으로 나눈다. 예: A 10 s, B 15 s → 15 ÷ 10 = 1.5, A가 1.5배 빠르다. 답이 1보다 작게 나오면 분자·분모를 뒤집은 것이다. [Ch1 p.24]

경과 시간과 CPU 시간: 경과 시간(elapsed time, 벽시계 시간 wall clock)은 시작부터 끝까지의 전체 시간으로, 디스크·I/O(입출력) 기다림과 OS 오버헤드까지 모두 포함한다. CPU 시간은 CPU가 그 작업을 계산하는 데 쓴 시간만이며, I/O 시간·다른 프로그램이 쓴 시간은 포함하지 않는다(NOT). CPU 시간 = 사용자 CPU 시간(프로그램 자기 코드) + 시스템 CPU 시간(그 프로그램을 위해 OS가 일한 시간). 예: 10 s 걸렸고 자기 코드 6 s, OS 일 1 s였다면 경과 시간 10 s, CPU 시간 7 s. [Ch1 p.25]

성능을 높이는 두 길: 하드웨어는 클럭이라는 박자에 맞춰 움직이고, 한 박자를 클럭 사이클, 그 길이를 클럭 주기라 한다(다음 덩어리에서 자세히). CPU 시간 = 클럭 사이클 수 × 클럭 주기이므로 프로그램에 필요한 클럭 사이클 수를 줄이거나 클럭 주기를 줄이면 빨라진다. 둘은 트레이드오프라 한쪽을 좋게 하면 다른 쪽이 나빠지기 쉽다. [Ch1 p.27]

2 · 핵심 정리

- 응답 시간(= 실행 시간) = 작업 하나의 시작~끝 / 처리량(= 대역폭) = 정해진 시간에 한 일의 총량 [Ch1 p.24]
- 프로세서 교체 → 둘 다 개선 / 프로세서 추가 → 처리량만 개선 [Ch1 p.24]
- 성능 = 1 / 실행 시간, X가 n배 빠르다 ⇔ $n = \text{시간}_Y / \text{시간}_X$ (느린 쪽 ÷ 빠른 쪽) [Ch1 p.24]
- 경과 시간(wall clock) = I/O·OS 오버헤드 모두 포함 / CPU 시간 = I/O·다른 프로그램 시간 포함하지 않는다 (NOT) = 사용자 CPU + 시스템 CPU 시간 [Ch1 p.25]
- 성능 개선 = 클럭 사이클 수 줄이기 또는 클럭 주기 줄이기, 둘은 트레이드오프 [Ch1 p.27]

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

O/X: 프로세서를 더 추가하면 응답 시간과 처리량이 모두 개선된다.

답 보기

O/X: CPU 시간은 프로그램이 디스크 입출력을 기다린 시간까지 포함한 전체 시간이다.

답 보기

CPU 시간을 이루는 두 가지는?

답 보기

X가 6 s, Y가 15 s 걸린다. 어느 쪽이 몇 배 빠른가?

답 보기

응답 시간과 처리량의 다른 이름은 각각?

답 보기

Ch1 · 5 약 10분

CPU 시간 공식과 3요소

기출 21-2: 성능을 결정하는 3가지. CPU 시간은 계산 문제 1순위

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: CPU 시간 = 명령어 수(IC) × CPI × 클럭 주기. 성능을 결정하는 3요소가 곱으로 들어가므로 셋 중 하나만 줄어도 시간이 준다. [Ch1 p.28]

클럭: 하드웨어가 박자를 맞추는 한 번의 똑딱이 클럭 사이클(clock cycle)이다. 사이클 하나의 길이가 클럭 주기(예 250 ps), 1초에 도는 사이클 수가 클럭 속도(예 4 GHz, Hz = 초당 횟수)이고, 클럭 속도 = 1 / 클럭 주기다. 예: 250 ps = 250×10^{-12} s → $1 \div (250 \times 10^{-12}) = 4 \times 10^{9}$ Hz = 4 GHz. 400 ps면 $1 \div (4 \times 10^{-10}) = 2.5$ GHz. [Ch1 p.26]

공식이 나오는 이유: CPU 시간 = 사이클 수 × 클럭 주기 = 사이클 수 ÷ 클럭 속도다. 사이클 수는 IC(Instruction Count, 실행된 명령어 개수) × CPI(Cycles Per Instruction, 명령어 하나에 드는 평균 사이클 수)이므로 CPU 시간 = IC × CPI × 클럭 주기 = IC × CPI ÷ 클럭 속도다. 클럭 속도가 1초당 사이클 수라서 사이클 수 = 실행 시간 × 클럭 속도로도 구한다. [Ch1 p.27–28]

평균 CPI: 명령어 종류마다 CPI가 다르다(산술 add, 메모리 읽기·쓰기 load/store = lw·sw, 분기 beq 등). 평균 CPI = 전체 사이클 수 ÷ 전체 명령어 수 = $\Sigma(\text{CPI}_i \times \text{개수}_i) \div$ 전체 개수로, 많이 실행된 종류가 더 크게 반영되는 개수 가중 평균이다. 예: CPI 1인 명령어 3개, CPI 5인 명령어 1개 → 사이클 $3 \times 1 + 1 \times 5 = 8$, 평균 CPI = $8 \div 4 = 2$. CPI 값만 평균 낸 $(1 + 5) / 2 = 3$은 개수를 무시해 틀린다. [Ch1 p.28]

두 컴퓨터 비교·사이클 문제: 같은 ISA로 같은 프로그램을 돌리면 IC가 같아 약분되므로 명령어 하나당 시간 = CPI ÷ 클럭 속도(= CPI × 클럭 주기)만 비교한다. 예: A(250 ps, CPI 2.0) → 500 ps, B(500 ps, CPI 1.2) → 600 ps, $600 \div 500 = 1.2$라 A가 1.2배 빠르다. 사이클 문제 예: A가 2 GHz에서 10 s → 사이클 20×10^{9}. B는 1.2배 사이클(24×10^{9})이 필요하고 6 s에 끝내야 하면 $24 \times 10^{9} \div 6$ s = 4 GHz. [Ch1 p.27–28]

누가 정하나: Ch4 p.2 기준 IC = ISA·컴파일러, CPI = CPU 하드웨어, 클럭 주기 = CPU 하드웨어(마이크로아키텍처 = 내부 회로 설계, 공정 기술). Ch1 p.28은 IC를 정하는 것에 프로그램도 넣는다(프로그램·ISA·컴파일러). 출처를 밝히지 않은 문제면 IC에 프로그램까지 쓰면 안전하다. [Ch1 p.28, Ch4 p.2]

주의: 클럭 속도가 높다고 무조건 빠르지 않다(CPI가 크면 뒤집힌다). 클럭 주기는 곱하고 클럭 속도로는 나눈다. 계산 전에 단위를 s·Hz로 바꾼다(2 GHz = 2×10^{9} Hz, 6억 = 6×10^{8}).

2 · 핵심 정리

- CPU 시간 = IC × CPI × 클럭 주기 = IC × CPI ÷ 클럭 속도 [Ch1 p.28]
- 사이클 수 = 실행 시간 × 클럭 속도 = IC × CPI [Ch1 p.27–28] / 클럭 속도 = 1 / 클럭 주기 (250 ps ↔ 4 GHz) [Ch1 p.26]
- 평균 CPI = $\Sigma(\text{CPI}_i \times \text{개수}_i)$ ÷ 전체 명령어 수 (개수로 가중) [Ch1 p.28]
- 결정 주체(Ch4 기준): IC = ISA·컴파일러, CPI = CPU 하드웨어, 클럭 주기 = CPU 하드웨어 (마이크로아키텍처·공정) [Ch4 p.2] / Ch1 p.28은 IC에 프로그램도 넣음

외우는 고리

3요소 = 개수 × 사이클 × 길이: 명령어 몇 개 × 하나에 몇 사이클 × 사이클 하나 몇 초

---

3 풀이 예

푸는 순서

① 단위를 s·Hz로 바꾼다(2 GHz = 2×10^9 Hz, 6억 = 6×10^8)

② 사이클 수 = Σ(개수 × CPI)

③ 평균 CPI = 사이클 수 ÷ 전체 명령어 수

④ CPU 시간 = 사이클 수 ÷ 클럭 속도

클럭 2 GHz. 산술 40×10^6개(CPI 1), load/store 30×10^6개(CPI 5), 분기 30×10^6개(CPI 2). 평균 CPI와 CPU 시간은?

① 2 GHz = 2×10^9 Hz, 전체 명령어 = $100 \times 10^6 = 10^8$개

② 사이클 = 40 × 1 + 30 × 5 + 30 × 2 = 250 → 250×10^6

③ 평균 CPI = $250 \times 10^6 \div (100 \times 10^6) = 2.5$ ((1 + 5 + 2) / 3 ≈ 2.67이 아님)

④ CPU 시간 = $250 \times 10^6 \div (2 \times 10^9) = 125 \times 10^{-3} = 0.125$ s

답: **평균 CPI 2.5, CPU 시간 0.125 s**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

클럭 4 GHz. 산술 50×10^6개(CPI 1), load/store 20×10^6개(CPI 4), 분기 30×10^6개(CPI 2). 평균 CPI와 CPU 시간은?

① 4 GHz = 4×10^9 Hz, 전체 명령어 = 100×10^6개

② 사이클 = 50 × 1 + 20 × 4 + 30 × 2 = 190 → 190×10^6

③ 평균 CPI = $190 \times 10^6 \div (100 \times 10^6) = 1.9$

④ CPU 시간 = $190 \times 10^6 \div (4 \times 10^9) = 47.5 \times 10^{-3} = 0.0475$ s

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

프로그램의 CPU 실행 시간을 결정하는 세 요소는?

답 보기

세 요소를 각각 무엇이 정하는가(Ch4 슬라이드 기준)?

답 보기

CPI 1인 명령어 6개와 CPI 4인 명령어 2개를 실행한다. 평균 CPI는?

답 보기

같은 ISA인 A(2 GHz, CPI 1.2)와 B(3 GHz, CPI 2.4)가 같은 프로그램을 실행한다. 어느 쪽이 몇 배 빠른가?

답 보기

프로그램이 3 GHz인 A에서 10 s 걸린다. B는 A보다 1.2배 많은 클럭 사이클이 필요하다. B에서 6 s에 끝내려면 B의 클럭 속도는?

답 보기

클럭 주기 400 ps의 클럭 속도는?

답 보기

Ch1 · 6 약 8분

MIPS 계산과 한계

기출 21-2, 26-1: CPI 등을 주고 MIPS 계산(단위 주의). 왜 부적절한가 단답

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: MIPS(Million Instructions Per Second)는 1초에 실행한 명령어 수를 백만(10^6) 단위로 센 값이다. 계산은 쉽지만 성능 지표로는 부적절하다. [Ch1 p.34]

식 두 개: 뜻 그대로 쓰면 MIPS = IC ÷ (실행 시간 × 10^6)이다. 여기에 실행 시간 = IC × CPI ÷ 클럭 속도를 넣으면 IC가 약분되어 MIPS = 클럭 속도 ÷ (CPI × 10^6)이 된다. IC와 시간을 주면 앞 식, 클럭과 CPI를 주면 뒤 식을 쓰고 다른 식으로 검산한다. CPI가 안 주어졌으면 CPI = 사이클 수 ÷ IC = 시간 × 클럭 속도 ÷ IC로 구한다. [Ch1 p.34]

단위: 클럭 속도는 Hz(1초에 몇 번)로 넣는다. M(메가) = 10^6, G(기가) = 10^9이라 500 MHz = 5 × 10^8 Hz이고, 1억 = 10^8이라 4억 = 4 × 10^8이다. MHz 숫자 500을 그대로 넣고 10^6으로 또 나누거나, 분모의 10^6을 빠뜨리면 답이 백만 배 틀린다.

예: 클럭 4 GHz, CPI 2 → 4 × 10^9 ÷ (2 × 10^6) = 2 × 10^3 = 2000 MIPS. 1초에 사이클 4 × 10^9번, 명령어 하나에 2사이클이라 1초에 2 × 10^9개, 백만 단위로 2000이다.

부적절한 이유 세 가지: ① ISA 차이를 반영하지 못한다. ISA가 다르면 같은 일에 필요한 명령어 수가 달라 개수 비교가 무의미하다. ② 명령어 복잡도 차이를 반영하지 못한다. 단순한 명령어와 복잡한 명령어를 똑같이 1개로 센다. ③ 같은 CPU에서도 프로그램마다 CPI가 달라 MIPS가 하나로 정해지지 않는다. [Ch1 p.34]

예: 2 GHz 컴퓨터에서 컴파일러 1은 명령어 10^9개·CPI 1.0(0.5 s, 2000 MIPS), 컴파일러 2는 0.6 × 10^9개·CPI 1.5(0.45 s, 약 1333 MIPS)를 만든다. MIPS는 1이 높지만 실제로는 2가 더 빠르다. 그래서 「MIPS가 높으면 항상 빠르다」는 X다.

2 · 핵심 정리

- MIPS = 클럭 속도 ÷ (CPI × 10^6) = IC ÷ (실행 시간 × 10^6) [Ch1 p.34]
- 클럭 속도는 Hz로 넣는다(500 MHz = 5 × 10^8 Hz). 10^6을 빠뜨리거나 MHz 숫자를 그대로 넣으면 백만 배 틀림
- 부적절 이유 ① ISA 차이 ② 명령어 복잡도 차이를 반영 못 함 ③ 같은 CPU도 프로그램마다 CPI가 다름 [Ch1 p.34]

외우는 고리

부적절 3 = I·복·프 (ISA · 복잡도 · 프로그램마다 CPI)

---

3 풀이 예

푸는 순서

① 클럭을 Hz로, 명령어 수를 10의 거듭제곱으로 바꾼다

② 주어진 값에 맞는 식을 고른다: 클럭·CPI면 클럭 속도 ÷ (CPI × 10^6), IC·시간이면 IC ÷ (시간 × 10^6)

③ 다른 식으로 검산한다(필요하면 CPI = 시간 × 클럭 속도 ÷ IC)

클럭 800 MHz인 컴퓨터가 4억 개의 명령어를 0.8 s에 실행했다. MIPS와 평균 CPI는?

① 4억 = 4 × 10^8, 800 MHz = 8 × 10^8 Hz

② MIPS = 4 × 10^8 ÷ (0.8 × 10^6) = 5 × 10^2 = 500

③ 사이클 = 0.8 × 8 × 10^8 = 6.4 × 10^8, CPI = 6.4 × 10^8 ÷ (4 × 10^8) = 1.6. 검산: 8 × 10^8 ÷ (1.6 × 10^6) = 500

답: **500 MIPS, CPI 1.6**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

클럭 1.5 GHz인 컴퓨터가 6억 개의 명령어를 0.5 s에 실행했다. MIPS와 평균 CPI는?

① 6억 = 6×10^8, 1.5 GHz = 1.5×10^9 Hz

② MIPS = $6 \times 10^8 \div (0.5 \times 10^6)$ = 1200

③ 사이클 = $0.5 \times 1.5 \times 10^9 = 7.5 \times 10^8$, CPI = $7.5 \times 10^8 \div (6 \times 10^8) = 1.25$. 검산: $1.5 \times 10^9 \div (1.25 \times 10^6) = 1200$

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

클럭 2.4 GHz, 평균 CPI 1.5인 컴퓨터의 MIPS는?

답 보기

클럭 500 MHz, CPI 2일 때 MIPS는? 클럭 속도를 식에 어떻게 넣는가?

답 보기

명령어 3×10^9개를 2 s에 실행했다. MIPS는?

답 보기

MIPS가 성능 지표로 부적절한 이유 세 가지는?

답 보기

O/X: ISA가 다른 두 컴퓨터는 MIPS가 높은 쪽이 항상 더 빠르다.

답 보기

Ch1 · 7 약 7분

실행 시간 표와 개선율

기출 26-1: 실행 명령어·시간 표 → 성능 개선율

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 무언가를 개선한 효과는 속도 향상 = 개선 전 시간 ÷ 개선 후 시간으로 잰다. [Ch1 p.24]

속도 향상과 %: 성능 = 1 / 시간이므로 앞 덩어리의 「n배 빠르다」와 같은 방향, 즉 $T_{전} \div T_{후}$(1보다 크게)다. 성능이 몇 % 좋아졌는지는 (속도 향상 − 1) × 100이고, 시간이 몇 % 줄었는지는 $(T_{전} - T_{후}) \div T_{전}$이라 숫자가 다르다. 예: 10 s → 8 s면 속도 향상 1.25배, 성능 25% 향상, 시간 20% 감소.

사이클만 비교해도 되는 경우: 시간 = 사이클 수 ÷ 클럭 속도이므로 클럭이 같은 두 실행은 클럭 속도가 약분되어 사이클 수(= Σ 개수 × CPI)만 비교하면 된다. 문제가 명령어 종류별 개수·CPI 표를 주면 개선 전·후 사이클을 각각 더해 나눈다. 캐시(메모리 앞의 작고 빠른 메모리)를 개선하면 메모리에 접근하는 load/store의 CPI가 작아진다. [Ch1 p.17, 28]

예: 클럭 2 GHz, 산술 40 × 10^6개(CPI 1), load/store 30 × 10^6개(CPI 5), 분기 30 × 10^6개(CPI 2). 전: 40 + 150 + 60 = 250 × 10^6 사이클 = 0.125 s. load/store CPI가 2가 되면 후: 40 + 60 + 60 = 160 × 10^6 사이클 = 0.08 s. 속도 향상 = 0.125 ÷ 0.08 ≈ 1.56배(250 ÷ 160과 같다).

명령어 수만 보면 안 된다: 교재 예제에서 CPI 1·2·3인 명령어를 코드 1은 2·1·2개(5개, 사이클 10), 코드 2는 4·1·1개(6개, 사이클 9) 쓴다. 명령어가 더 많은 코드 2가 사이클이 적어 더 빠르다. 그래서 개선 전후는 IC × CPI(사이클 수)로 비교한다. [Ch1 p.28]

2 · 핵심 정리

- 속도 향상 = $T_{전} \div T_{후}$ (1보다 크게) [Ch1 p.24]
- 성능 % 향상 = (속도 향상 − 1) × 100 / 시간 감소율 = $(T_{전} - T_{후}) \div T_{전}$
- 클럭이 같으면 시간 대신 사이클 수(= Σ 개수 × CPI)만 비교해도 된다 [Ch1 p.28]
- 명령어 수가 줄어도 CPI가 커지면 느려질 수 있다: IC × CPI로 비교 [Ch1 p.28]

3 풀이 예

푸는 순서

① 개선 전 사이클 수 = Σ(개수 × CPI), 시간 = 사이클 ÷ 클럭 속도

② 바뀐 종류의 CPI(또는 개수)만 바꿔 개선 후 사이클·시간

③ 속도 향상 = $T_{전} \div T_{후}$, 필요하면 % 로

클럭 2 GHz. 산술 40 × 10^6개(CPI 1), load/store 30 × 10^6개(CPI 5), 분기 30 × 10^6개(CPI 2). 캐시를 개선해 load/store의 CPI가 2가 되면 속도 향상은?

① 전: 40 + 150 + 60 = 250 × 10^6 사이클 → 250 × 10^6 ÷ (2 × 10^9) = 0.125 s

② 후: 40 + 60 + 60 = 160 × 10^6 사이클 → 0.08 s

③ 0.125 ÷ 0.08 = 1.5625 ≈ 1.56배 (성능 약 56% 향상, 시간 36% 감소)

답: **약 1.56배**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

클럭 2 GHz. 산술 40 × 10^6개(CPI 1), load/store 30 × 10^6개(CPI 4), 분기 30 × 10^6개(CPI 2). load/store의 CPI가 2가 되면 속도 향상은?

① 전: 40 + 120 + 60 = 220 × 10^6 사이클 → 0.11 s

② 후: 40 + 60 + 60 = 160 × 10^6 사이클 → 0.08 s

③ 0.11 ÷ 0.08 = 1.375배 (성능 37.5% 향상)

답 보기

떠올리기 카드 4장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

개선 전 12 s, 개선 후 8 s. 속도 향상은 몇 배이고 성능은 몇 % 향상인가?

답 보기

같은 클럭에서 개선 전 명령어 2 × 10^9개(CPI 2), 개선 후 1.6 × 10^9개(CPI 2.5). 속도 향상은?

답 보기

클럭이 같은 두 실행을 비교할 때 시간 대신 무엇만 비교해도 되는가?

답 보기

속도 향상(speedup)의 식은?

답 보기

Ch1 · 8 약 8분

암달의 법칙

map 상: 계산 단골. 슬라이드 수치(2.5배 → n = 4, 최대 1.25배)

1 읽기 | 2 핵심 | 3 풀이 예 | 4 떠올리기

---

1 읽기

한 줄 요약: 프로그램의 일부만 빨라지면 전체는 그만큼 빨라지지 않는다. 빨라지지 않은 나머지 시간이 그대로 남기 때문이다. [Ch1 p.33]

식: $T_{영향}$ = 개선이 적용되는 부분의 시간, $T_{비영향}$ = 손대지 않아 그대로 남는 시간, n = 영향 부분이 빨라진 배수다. 영향 부분은 n배 빨라져 $T_{영향} \div n$이 되고 나머지는 그대로라서 $T_{후} = T_{영향} \div n + T_{비영향}$이다. 예: 10 s 중 6 s 부분을 3배 빠르게 → 6 ÷ 3 + 4 = 6 s, 속도 향상 10 ÷ 6 ≈ 1.67배.

최대 속도 향상: n을 아무리 키워도 $T_{영향} \div n$은 0에 가까워질 뿐이라 시간은 $T_{비영향}$ 밑으로 내려가지 않는다. 그래서 최대 속도 향상 = $T_{전} \div T_{비영향}$이다(위 예는 10 ÷ 4 = 2.5배). 목표 시간이 $T_{비영향}$ 이하이면 어떤 n으로도 불가능하다. [Ch1 p.33]

슬라이드 예: 100 s 중 곱셈이 80 s. 전체를 2.5배 빠르게 하려면 목표 = 100 ÷ 2.5 = 40 s, 비영향 = 100 − 80 = 20 s. 40 = 80 ÷ n + 20 → 80 ÷ n = 20 → n = 4(곱셈 4배). 곱셈 외 20 s만 개선하면 곱셈 80 s가 그대로 남아 최대 100 ÷ 80 = 1.25배라 2.5배는 불가능하다. [Ch1 p.33]

교훈: 시간을 많이 차지하는 부분을 빠르게 해야 효과가 크다. 이것이 8대 아이디어의 「자주 생기는 경우를 빠르게」다. 비유: 100분 통학 중 버스 80분, 걷기 20분이면 버스를 아무리 빠르게 해도 20분 아래로는 못 줄인다. 안 맞는 점: 프로그램은 어느 부분이 몇 초인지 재 봐야 안다. [Ch1 p.12, 33]

주의: n은 영향 받는 부분에만 나눈다. 전체 시간을 n으로 나누면 틀린다.

2 · 핵심 정리

- $T_{후}$ = $T_{영향} \div n$ + $T_{비영향}$ [Ch1 p.33]
- 최대 속도 향상 = $T_{전} \div T_{비영향}$ (n이 아무리 커도) [Ch1 p.33]
- 슬라이드: 100 s 중 곱셈 80 s, 전체 2.5배 → 40 = 80 / n + 20 → n = 4. 곱셈 외만 개선 → 최대 1.25배, 불가능 [Ch1 p.33]
- 교훈 = 자주 생기는 경우를 빠르게 [Ch1 p.12, 33]

3 풀이 예

푸는 순서

① 목표 시간 = $T_{전}$ ÷ 목표 배수

② $T_{비영향}$ = $T_{전}$ − $T_{영향}$

③ 목표 = $T_{영향}$ / n + $T_{비영향}$ 을 n에 대해 푼다

④ 목표 ≤ $T_{비영향}$ 이면 불가능, 최대 = $T_{전}$ ÷ $T_{비영향}$

100 s 걸리는 프로그램 중 곱셈이 80 s다. 전체를 2.5배 빠르게 하려면 곱셈을 몇 배 개선해야 하는가? 곱셈 외 부분만 개선하면?

① 목표 = 100 ÷ 2.5 = 40 s

② 비영향 = 100 − 80 = 20 s

③ 40 = 80 / n + 20 → 80 / n = 20 → n = 4

④ 곱셈 외만 개선하면 80 s가 남아 최대 100 ÷ 80 = 1.25배 < 2.5 → 불가능

답: **n = 4, 곱셈 외 개선은 불가능(최대 1.25배)**

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

120 s 걸리는 프로그램 중 부동소수점 연산이 90 s다. 전체를 3배 빠르게 하려면 부동소수점을 몇 배 개선해야 하는가? 부동소수점 외 부분만 개선하면?

① 목표 = 120 ÷ 3 = 40 s

② 비영향 = 120 − 90 = 30 s

③ 40 = 90 / n + 30 → 90 / n = 10 → n = 9

④ 부동소수점 외만 개선하면 90 s가 남아 최대 120 ÷ 90 ≈ 1.33배 < 3 → 불가능

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

슬라이드 암달 예: 100 s 중 곱셈 80 s, 전체를 2.5배 빠르게 하려면 곱셈을 몇 배?

답 보기

같은 예에서 곱셈 외 부분만 개선하면 최대 몇 배까지 가능한가?

답 보기

---

50 s 중 30 s 부분을 3배 빠르게 하면 전체 속도 향상은?

답 보기

---

50 s 중 30 s 부분을 아무리 빠르게 해도 넘을 수 없는 최대 속도 향상은?

답 보기

---

100 s 중 곱셈 80 s일 때 전체를 5배 빠르게 할 수 있는가?

답 보기

Ch1 · 9 약 10분

전력 장벽·멀티코어·수율

map 상: 전력 NOT 두 줄·계산. 수율 공식은 중(빈칸)

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 칩의 전력은 전압의 제곱에 비례한다. 전압을 더 낮출 수 없고 열도 더 뺄 수 없게 되자(전력 장벽) 클럭 대신 코어 수를 늘렸다. 칩 한 개 값은 수율로 정해진다.

에너지와 전력: 지금 칩의 주류 기술 CMOS에서 동적 에너지 ∝ 용량성 부하 × 전압^{2}이다(∝ = 비례). 용량성 부하는 트랜지스터가 한 번 켜지고 꺼질 때 채우고 비워야 하는 전기의 양을 정한다. 전력은 여기에 1초당 전환 횟수인 주파수(= 클럭 속도)를 곱한다: 전력 ∝ 용량성 부하 × 전압^2 × 주파수. 에너지 식에는 주파수가 없다. [Ch1 p.30–31]

비율로 계산: 새 전력 ÷ 옛 전력을 구하면 비례 상수가 약분되어 (부하 비율) × (전압 비율)^2 × (주파수 비율)만 남는다. 예: 교재 예제는 셋 다 0.85배라 0.85 × 0.85^2 × 0.85 = 0.85^4 ≈ 0.52배. 다른 것은 그대로 두고 전압만 0.5배로 낮추면 0.5^2 = 0.25배다. [Ch1 p.31]

전력 장벽과 멀티코어: 지난 20년간 클럭은 1000배 올랐는데 전력은 30배만 늘었다. 전압을 5 V에서 1 V로(세대마다 약 15%씩) 낮췄고 전압이 제곱으로 들어가기 때문이다. 그런데 ① 누설(leakage, 꺼져 있어도 전류가 새는 것) 때문에 전압을 더 낮출 수 없고(NOT) ② 냉각 비용 때문에 열을 더 제거할 수 없다(NOT). 그래서 2006년 무렵부터 칩 하나에 코어(프로세서) 여러 개를 넣는 멀티코어로 바꿨고, 이득은 응답 시간보다 처리량 쪽에 크다. [Ch1 p.31–32]

비유: 페달을 빨리 밟을수록(클럭) 힘(전력)이 든다. 기어(전압)를 낮춰 버티다가 더 못 낮추고 열도 못 식히게 되자, 자전거를 여러 대 두고 여러 사람이 나눠 탔다(멀티코어). 안 맞는 점: 멀티코어는 프로그램이 일을 나눌 수 있게 짜여 있어야 한 프로그램이 빨라진다.

칩 비용과 수율: 칩은 둥근 실리콘 판인 웨이퍼(wafer)에 여러 개를 찍어 잘라 낸다. 잘라 낸 칩 하나가 다이(die), 만든 다이 중 정상인 비율이 수율(yield)이다. 다이당 비용 = 웨이퍼당 비용 ÷ (웨이퍼당 다이 수 × 수율)로, 웨이퍼 값을 정상 다이에만 나눠 매긴다. 수율 = 1 ÷ (1 + 면적당 결함 수 × 다이 면적 ÷ 2)^{N}이고 교재 예제는 N = 2다. [Ch1 p.21–22]

예: 면적당 결함 0.2개/cm^2, 다이 면적 1 cm^2 → 0.2 × 1 ÷ 2 = 0.1, 수율 = 1 ÷ 1.1^2 = 1 ÷ 1.21 ≈ 0.83. 웨이퍼 한 장 5000달러에 다이 500개면 다이당 5000 ÷ (500 × 0.826) ≈ 12.1달러. 다이가 커지면 다이 수도 줄고 수율도 떨어져 비용이 면적보다 빠르게 오른다. [Ch1 p.22]

2 · 핵심 정리

- 동적 에너지 ∝ 용량성 부하 × 전압^2 / 전력 ∝ 용량성 부하 × 전압^2 × 주파수 [Ch1 p.30–31]
- 클럭 1000배에 전력 30배뿐 = 전압 5 V → 1 V (세대당 약 15%) [Ch1 p.31]
- 전력 장벽 ① 전압을 더 낮출 수 없다(NOT): 누설 ② 열을 더 제거할 수 없다(NOT): 냉각 비용 [Ch1 p.31]
- 2006년부터 멀티코어, 이득은 응답 시간보다 처리량 [Ch1 p.32]
- 다이당 비용 = 웨이퍼당 비용 ÷ (웨이퍼당 다이 수 × 수율) / 수율 = 1 ÷ (1 + 면적당 결함 수 × 다이 면적 / 2)^N (교재 예제 N = 2) [Ch1 p.22]

외우는 고리

전력 = 부·전²·주 (전압만 제곱). 장벽 = 누설·냉각 → 멀티코어

---

3 풀이 예

푸는 순서

① 새 값 ÷ 옛 값을 요소별 비율로 쓴다(비례 상수는 약분)

② 전압 비율만 제곱한다

③ 부하 비율 × 전압 비율^2 × 주파수 비율

교재 예제: 새 프로세서는 용량성 부하·전압·주파수가 모두 기존의 0.85배다. 동적 전력은 몇 배인가?

① 비율: 부하 0.85, 전압 0.85, 주파수 0.85

② 전압만 제곱: 0.85^2

③ $0.85 \times 0.85^2 \times 0.85 = 0.85^4 \approx 0.52$

답: **약 0.52배**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

새 프로세서는 용량성 부하가 0.9배, 전압이 0.8배, 주파수가 1.5배다. 동적 전력은 몇 배인가?

① 비율: 부하 0.9, 전압 0.8, 주파수 1.5

② 전압만 제곱: 0.8^2 = 0.64

③ 0.9 × 0.64 × 1.5 = 0.9 × 0.96 = 0.864

답 보기

떠올리기 카드 7장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

빈칸: 전력 ∝ ( ) × 전압^2 × ( )

답 보기

전력 장벽의 두 가지(무엇을 할 수 없는가와 그 이유)는?

답 보기

멀티코어로 바꾼 뒤 주로 개선되는 것은 응답 시간인가 처리량인가?

답 보기

다른 조건은 그대로 두고 전압만 0.5배로 낮추면 동적 전력은?

답 보기

20년간 클럭은 1000배 올랐는데 전력은 30배만 늘어난 이유는?

답 보기

빈칸: 다이당 비용 = 웨이퍼당 비용 ÷ (× 수율)

답 보기

면적당 결함 0.2개/cm^2, 다이 면적 2 cm^2, N = 2일 때 수율은?

답 보기

Ch2 · 1 약 7분

레지스터 역할

기출 26-1 실행 후 레지스터 값, 21-2 스택 문제의 바탕. 「19개」는 O/X 함정

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: RISC-V CPU 안에는 값을 담는 32비트 칸(레지스터) 32개 x0~x31이 있고, 몇몇 번호는 맡은 역할이 정해져 있다.

레지스터(register) = CPU 안의 아주 빠른 저장 칸이다. 칸 하나는 32비트이고, 32비트 덩어리를 word(워드), 그 두 배인 64비트를 doubleword라 부른다. 개수를 32개로 묶어 둔 이유는 설계 원칙 2(작을수록 빠르다)에서 다시 나온다. [Ch2 p.10]

특별한 번호 넷: **x0** = 배선으로 0에 고정, 읽으면 언제나 0. **x1** = 복귀 주소(return address), 즉 함수를 마친 뒤 돌아갈 명령어 주소. 함수로 건너가는 명령어 jal이 jal 다음 명령어 주소(PC + 4)를 x1에 써 넣는다. PC(program counter)는 지금 실행 중인 명령어의 주소이고, 명령어 하나가 4바이트라 다음 명령어는 +4다. **x2 = sp**(stack pointer) = 값을 잠시 맡기는 메모리 영역(스택)의 맨 위 주소, **x8 = fp**(frame pointer) = 함수 하나가 실행되는 동안 변하지 않는 기준 주소. [Ch2 p.16, 34–37, 44]

함수(프로시저)를 부를 때 쓰는 칸: 인자(argument, 함수에 넘기는 값)는 x10~x17 여덟 개에 넣고, 반환값(return value, 함수가 돌려주는 결과)은 그중 x10·x11에 넣는다. 반환값 칸이 인자 칸 안에 있으므로 따로 세지 않는다. 그래서 호출용 레지스터는 x10~x17(8개) + x1(1개) = 9개다(11개로 세면 틀림). [Ch2 p.34]

나머지는 두 그룹이다. 임시(temporary) = caller-saved = x5~x7, x28~x31(3 + 4 = 7개): 불려 간 함수가 마음대로 바꿔도 되는 칸. 저장(saved) = callee-saved = x8~x9, x18~x27(2 + 10 = 12개): 불려 간 함수가 쓰려면 원래 값을 보관했다가 돌려놓아야 하는 칸. 슬라이드 p.39가 나눈 두 그룹은 이 7 + 12 = 19개뿐이고, x10~x17(임시 쪽)과 x1(저장 쪽)은 호출 규약(p.41·43)에서 따로 더해진다. 그래서 「x10~x17·x1 포함 19개」는 X. 슬라이드 예제가 x9를 중간값 칸으로 쓰기도 하지만 규약상 x9는 저장 레지스터다. 누가 언제 저장하는지는 「프로시저와 스택」 덩어리에서. [Ch2 p.39, 41, 43]

예(x0 활용): `sub x6, x0, x5` = 0 − x5, 즉 부호 반전. x5 = 7이면 x6 = −7이고, 32비트 2의 보수로는 0x00000007을 반전한 0xFFFFFFF8에 1을 더한 0xFFFFFFF9다. `add x7, x6, x0` = x6 + 0이라 복사와 같다. `beq x0, x0, L`은 「0과 0이 같으면 L로」인데 늘 같으므로 항상 건너뛰는 무조건 분기가 된다. [Ch2 p.16, 29]

주의: x0을 결과 칸에 두면(`addi x0, x5, 3`) 계산은 해도 쓰기가 무시되어 x0은 여전히 0이다(PDF 외 보충).

2 · 핵심 정리

- 32비트 레지스터 32개 x0~x31, word = 32비트, doubleword = 64비트 [Ch2 p.10]
- x0 = 항상 0 · x1 = 복귀 주소(jal이 저장) · x2 = sp · x8 = fp [Ch2 p.16, 34–37, 44]
- x10~x17 = 인자 8개, 그중 x10·x11 = 반환값 → 호출용 = x10~x17 + x1 = 9개 [Ch2 p.34]
- 임시(caller-saved) = x5~x7, x28~x31 7개 / 저장(callee-saved) = x8~x9, x18~x27 12개 → 합 19개 [Ch2 p.39]
- 호출 규약에서 추가: 임시 쪽 x10~x17, 저장 쪽 x1 [Ch2 p.41, 43]
- x0 활용: `sub x1, x0, x1` = 부호 반전, `beq x0, x0, L` = 무조건 분기 [Ch2 p.16, 29]

외우는 고리

번호 고리: 0 = 영(항상 0), 1 = 일 끝나고 돌아갈 곳(복귀 주소), 2 = sp(두 글자), 8 = fp(프레임). 임시 7 + 저장 12 = 19, 인자·x1은 19개에 없음

---

4 떠올리기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

jal이 복귀 주소를 남기는 레지스터는?

답 보기

스택 포인터 sp와 프레임 포인터 fp의 레지스터 번호는?

답 보기

O/X: RISC-V는 x10~x17과 x1을 포함한 19개 레지스터를 임시·저장 두 그룹으로 나눈다.

답 보기

프로시저 호출에 쓰는 레지스터는 몇 개이고 무엇인가?

답 보기

처음 x5 = 9. `sub x6, x0, x5` 뒤 x6을 16진 8자리로?

답 보기

규약상 x9는 임시 레지스터인가 저장 레지스터인가?

답 보기

Ch2 · 2 약 9분

바이트 주소·엔디언

기출 26-1: 코드 실행 뒤 레지스터 값, little/big endian 배치

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 메모리는 바이트마다 주소가 붙은 긴 줄이다. 계산은 레지스터에서만 하므로 lw로 가져오고 sw로 써 넣으며, 4바이트 값을 주소 4개에 놓는 순서가 엔디언이다.

RISC-V의 산술 명령어(add, sub 등)는 레지스터끼리만 계산하고 메모리 값을 바로 쓰지 못한다. 메모리 값은 load(lw)로 레지스터에 읽어 오고, 결과는 store(sw)로 메모리에 쓴다. 레지스터가 훨씬 빠르므로 컴파일러는 변수를 최대한 레지스터에 두고, 덜 쓰는 것만 메모리로 내보낸다(spill). [Ch2 p.11, 15]

`lw rd, 오프셋(rs1)` = 주소 rs1 + 오프셋에서 4바이트를 읽어 rd에. `sw rs2, 오프셋(rs1)` = rs2의 값을 주소 rs1 + 오프셋에 쓴다. sw는 맨 앞 레지스터가 결과를 받는 칸이 아니라 **저장할 값**이다. 주소는 바이트마다 하나씩 붙고 int 하나는 4바이트라 A[i]는 시작 주소에서 i × 4바이트 뒤에 있다. 그래서 A[8]의 오프셋은 8이 아니라 32다. [Ch2 p.12, 14]

예: `A[12] = h + A[8];` (h = x21, A의 시작 주소 = x22) → `lw x9, 32(x22)` · `add x9, x21, x9` · `sw x9, 48(x22)`. 거꾸로 오프셋에서 인덱스는 ÷ 4. x22 = 0x00001000일 때 `lw x7, 40(x22)`는 40 ÷ 4 = 10이라 A[10], 40 = 2 × 16 + 8 = 0x28이므로 주소 0x1028부터 읽는다. [Ch2 p.14]

엔디언(endianness): 32비트 값은 16진 8자리이고 두 자리가 1바이트라 4바이트다. 맨 왼쪽 두 자리 = 최상위 바이트(자릿값이 가장 큼), 맨 오른쪽 두 자리 = 최하위 바이트. **빅 엔디언** = 최상위 바이트를 가장 낮은 주소에, **리틀 엔디언** = 최하위 바이트를 가장 낮은 주소에. **RISC-V는 리틀 엔디언**이다. 예: 0x00112233을 주소 0~3에 두면 빅은 00 11 22 33, 리틀은 33 22 11 00. [Ch2 p.13]

예(추적): x22 = 0x00000100, 주소 0x104~0x107에 바이트 78, 56, 34, 12. `lw x5, 4(x22)`는 0x104부터 읽고, 리틀이라 높은 주소의 바이트부터 왼쪽에 이어 붙여 x5 = 0x12345678(빅 엔디언 기계라면 쓰인 순서 그대로 0x78563412). `addi x5, x5, 1` → 0x12345679. `sw x5, 12(x22)`는 주소 0x10C(12 = 0xC)에 오른쪽 끝 두 자리부터: 0x10C = 79, 0x10D = 56, 0x10E = 34, 0x10F = 12. [Ch2 p.13–14]

주의: 뒤집는 것은 바이트 순서뿐이고 바이트 안의 두 자리(79)는 그대로다. 또 RISC-V는 정렬 제한이 없어서, word가 4의 배수 주소에서 시작하지 않아도(예 0x1002) lw로 읽을 수 있다. [Ch2 p.13]

2 · 핵심 정리

- 산술은 레지스터에서만 → 메모리는 load·store [Ch2 p.11]
- A[i]의 오프셋 = i × 4 (바이트 주소, int 4바이트) [Ch2 p.12, 14]
- `lw rd, 오프셋(rs1)` 메모리 → 레지스터 / `sw rs2, 오프셋(rs1)` = rs2 값을 메모리에 [Ch2 p.14]
- 빅 = 최상위 바이트를 가장 낮은 주소에 / 리틀 = 최하위 바이트를 가장 낮은 주소에, RISC-V = 리틀 [Ch2 p.13]
- 0x00112233 → 주소 0~3: 빅 00 11 22 33 / 리틀 33 22 11 00 [Ch2 p.13]
- 정렬 제한 없음: word가 4의 배수 주소에서 시작할 필요 NOT [Ch2 p.13]

외우는 고리

리틀 = 작은 끝(오른쪽 두 자리)이 작은 주소에 먼저 들어간다

---

3 풀이 예

푸는 순서

① 주소 = 베이스 + 인덱스 × 4 (10진 오프셋을 16진으로 바꿔 더하기)

② 16진 8자리를 두 자리씩 끊는다. 오른쪽 끝 두 자리 = 최하위 바이트

③ lw(리틀): 높은 주소 바이트부터 왼쪽에 이어 붙인다

④ sw(리틀): 오른쪽 끝 두 자리를 가장 낮은 주소에

⑤ 바이트 안의 두 자리는 뒤집지 않는다

x22 = 0x00000200, 주소 0x208~0x20B에 바이트 9A, 78, 56, 34가 차례로 있다. `lw x5, 8(x22)` · `addi x5, x5, 2` · `sw x5, 20(x22)` 뒤 x5와 주소 0x214~0x217의 바이트는?

① lw 주소 = 0x200 + 8 = 0x208 (A[2])

② 리틀: 높은 주소부터 34 56 78 9A → x5 = 0x3456789A

③ 2를 더함 → 0x3456789C

④ sw 주소 = 0x200 + 20 = 0x214 (20 = 0x14, A[5])

⑤ 최하위 9C부터: 0x214 = 9C, 0x215 = 78, 0x216 = 56, 0x217 = 34

답: **x5 = 0x3456789C, 0x214~0x217 = 9C 78 56 34**

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

x9 = 0x00000400, 주소 0x40C~0x40F에 바이트 01, EF, CD, AB가 차례로 있다. `lw x6, 12(x9)` · `addi x6, x6, 1` · `sw x6, 28(x9)` 뒤 x6과 주소 0x41C~0x41F의 바이트는?

① lw 주소 = 0x400 + 12 = 0x40C (A[3])

② 높은 주소부터 이어 붙임 → x6 = 0xABCDEF01

③ 1을 더함 → 0xABCDEF02

④ sw 주소 = 0x400 + 28 = 0x41C (28 = 0x1C)

⑤ 0x41C = 02, 0x41D = EF, 0x41E = CD, 0x41F = AB

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

x22 = 0x00002000. `lw x7, 52(x22)`는 A의 몇 번째 원소? 읽는 시작 주소는?

답 보기

A[12] = h + A[8];의 lw 오프셋과 sw 오프셋은?

답 보기

주소 0x80~0x83에 바이트 11, 22, 33, 44. `lw` 결과는 RISC-V에서? 빅 엔디언이면?

답 보기

0x0A0B0C0D를 주소 0x100에 sw. 0x100의 바이트는 RISC-V에서? 빅 엔디언이면?

답 보기

O/X: RISC-V에서 주소 0x1006에서 시작하는 word는 4의 배수 주소가 아니라 lw로 읽을 수 없다.

답 보기

Ch2 · 3 약 11분

C ↔ 어셈블리

기출 26-1: C → 어셈블리 2문제, 어셈블리 → C 1문제, 실행 후 레지스터 값

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: C 한 줄은 정해진 틀 몇 가지(산술, 배열, if-else, while, 큰 상수) 중 하나로 바뀐다. 틀을 익히고 문제에 주어진 레지스터 번호만 끼워 넣는다.

산술·배열: `f = g + h;` → `add x19, x20, x21` (소스 2 + 목적지 1). 상수는 addi로 더하고, 빼기 즉시값 명령어(subi)는 없어 `addi x19, x20, -10`처럼 음수를 더한다. 인덱스가 변수 i면 오프셋 자리에 i × 4를 쓸 수 없다. 오프셋은 명령어에 적힌 **상수**라 어셈블할 때 정해지는데 i는 실행 중에 바뀌기 때문이다. 그래서 `slli x5, x22, 2` (왼쪽 2칸 = × 4, 예 i = 3이면 11_2 → 1100_2 = 12) → `add x5, x5, x25` (시작 주소 더하기) → `lw x19, 0(x5)` (오프셋 0)로 주소 전체를 레지스터에 만든다. [Ch2 p.14, 16, 30]

조건 분기는 「조건이 참이면 라벨로 건너뛰고, 거짓이면 바로 다음 줄」이다. 틀은 then 부분을 분기 바로 아래에 두므로, C 조건이 참이면 그대로 내려가고 거짓일 때만 Else로 건너뛰어야 한다. 그래서 **분기 조건 = C 조건의 반대**다. then이 끝나면 `beq x0, x0, Exit` (늘 참인 무조건 분기)로 else를 건너뛴다. 이 줄을 빠뜨리면 else까지 실행된다. [Ch2 p.28–29]

예(f~j = x19~x23): `if (i == j) f = g + h; else f = g - h;`

```
bne x22, x23, Else   (i != j면 Else)
add x19, x20, x21
beq x0, x0, Exit
Else: sub x19, x20, x21
Exit:
```

while도 같다: `Loop:`에서 반대 조건이면 Exit로 나가고, 몸체 뒤 `beq x0, x0, Loop`로 처음으로 돌아간다. [Ch2 p.29–30]

반대 조건 짝: == → bne, != → beq, < → bge, >= → blt. RISC-V에는 bgt(>)·ble(<=)가 없다. 그래서 a > b의 반대 a <= b는 b >= a로 바꿔 `bge b, a, Else`, a <= b의 반대 a > b는 `blt b, a, Else`. blt·bge는 **부호 있는** 비교, 끝에 u가 붙은 bltu·bgeu는 **무부호** 비교다. 0xFFFFFFFF와 1: blt는 −1 < 1이라 참, bltu는 $2^{32} - 1 < 1$이라 거짓. [Ch2 p.31]

예(추적, i = x20, A = x21): `while (A[i] != 0) i = i + 1;` → `Loop: slli x5, x20, 2` · `add x5, x5, x21` · `lw x6, 0(x5)` · `beq x6, x0, Exit` · `addi x20, x20, 1` · `beq x0, x0, Loop` · `Exit:`. 처음 i = 0, A = {3, 7, 0, 5}면 A[0], A[1]에서 계속, A[2] = 0에서 탈출 → i = 2, addi 2번. 어셈블리를 C로 되돌릴 때는 오프셋 ÷ 4 = 인덱스, 분기 조건 다시 뒤집기, 뒤로 가는 분기 = 반복문으로 읽는다. 추적할 때 음수는 32비트 2의 보수로 쓴다.

32비트 큰 상수: addi 즉시값은 12비트뿐이라 두 번에 넣는다. `lui rd, 상수` (load upper immediate) = 20비트 상수를 31~12번 비트에, 아래 12비트는 0. 16진 8자리를 위 5자리 | 아래 3자리로 잘라 lui에 위, addi에 아래를 준다. 예 0x003D0500 → `lui x19, 0x003D0` + `addi x19, x19, 0x500`. 아래 3자리가 0x800 이상(11번 비트 1)이면 addi가 12비트 2의 보수 음수(값 − 4096)로 읽으므로 lui를 1 늘리고 addi에 (값 − 4096)을 준다. 예 0x00012800 → `lui x5, 0x00013` + `addi x5, x5, -2048` (2048 − 4096), 0x13000 − 0x800 = 0x12800. [Ch2 p.46] (0x800 이상 처리는 PDF 외 보충)

2 · 핵심 정리

- 변수 인덱스 A[i]: slli x5, i, 2 → add x5, x5, 베이스 → lw rd, 0(x5) [Ch2 p.30]
- if-else: 뒤집은 조건으로 Else → then → beq x0, x0, Exit → Else: → Exit: [Ch2 p.29]
- while: Loop: 뒤집은 조건으로 Exit → 몸체 → beq x0, x0, Loop [Ch2 p.30]
- Else로 가는 분기: == → bne, != → beq, < → bge, >= → blt, > → bge b, a, <= → blt b, a
- blt·bge = 부호 있는 비교, bltu·bgeu = 무부호: 0xFFFFFFFF vs 1 → blt 참, bltu 거짓 [Ch2 p.31]
- 32비트 상수 = lui(위 20비트) + addi(아래 12비트). 아래 3자리가 0x800 이상이면 lui +1 [Ch2 p.46]

외우는 고리

분기는 「참이면 건너뛴다」 → then을 건너뛸 조건 = C 조건의 반대

---

3 풀이 예

푸는 순서

① 문제의 변수 ↔ 레지스터 대응을 옮겨 적는다

② C 조건을 뒤집어 Else(반복문은 Exit)로 가는 분기를 쓴다

③ then(몸체)을 쓰고 끝에 beq x0, x0, Exit (반복문은 Loop)

④ 실행 추적: 줄마다 레지스터 값 표, 음수는 32비트 2의 보수

a, b, c = x18, x19, x20(부호 있는 정수). `if (a >= b) c = a - b; else c = b - a;` **를 RISC-V로 쓰고, a = 3, b = 8일 때 c는?**

① C 조건 a >= b의 반대 = a < b → `blt x18, x19, Else`

② then: `sub x20, x18, x19`

③ `beq x0, x0, Exit`로 else를 건너뛴다

④ `Else: sub x20, x19, x18`, 다음 줄 `Exit:`

⑤ 3 < 8 참 → Else로 → c = 8 − 3 = 5

답: **blt x18, x19, Else · sub x20, x18, x19 · beq x0, x0, Exit · Else: sub x20, x19, x18 · Exit:, c = 5**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

f, g, h, i, j = x19, x20, x21, x22, x23. `if (i != j) f = g + 5; else f = h;`를 RISC-V로 쓰시오.

① C 조건 i != j의 반대 = i == j → `beq x22, x23, Else`

② then: `addi x19, x20, 5`

③ `beq x0, x0, Exit`

④ Else: add x19, x21, x0 (f = h 복사, addi x19, x21, 0도 같음), 다음 줄 Exit:

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

실행 뒤 x5, x6(16진 8자리)과 add 실행 횟수는?

```
addi x5, x0, 0
addi x6, x0, 4
Loop: beq x6, x0, Exit
add x5, x5, x6
addi x6, x6, -1
beq x0, x0, Loop
Exit:
```

답 보기

g = x19, A의 시작 주소 = x23. `lw x9, 12(x23)` · `add x9, x9, x19` · `sw x9, 4(x23)`를 C 한 줄로?

답 보기

---

x5 = 0xFFFFFFFE, x6 = 3. `blt x5, x6, L`과 `bltu x5, x6, L` 중 분기하는 것은?

답 보기

---

a = x5, b = x6. if (a > b) …에서 Else로 가는 분기 한 줄은?

답 보기

---

x6에 0x00ABC9A0을 넣으려 한다. lui 값과 addi 즉시값(10진)은?

답 보기

변수 i로 A[i]를 읽을 때 lw 오프셋 자리에 i × 4를 못 쓰는 이유는?

답 보기

Ch2 · 4 약 11분

프로시저와 스택

기출 21-2: 레지스터 번호 주고 스택 할당 → 덧셈 → 원래 주소로 복귀

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 함수(프로시저)를 부르는 일은 6단계로 진행되고, 레지스터 값을 잠시 맡길 때는 메모리의 스택 자리를 sp로 직접 늘렸다 줄인다.

프로시저(procedure) = 함수. 부르는 쪽 = 호출자(caller), 불려 가는 쪽 = 피호출자(callee). 6단계: ① 인자 배치(피호출자가 읽을 수 있는 x10~x17에) ② 제어 이동(jal) ③ 저장 공간 확보(스택에 자리) ④ 작업 수행 ⑤ 결과 배치(호출자가 읽을 수 있는 x10에) ⑥ 복귀(jalr). 호출자가 ①②, 피호출자가 ③~⑥을 한다. 앞 글자 「인제 저작 결복」. [Ch2 p.32–33]

`jal x1, L` (jump and link) = x1 = PC + 4(다음 명령어 주소)를 남기고 PC = L로 건너간다. `jalr x0, 0(x1)` (jump and link register) = PC = x1 + 0으로 돌아간다. jalr도 PC + 4를 앞 레지스터(rd)에 쓰는데, 복귀할 때는 더 돌아올 곳이 없으니 rd = x0으로 두어 버린다. 여기서 x0은 「0번지」가 아니라 링크를 **버리는 곳**이다. 예: 주소 0x1000의 `jal x1, proc` → x1 = 0x1004, proc 끝의 jalr → PC = 0x1004. [Ch2 p.35–36]

스택(stack) = 레지스터 값을 잠시 맡기는 메모리 영역. 나중에 넣은 것을 먼저 꺼내는 LIFO(last-in-first-out)이고, push/pop 명령어가 없어 sw로 넣고 lw로 꺼낸다. 스택은 **높은 주소 → 낮은 주소**로 자라므로 자리를 만들 때 sp를 줄인다. 레지스터 하나 = 4바이트라 k개면 `addi sp, sp, -4k`, 칸은 4(k − 1)(sp), …, 4(sp), 0(sp)이고 가장 높은 칸 = 원래 sp − 4. 결과는 `addi x10, x20, 0`처럼 x10에 옮기고(⑤), 같은 오프셋에서 lw로 복원, `addi sp, sp, 4k`로 반납, `jalr x0, 0(x1)`로 복귀. [Ch2 p.37]

예(sp 계산): 원래 sp = 0x00001000, 3개 저장 → 3 × 4 = 12 → `addi sp, sp, -12`. 0x1000 − 12 = 4096 − 12 = 4084 = 0x00000FF4. 16진으로 빼면 끝 자리 0에서 C를 못 빼 빌림이 맨 앞 1까지 이어진다(0x1000 = 0x0FF0 + 0x10). 칸: 0(sp) = 0xFF4, 4(sp) = 0xFF8, 8(sp) = 0xFFC(= 원래 sp − 4). 빌림이 없으면 끝 두 자리만 10진으로: 0x7FFFFF10이면 0x10 = 16, 16 − 12 = 4 → 0x7FFFFF04.

누가 저장하나: callee-saved(x8~x9, x18~x27, 그리고 x1)는 피호출자가 쓰기 전에 저장하고 복귀 전에 복원한다. caller-saved(x5~x7, x28~x31, x10~x17)는 피호출자가 마음대로 바꿔도 되므로, **호출 뒤에도 그 값이 필요할 때만 호출자가** 호출 전에 저장한다. 다른 함수를 부르는 nonleaf나 자기 자신을 부르는 재귀(recursion)는 jal이 x1을 새 복귀 주소로 덮으므로 자기 x1도 저장한다. 예: P가 x7, x21을 쓰고 Q를 부르며 Q 뒤 x7이 필요 없으면 P가 저장할 것은

x21(callee-saved)과 x1 → `addi sp, sp, -8`. 팩토리얼 fact도 x1과 인자 x10 두 개를 저장해 −8. [Ch2 p.39, 41–43]

메모리 배치: text(기계어 코드) · static data(상수·전역 변수) · heap(실행 중 malloc으로 빌리는 동적 영역, 위로 자람) · stack(지역 변수·저장한 레지스터, 아래로 자람). 스택과 힙은 **서로를 향해** 자란다. fp(x8)는 함수 실행 중 sp가 바뀌어도 변하지 않는 기준이라 지역 변수를 같은 오프셋으로 찾게 해 준다. [Ch2 p.44–45]

2 · 핵심 정리

- 6단계: 인자 배치 → 제어 이동(jal) → 저장 공간 확보 → 작업 수행 → 결과 배치 → 복귀(jalr) [Ch2 p.33]
- `jal x1, L` : x1 = PC + 4, PC = L / `jalr x0, 0(x1)` : PC = x1, 링크는 x0에 버림 [Ch2 p.35–36]
- 스택: LIFO, push/pop 없음(sw/lw), 높은 → 낮은 주소로 자람 [Ch2 p.37]
- k개 저장: `addi sp, sp, -4k` → 오프셋 4(k − 1), …, 4, 0 → 같은 오프셋 lw → `addi sp, sp, 4k`
- nonleaf·재귀: jal이 덮으므로 x1 저장 (fact = x1·x10 두 개 → −8) [Ch2 p.41–42]
- 메모리 배치: text · static data · heap(위로) · stack(아래로), 스택과 힙은 서로를 향해 자람 [Ch2 p.44–45]

외우는 고리

6단계 앞 글자 「인제 저작 결복」: 인자 놓고 → 제어 넘기고 → 저장 자리 잡고 → 작업 → 결과 놓고 → 복귀

---

3 풀이 예

푸는 순서

① 저장할 레지스터 수 k를 센다

② `addi sp, sp, -4k` (바이트 수, 빼기)

③ 새 sp = 원래 sp − 4k (16진 끝 두 자리를 10진으로 계산, 빌림 주의)

④ 칸 주소 = 새 sp + 4(k − 1), …, + 0. 가장 높은 칸 = 원래 sp − 4

⑤ 결과를 x10에 → 같은 오프셋 lw → `addi sp, sp, 4k` → `jalr x0, 0(x1)`

처음 sp = 0x00002000. x1, x18, x19, x20 네 개를 저장한다. 자리 확보 명령어, 새 sp, 가장 높은 칸 12(sp)의 주소는?

① k = 4 → 4 × 4 = 16바이트 → `addi sp, sp, -16`

② 0x2000 − 0x10: 끝 자리들이 0이라 빌림이 앞자리까지 이어진다. 0x2000 = 0x1FF0 + 0x10 → sp = 0x00001FF0

③ 12(sp) = 0x1FF0 + 12 = 0x1FFC (= 원래 sp − 4)

답: **addi sp, sp, -16, sp = 0x00001FF0, 12(sp) = 0x00001FFC**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

처음 sp = 0x7FFFFF20. 레지스터 세 개를 저장한다. 자리 확보 명령어, 새 sp, 가장 높은 칸 8(sp)의 주소는?

① k = 3 → 12 바이트 → `addi sp, sp, -12`

② 끝 두 자리 0x20 = 32, 32 − 12 = 20 = 0x14 → sp = 0x7FFFFF14

③ 8(sp) = 20 + 8 = 28 = 0x1C → 0x7FFFFF1C

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

빈칸: 인자 배치 → 제어 이동 → ( ) → 작업 수행 → 결과 배치 → 복귀

답 보기

주소 0x00400100의 `jal x1, proc` 실행 직후 x1은?

답 보기

`jalr x0, 0(x1)`에서 x0의 뜻은?

답 보기

처음 sp = 0x00003010. 5개를 저장할 자리를 만든 뒤 sp와 16(sp)의 주소는?

답 보기

(21-2 유형) int f(int g, int h) { return g + h; }, g·h = x10·x11, 결과는 x20에 계산하고 x20을 스택에 저장한다. 빈칸: `addi sp, sp, (가)` · `sw x20, 0(sp)` · `add x20, x10, x11` · (나) · `lw x20, 0(sp)` · `addi sp, sp, 4` · (다)

답 보기

P는 x6, x19를 쓰고 Q를 호출한다. Q 호출 뒤에도 x6 값이 필요하다. P가 저장해야 하는 레지스터는?

답 보기

Ch2 · 5 약 7분

명령어 형식 칸

기출 26-1: encoding 이름 빈칸. 인코딩 표를 또 줄지 몰라 칸 순서·비트 위치는 외움

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 기계어(machine code) = 명령어를 바꾼 32비트 2진 값이고, 형식(instruction format) = 그 32비트를 어떤 칸(필드)으로 나누는지 정한 배치다. R·I·S·SB·U·UJ 여섯 가지가 있다. [Ch2 p.21]

칸 이름: **opcode**(operation code) = 무슨 종류 명령어인지 알려 주는 7비트 칸으로, 모든 형식에서 맨 오른쪽 6~0번 비트다. **funct3·funct7** = opcode가 같은 명령어(예 add와 sub)를 구별하는 3비트·7비트 칸. **rd·rs1·rs2** = 레지스터 번호 5비트 칸(레지스터 32개라 5비트). **imm** = 즉시값 칸, imm[11:5]는 즉시값의 11~5번 비트라는 뜻. 비트 번호는 왼쪽 끝이 31번이다. [Ch2 p.22]

R형(레지스터 셋, add·sub·sra): funct7(31–25) · rs2(24–20) · rs1(19–15) · funct3(14–12) · rd(11–7) · opcode(6–0). 칸 크기 7·5·5·3·5·7을 더하면 32. rs2가 rs1보다 왼쪽이다. **I형**(레지스터 + 12비트 상수, lw·addi·slli·jalr): imm[11:0](31–20) · rs1 · funct3 · rd · opcode. 즉시값이 12비트 2의 보수라 범위는 -2^{11} ~ $2^{11}-1$ = −2048 ~ 2047. [Ch2 p.22–23]

S형(sw): sw는 메모리에 쓰기만 하니 결과를 받을 rd가 없다. 그 rd 자리(11–7)에 즉시값 아래 5비트 imm[4:0], funct7 자리(31–25)에 위 7비트 imm[11:5]를 넣는다: imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode. 즉시값을 굳이 쪼개는 이유는 rs1·rs2·funct3·opcode를 **모든 형식에서 같은 자리**에 두기 위해서다. 그러면 하드웨어가 형식을 몰라도 레지스터 번호를 같은 곳에서 읽는다(설계 원칙 3 「좋은 설계에는 좋은 타협(Good design demands good compromises)」의 내용 중 「형식은 최대한 비슷하게」). [Ch2 p.23–24]

SB형(조건 분기 beq·bne·blt·bge·bltu·bgeu): 비교만 하므로 rd가 없고, 칸 자리는 S형과 같다. 분기 오프셋은 늘 짝수라 0번 비트를 저장하지 않고 imm[12:1]만 넣는다. 배치: imm[12] → 31번, imm[10:5] → 30–25번, imm[4:1] → 11–8번, imm[11] → 7번. imm[12]는 모든 형식에서 부호 비트가 있는 31번에, imm[11]은 S형에서 imm[0]이 있던 7번에 둔다(교재 기준). 오프셋 계산은 「SB 분기 오프셋」 덩어리. [Ch2 p.47]

U형 = lui, UJ형 = jal: 둘 다 20비트 즉시값 · rd · opcode. 명령어별 형식: R = add·sub·sra(시프트 양이 레지스터), I = lw·addi·slli(시프트 양이 즉시값)·jalr(레지스터 + 12비트 상수), S = sw, SB = beq·bne·blt·bltu. 주의: jalr은 이름에 jump가 있어도 I형이고, UJ형은 jal 하나뿐이다. [Ch2 p.46–48, 50]

2 · 핵심 정리

- R: funct7 31–25 · rs2 24–20 · rs1 19–15 · funct3 14–12 · rd 11–7 · opcode 6–0 [Ch2 p.22]
- I: imm[11:0] 31–20 · rs1 · funct3 · rd · opcode, 즉시값 −2048 ~ 2047 [Ch2 p.23]
- S: imm[11:5] 31–25 · rs2 · rs1 · funct3 · imm[4:0] 11–7 · opcode (rd 없음) [Ch2 p.24]
- SB: imm[12] → 31 · imm[10:5] → 30–25 · imm[4:1] → 11–8 · imm[11] → 7번 [Ch2 p.47]
- U = lui, UJ = jal (20비트 즉시값 · rd · opcode) [Ch2 p.46–48]
- R: add·sub·sra / I: lw·addi·slli·jalr / S: sw / SB: beq·bne·blt·bltu [Ch2 p.50]

외우는 고리

R형 칸 크기 7·5·5·3·5·7 = 32. 「rs2가 rs1보다 왼쪽」, opcode는 늘 맨 오른쪽 7비트

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

인코딩 형식 이름: jalr, jal, lui

답 보기

인코딩 형식 이름: slli, sra, sw, bltu

답 보기

---

rd 칸이 없는 형식 두 가지와, 그 형식이 즉시값을 조각내 넣는 이유는?

답 보기

S형에서 imm[4:0]이 들어가는 비트 번호는?

답 보기

SB형에서 imm[11]과 imm[12]가 놓이는 비트 번호는?

답 보기

Ch2 · 6 약 10분

인코딩·디코딩

기출 26-1: 어셈블리 → 기계어 → little endian으로 쓰기(encoding 표 줌)

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 인코딩 = 어셈블리 한 줄을 형식의 칸에 맞춰 32비트로 채우고 4비트씩 끊어 16진 8자리로 읽는 것, 디코딩 = 그 반대다. 32칸 양식지 채우기라고 생각하면 된다.

자주 쓰는 값(p.50 표 일부, 외울 것):
add: R, opcode 0110011, funct3 000, funct7 0000000
sub: R, 0110011, 000, funct7 0100000
addi: I, 0010011, 000 / slli: I, 0010011, 001
lw: I, 0000011, 010 / jalr: I, 1100111, 000
sw: S, 0100011, 010
분기: SB, 1100011, funct3 beq 000 · bne 001 · blt 100 · bge 101 · bltu 110 · bgeu 111
add와 sub는 opcode·funct3가 같고 funct7만 다르다. [Ch2 p.50]

어셈블리에 쓰는 레지스터 순서 ≠ 칸 순서: `add rd, rs1, rs2`는 칸에 왼쪽부터 rs2 · rs1 · rd 순으로 들어간다. `lw rd, 상수(rs1)`, `sw rs2, 상수(rs1)`: sw의 앞 레지스터는 rd가 아니라 저장할 값 rs2다. 분기 `bne rs1, rs2, 라벨`은 앞이 rs1. [Ch2 p.14, 22, 47]

인코딩 순서: ① 표에서 형식·opcode·funct를 옮긴다 ② 레지스터 번호 → 5비트, 즉시값 → 2의 보수 ③ 칸 순서대로 이어 쓴다 ④ 4비트씩 끊어 16진 ⑤ 메모리에 둘 때는 리틀 엔디언이라 오른쪽 두 자리부터 낮은 주소에. 예 `add x7, x5, x6`: funct7 0000000 · rs2 = 6 = 00110 · rs1 = 5 = 00101 · funct3 000 · rd = 7 = 00111 · opcode 0110011 → 0000 0000 0110 0010 1000 0011 1011 0011 → 0x006283B3.

예(S형) `sw x18, 36(x2)`: rs2 = 저장할 값 x18 = 10010, rs1 = x2 = 00010. 36 = 0000 0010 0100(12비트)을 위 7비트 0000001 | 아래 5비트 00100으로 나눈다. imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode = 0000001 10010 00010 010 00100 0100011 → 0x03212223. 주소 0x300부터 바이트 23 22 21 03.

예(I형, 음수 즉시값) `addi x5, x5, -1`: −1의 12비트 2의 보수 = 1111 1111 1111(1을 반전 후 + 1). imm · rs1 · funct3 · rd · opcode = 111111111111 00101 000 00101 0010011 → 0xFFF28293.

디코딩: ① 메모리 바이트가 주어지면 높은 주소부터 모아 8자리로(리틀) ② 2진으로 바꿔 **맨 오른쪽 7비트 opcode**로 형식을 정한다 ③ 형식의 칸 폭대로 자른다 ④ funct3·funct7로 명령어를 고르고 어셈블리 순서로 쓴다. 예 0x500~0x503 = 33 05 B5 40 → 0x40B50533 → 0100000 | 01011 | 01010 | 000 | 01010 | 0110011 → R형, funct7 0100000이라 sub, rs2 = 11, rs1 = 10, rd = 10 → `sub x10, x10, x11`. [Ch2 p.50]

2 · 핵심 정리

- 인코딩: 표에서 opcode·funct → 레지스터 번호 5비트, 즉시값 2의 보수 → 칸 순서로 이어 쓰기 → 4비트씩 16진
- 어셈블리 순서 ≠ 칸 순서: `add rd, rs1, rs2` → 칸은 rs2 · rs1 · rd 순 [Ch2 p.22]
- `sw rs2, 상수(rs1)` : 앞 레지스터 = rs2 (저장할 값) [Ch2 p.14]
- 디코딩: 16진 → 2진 → 맨 오른쪽 opcode 7비트 로 형식 → 칸 자르기 → funct3·funct7로 명령어 [Ch2 p.50]
- 메모리에 쓰기(리틀): 8자리의 오른쪽 두 자리 를 주소 +0에. 덤프를 읽을 땐 높은 주소 부터 모은다
- add 0110011·000·0000000 / sub funct7 0100000 / addi 0010011·000 / lw 0000011·010 / sw 0100011·010 [Ch2 p.50]

---

3 풀이 예

푸는 순서

① 표에서 형식·opcode·funct3·funct7을 옮긴다

② 레지스터 번호 → 5비트, 즉시값 → 2의 보수

③ 형식의 칸 순서 대로 이어 쓴다(rs2가 rs1보다 왼쪽)

④ 4비트씩 끊어 16진 8자리

⑤ 리틀 엔디언: 맨 오른쪽 바이트 를 주소 +0에

주소 0x100의 `sub x7, x8, x9` 를 16진 기계어로 바꾸고 0x100~0x103의 바이트를 쓰시오.

① R형, opcode 0110011, funct3 000, funct7 0100000

② rd = 7 = 00111, rs1 = 8 = 01000, rs2 = 9 = 01001

③ funct7 · rs2 · rs1 · funct3 · rd · opcode: 0100000 01001 01000 000 00111 0110011

④ 4비트씩: 0100 0000 1001 0100 0000 0011 1011 0011 → 0x409403B3

⑤ 리틀: 0x100 = B3, 0x101 = 03, 0x102 = 94, 0x103 = 40

답: **0x409403B3, 바이트 B3 03 94 40**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

주소 0x200의 `lw x6, 20(x10)`을 16진 기계어로 바꾸고 0x200~0x203의 바이트를 쓰시오.

① I형, opcode 0000011, funct3 010

② imm = 20 = 0000 0001 0100, rs1 = x10 = 01010, rd = x6 = 00110

③ imm · rs1 · funct3 · rd · opcode: 000000010100 01010 010 00110 0000011

④ 4비트씩: 0000 0001 0100 0101 0010 0011 0000 0011 → 0x01452303

⑤ 0x200 = 03, 0x201 = 23, 0x202 = 45, 0x203 = 01

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

`sw x9, 24(x2)`의 16진 기계어는?

답 보기

메모리 0x600~0x603에 바이트 B3 02 73 00이 있다. 어셈블리는?

답 보기

0x00A30393을 어셈블리로?

답 보기

슬라이드 예 0x00578833을 어셈블리로?

답 보기

---

add와 sub를 구별하는 칸과 각 값은?

답 보기

Ch2 · 7 약 10분

SB 분기 오프셋

기출 26-1 빈칸: branch offset. 오프셋 계산 → 조각 배치 → 기계어

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 조건 분기(SB형)는 목표 주소를 그대로 넣지 않고, 「분기 명령어 자신에서 몇 바이트 떨어졌나」(오프셋)를 13비트로 만들어 조각내 넣는다.

PC 상대 주소 지정(PC-relative): 목표 = PC + 오프셋. 분기를 실행하는 동안 PC에는 그 분기 명령어 **자신의 주소**가 들어 있으므로 오프셋 = 목표 주소 − 분기 명령어 자신의 주소다(PC + 4 기준 아님). 단위는 **바이트**라 명령어 1개 = 4, 뒤(낮은 주소)로 가면 음수. 이렇게 하는 이유: 주소를 20비트 칸에 그대로 넣으면 프로그램이 2^{20} B = 1 MiB를 넘을 수 없고, if/for/while의 분기 목표는 대부분 가까워서 PC가 이상적인 기준이다. [Ch2 p.48]

분기 목표는 짝수 주소뿐이라 오프셋의 0번 비트는 늘 0이다. 그래서 칸에는 imm[12:1] 12비트만 저장하고(저장값 = 오프셋 ÷ 2), 버린 0번까지 치면 오프셋은 13비트 2의 보수(12번 비트 = 부호)다. 범위는 -2^{12} ~ $2^{12} - 1$ = −4096 ~ 4095인데 짝수만 되므로 **−4096 ~ 4094 바이트**(2의 배수). word로 바꾸면 4096 ÷ 4 = 2^{10}이라 p.48은 $\pm 2^{10}$ word로 적는다. jal은 $\pm 2^{20}$ 바이트 = $\pm 2^{18}$ word. [Ch2 p.47–48]

인코딩 순서: ① 오프셋 = 목표 − 자기 주소(16진을 10진으로 바꿔 빼기) ② 13비트 2의 보수로(음수면 크기를 13비트로 쓰고 반전 + 1) ③ 왼쪽부터 12번 | 11번 | 10~5번(6개) | 4~1번(4개) | 0번으로 끊는다 ④ imm[12] · imm[10:5] · rs2 · rs1 · funct3 · imm[4:1] · imm[11] · opcode 1100011 순으로 이어 쓴다(0번은 버림) ⑤ 4비트씩 16진.

예(앞으로): 0x200 `beq x10, x0, Exit`, Exit = 0x208. 오프셋 = +8 = 0 0000 0000 1000 → imm[12] = 0, imm[11] = 0, imm[10:5] = 000000, imm[4:1] = 0100. beq funct3 000, rs1 = x10 = 01010, rs2 = x0 = 00000 → 0 000000 00000 01010 000 0100 0 1100011 → 0x00050463.

예(뒤로): 0x100 `Loop:` … / 0x104 `bne x5, x0, Loop`. 오프셋 = 0x100 − 0x104 = −4(명령어 1개 뒤로). 4 = 0 0000 0000 0100 → 반전 1 1111 1111 1011 → + 1 → 1 1111 1111 1100. 끊으면 imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 1110, 저장값 imm[12:1] = 111111111110(= −2 = −4 ÷ 2). bne funct3 001, rs1 = 00101, rs2 = 00000 → 1 111111 00000 00101 001 1110 1 1100011 → 0xFE029EE3.

되모으기(디코딩): 31번 → imm[12], 7번 → imm[11], 30~25번 → imm[10:5], 11~8번 → imm[4:1]을 이 순서로 이어 쓰고 끝에 0을 붙여 13비트를 만든다. 맨 앞이 1이면 음수라 반전 + 1로 크기를 구한다. 예 저장값 111111111000 → 1 1111 1111 0000 → 반전 0 0000 0000 1111, + 1 = 16 → −16. 목표 = 자기 주소 + 오프셋.

2 · 핵심 정리

- 오프셋(바이트) = 목표 주소 − 분기 명령어 자신의 주소 (PC 상대) [Ch2 p.48]
- 뒤로 가면 음수, 명령어 1개 = 4바이트
- 13비트 2의 보수, 0번 비트는 늘 0이라 저장 안 함 → 저장값 imm[12:1] = 오프셋 ÷ 2 [Ch2 p.47]
- 끊기: 12번 | 11번 | 10~5번 | 4~1번 | 0번 → 칸 31 | 7 | 30–25 | 11–8 | 버림
- 범위: −4096 ~ 4094 바이트(짝수만) = $\pm 2^{10}$ word, jal $\pm 2^{18}$ word [Ch2 p.47–48]

---

3 풀이 예

푸는 순서

① 오프셋 = 목표 − 자기 주소 (16진을 10진으로 바꿔 빼기)

② 13비트 2의 보수로 쓴다 (음수면 크기를 13비트로 → 반전 → +1)

③ 12 | 11 | 10~5 | 4~1 | 0으로 끊는다

④ 칸 순서: imm[12] · imm[10:5] · rs2 · rs1 · funct3 · imm[4:1] · imm[11] · opcode 1100011

⑤ 4비트씩 16진

0x300 Loop: … / 0x318 `blt x5, x6, Loop`. 오프셋, 저장되는 imm[12:1], 16진 기계어는?

① 0x300 − 0x318 = 768 − 792 = −24 (명령어 6개 뒤로)

② 24 = 0 0000 0001 1000 → 반전 1 1111 1110 0111 → +1 → 1 1111 1110 1000

③ imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 0100, 0번 0은 버림 → imm[12:1] = 111111110100 (= −12)

④ blt funct3 100, rs1 = x5 = 00101, rs2 = x6 = 00110: 1 111111 00110 00101 100 0100 1 1100011

⑤ 1111 1110 0110 0010 1100 0100 1110 0011 → 0xFE62C4E3

답: **오프셋 −24, imm[12:1] = 111111110100, 0xFE62C4E3**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

0x404 Loop: … / 0x410 `bne x7, x0, Loop`. 오프셋과 16진 기계어는?

① 0x404 − 0x410 = −12 (명령어 3개 뒤로)

② 12 = 0 0000 0000 1100 → 반전 → +1 → 1 1111 1111 0100

③ imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 1010

④ bne funct3 001, rs1 = x7 = 00111, rs2 = x0 = 00000: 1 111111 00000 00111 001 1010 1 1100011

⑤ 1111 1110 0000 0011 1001 1010 1110 0011 → 0xFE039AE3

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

0x200 `beq x10, x0, Exit`, Exit = 0x210. 오프셋과 16진 기계어는?

답 보기

SB형에 저장된 imm[12:1]이 111111111110이다. 실제 오프셋은?

답 보기

빈칸: SB 조건 분기 범위는 ( ) ~ ( ) 바이트, ( )의 배수만

답 보기

O/X: 분기 오프셋은 다음 명령어 주소(PC + 4)를 기준으로 word 단위로 센다.

답 보기

분기에 PC 상대 주소 지정을 쓰는 이유 두 가지는?

답 보기

Ch2 · 8 약 5분

설계 원칙·주소 지정

설계 원칙 3은 문장 그대로 출제되기 쉬움(상), 주소 지정 4는 jalr = 베이스 함정

1 읽기 2 핵심 3 떠올리기

---

1 읽기

한 줄 요약: RISC-V를 설계할 때 따른 원칙 3가지(영어 문장 그대로 외움)와, 명령어가 피연산자·주소를 얻는 방법(주소 지정 방식) 4가지.

원칙 1 **Simplicity favors regularity**(단순성은 규칙성을 선호): 규칙이 일정하면 하드웨어가 단순해진다. 그래서 산술 명령어는 모두 소스 2개 + 목적지 1개라는 같은 모양이다(`add a, b, c`). [Ch2 p.9]

원칙 2 **Smaller is faster**(작을수록 빠르다): 레지스터를 32개로 제한한 근거다. 레지스터가 많으면 클럭 주기가 늘 수 있으므로, 레지스터가 많다고 무조건 빠른 것이 아니다. [Ch2 p.10]

원칙 3 **Good design demands good compromises**(좋은 설계에는 좋은 타협): 형식을 R·I·S 등 여러 개 두면 해독은 복잡해진다. 대신 모든 명령어를 **32비트**로 같게 하고 형식끼리 칸 자리를 최대한 비슷하게 맞췄다. 해독의 복잡함을 감수하고 길이를 통일한 타협이다. 세 원칙 앞 글자 「단·작·타」. [Ch2 p.23]

주소 지정 방식(addressing mode) 4가지:
즉시값(immediate): 피연산자 = 명령어 안의 상수 (addi, xori, srai)
레지스터(register): 피연산자 = 레지스터 (add, sub, xor)
베이스/변위(base or displacement): 메모리 주소 = 레지스터 + 상수 (lw, sw, jalr)
PC 상대(PC-relative): 분기 주소 = PC + 상수 (beq, bne, jal)
한 연산이 여러 방식을 쓸 수 있다(덧셈 = add·addi). 앞 글자 「즉·레·베·피」. [Ch2 p.49]

판단법: 값이나 주소를 무엇으로 만드나? 상수만 → 즉시값, 레지스터만 → 레지스터, 레지스터 + 상수 → 베이스, PC + 상수 → PC 상대. 주의: jalr은 이름에 jump가 있지만 주소를 레지스터 + 상수로 만드므로 **베이스**, PC 상대는 jal(과 조건 분기)이다.

2 · 핵심 정리

- 원칙 1: Simplicity favors regularity (단순성은 규칙성을 선호) → 산술 = 소스 2 + 목적지 1 [Ch2 p.9]
- 원칙 2: Smaller is faster (작을수록 빠르다) → 레지스터가 많으면 클럭 주기가 늘 수 있음 [Ch2 p.10]
- 원칙 3: Good design demands good compromises (좋은 설계에는 좋은 타협) → 형식 여럿, 모두 32비트, 최대한 비슷하게 [Ch2 p.23]
- 주소 지정 4: 즉시값(addi) · 레지스터(add) · 베이스/변위(lw, sw, jalr) · PC 상대(beq, bne, jal) [Ch2 p.49]
- 함정: jalr = 베이스 방식(레지스터 + 상수), jal = PC 상대

외우는 고리

설계 원칙 「단·작·타」: 단순(규칙) → 작게(빠름) → 타협(형식). 주소 지정 「즉·레·베·피」: 상수 → 레지스터 → 레지스터 + 상수 → PC + 상수

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

설계 원칙 1~3을 영어 문장으로 쓰시오.

답 보기

설계 원칙 3의 예를 한 줄로?

답 보기

레지스터가 많을수록 무조건 빠르지 않은 근거가 되는 설계 원칙은?

답 보기

주소 지정 방식: (a) jalr (b) bne (c) xori (d) sw (e) sub

답 보기

빈칸: 주소 지정 방식 = 즉시값 · 레지스터 · ( ) · PC 상대

답 보기

Ch2 · 9 약 7분

CISC·RISC, 수 표현, 번역

CISC vs RISC 비교 단답(상), 부호 표현 3가지, 번역 4단계 빈칸

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 명령어 집합을 만드는 두 방향(CISC·RISC), 음수를 비트로 나타내는 3가지 방법, C 프로그램이 실행되기까지의 번역 4단계를 묶은 문장 덩어리.

명령어 = 프로세서 동작 하나를 지시하는 기본 명령으로, 하드웨어와 소프트웨어의 경계다. ISA(instruction set architecture) = 프로세서가 이해하는 명령어 집합. 마이크로아키텍처 = 그 ISA를 설계 제약·목표 아래 실제로 구현한 것이고 소프트웨어에는 **드러나지 않는다**. 예: 파이프라이닝, 추측 실행(speculative execution), 메모리 접근 스케줄링, 산술 장치. 그래서 「추측 실행은 ISA 기능」은 X. [Ch2 p.3, 5]

CISC(예 x86): 명령어가 복잡하고 길이가 제각각(가변 길이)이며, CPU 안에서 마이크로코드(복잡한 명령어를 단순한 동작들로 풀어 놓은 것)로 바꿔 실행한다. 명령어 하나가 일을 많이 하니 장점은 어셈블리 프로그래밍이 쉽고(컴파일러 단순) 명령어 메모리를 적게 쓰는 것, 단점은 CPU 설계가 훨씬 어려운 것. **RISC**(ARM, MIPS, SPARC, RISC-V): 단순·규격화된 작은 집합이라 장점은 CPU 설계가 쉽고 클럭을 높일 수 있는 것, 단점은 같은 일에 명령어가 더 필요해 코드가 길어지고 메모리를 많이 쓰는 것. 현대 x86 대부분은 안에서 RISC 기법으로 구현한다. [Ch2 p.6–7]

부호 있는 수 표현 3가지: ① 부호-크기(sign and magnitude): 맨 앞(MSB) 비트만 부호로 ② 1의 보수: 음수 = 모든 비트 반전 ③ 2의 보수: 반전 + 1. 앞 둘은 +0과 −0이 두 개 생긴다(4비트 1의 보수 0000과 1111). 그래서 최종 채택은 2의 보수이고, 32비트 범위는 -2^{31} ~ $2^{31} - 1$, MSB가 1이면 음수다. 앞 글자 「부·1·2」. [Ch2 p.19]

2의 보수에서 부정(부호 반전) = ~x + 1(모든 비트 반전 후 1 더하기). 부호 확장(sign extension) = 적은 비트를 많은 비트로 늘릴 때 부호 비트를 왼쪽 빈자리에 복제하는 것이라 값이 그대로다. 예 8비트 1111 1010(−6) → 16비트 1111 1111 1111 1010(여전히 −6). 같은 생각으로 sra(산술 오른쪽 시프트)는 오른쪽으로 밀며 왼쪽 빈 칸을 부호 비트로 채운다(srl은 0으로). 예 1000 0100을 1칸 sra → 1100 0010. [Ch2 p.20, 25]

번역 4단계: **컴파일러**(C → 어셈블리) → **어셈블러**(어셈블리 → 기계어, 목적 파일 object file) → **링커**(여러 목적 파일을 합쳐 실행 파일로) → **로더**(실행 파일을 메모리에 올려 실행 시작). 앞 글자 「컴어링로」. 목적 파일을 만드는 단계는 어셈블러다. [Ch2 p.51]

2 · 핵심 정리

- CISC(x86): 복잡·가변 길이·마이크로코드 / 장점 어셈블리 쉬움·명령어 메모리 적음 / 단점 CPU 설계 어려움 [Ch2 p.6–7]
- RISC(ARM, MIPS, RISC-V): 단순·작은 집합 / 장점 CPU 설계 쉬움·높은 클럭 / 단점 코드 길어짐·메모리 많이 씀
- 부호 표현 3: 부호-크기 · 1의 보수 · 2의 보수(채택, $-2^{31} \sim 2^{31} - 1$) [Ch2 p.19]
- 부정 = ~x + 1, 부호 확장 = 부호 비트를 왼쪽에 복제(값 보존) [Ch2 p.20]
- 번역 4단계: 컴파일러 → 어셈블러 → 링커 → 로더 [Ch2 p.51]
- 마이크로아키텍처 = ISA의 구현, SW에 드러나지 않음(파이프라이닝, 추측 실행) [Ch2 p.5]

외우는 고리

부호 표현 「부·1·2」: 숫자가 커지는 순, 마지막 2가 채택. 번역 「컴어링로」: 컴파일 → 어셈블 → 링크 → 로드

---

4 떠올리기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

CISC의 장점 두 가지는?

답 보기

RISC의 단점 두 가지는?

답 보기

부호 있는 수 표현 3가지와 최종 채택은?

답 보기

8비트 1111 1101(−3)을 16비트로 부호 확장하면?

답 보기

빈칸: 컴파일러 → ( ) → ( ) → 로더. 목적 파일을 만드는 단계는?

답 보기

8비트 1100 0011을 sra로 1칸 시프트하면?

답 보기

Ch3 · 1 약 9분

오버플로 판정

map 상: 오버플로 조건 O/X·빈칸 단골, 뺄셈은 덧셈으로 바꿔 판정

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 오버플로는 계산 결과가 정해진 비트 수에 들어가지 않는 것이다. 2의 보수에서는 「같은 부호끼리 더했는데 결과 부호가 바뀌었는가」로 판정한다.

오버플로(overflow) = 결과를 정해진 비트 수의 하드웨어로 표현할 수 없는 것이다. 8비트 2의 보수는 −128 ~ 127만 담는다. 양수 + 양수의 진짜 답이 127을 넘으면 8비트에는 진짜 답 − 256이 남아 MSB가 1, 즉 음수로 읽힌다. 음수 + 음수가 −128보다 작으면 진짜 답 + 256이 남아 양수로 읽힌다. 그래서 2의 보수에서는 결과 부호가 뒤집혀 보인다.

예: 100 + 100 = 200 → 0110 0100 + 0110 0100 = 1100 1000. MSB 1이라 −56(200 − 256)으로 읽힌다. 양수 + 양수 = 음수, 기호로 **+ + + = −**. 반대로 (−100) + (−100) = −200은 8비트에 56(−200 + 256)이 남는다: **− + − = +**. 이 두 줄이 덧셈의 오버플로 경우다.

생기지 않는 경우와 뺄셈: 부호가 **다른** 두 수를 더하면 합이 두 수 사이에 있어(−10 + 4 = −6) 범위를 벗어날 수 없다. 뺄셈은 A − B = A + (−B)로 바꿔 본다(−B = ~B + 1). 양수 − 음수는 양수 + 양수가 되어 **+ − − = −**(결과가 음수면 오버플로), 음수 − 양수는 음수 + 음수가 되어 **− − + = +**. 부호가 **같은** 두 수의 뺄셈은 부호가 다른 덧셈이 되므로 생기지 않는다.

자리올림(carry)은 오버플로가 아니다: 8비트 덧셈기는 결과를 8비트만 남기고, 맨 왼쪽 밖으로 나간 9번째 비트는 **버린다**. 2의 보수는 버려야 정답이 되도록 만든 표현이다: −1 + 1 = 1111 1111 + 0000 0001 = 1 0000 0000 → 9번째 1을 버리면 0000 0000 = 0. 그래서 판정은 올림이 아니라 피연산자 부호와 결과 부호로만 한다.

무부호(unsigned, 8비트면 0 ~ 255, 주소 등): 덧셈 A + B = S가 넘치면 진짜 답 − 256이 남아 원래 수보다 작아진다(200 + 100 = 300 → 44). 그래서 **S < A 또는 S < B**면 오버플로. 뺄셈 A − B = S가 0 밑으로 가면 진짜 답 + 256이 남아 빼기 전보다 커진다(50 − 100 → 206). 그래서 **S > A**면 오버플로. 언어: C/C++·Java는 오버플로를 **무시**하고, Ada·Fortran은 **통보**를 요구한다.

2 · 핵심 정리

- 오버플로 = 결과를 정해진 비트 수로 표현할 수 없음, 2의 보수에서는 결과 부호가 뒤집혀 보임 [Ch3 p.4–5]
- 생기는 4경우: + + + = −, − + − = +, + − − = −, − − + = + [Ch3 p.5]
- 생기지 않음: 부호가 다른 두 수의 덧셈, 부호가 같은 두 수의 뺄셈 [Ch3 p.4]
- 맨 왼쪽 밖 자리올림(carry)은 버린다, 올림만으로는 오버플로 아님
- 무부호: 덧셈 S < A 또는 S < B, 뺄셈 S > A면 오버플로. C/C++·Java 무시, Ada·Fortran 통보 [Ch3 p.6]

외우는 고리

같은 부호끼리 더했는데 부호가 바뀌면 오버플로. 뺄셈은 A + (−B)로 바꾸고 같은 질문

---

3 풀이 예

푸는 순서

① 두 피연산자의 MSB로 부호를 읽는다(1이면 음수)

② 뺄셈이면 A − B = A + (−B)로 바꾼다(−B = ~B + 1)

③ 바꾼 덧셈의 두 부호가 다르면 끝: 오버플로 없음

④ 같으면 더해서 8비트만 남기고, 결과 MSB가 피연산자 부호와 다르면 오버플로

8비트 2의 보수 1001 0000 + 1010 0000의 결과와 오버플로 여부

① MSB 둘 다 1 → 음수 + 음수(−112, −96)

② 덧셈이라 그대로

③ 부호가 같으니 오버플로를 의심

④ 1001 0000 + 1010 0000 = 1 0011 0000 → 9번째 비트 버림 → 0011 0000(+48). − + − = + → 오버플로 (진짜 답 −208 < −128)

답: **0011 0000, 오버플로**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

8비트 2의 보수 0110 0000 − 1110 0000의 결과와 오버플로 여부

① 0110 0000 = +96, 1110 0000 = −32

② −(−32) = +32 = 0010 0000 → 0110 0000 + 0010 0000

③ 양수 + 양수라 오버플로를 의심

④ 0110 0000 + 0010 0000 = 1000 0000(MSB 1, −128로 읽힘) → + − − = − → 오버플로(진짜 답 128 > 127)

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

2의 보수에서 오버플로가 생기지 않는 두 경우는?

답 보기

8비트 2의 보수 0111 0000 + 0011 0000의 8비트 결과와 오버플로 여부는?

답 보기

8비트 2의 보수 1000 0000 − 0000 0001의 8비트 결과와 오버플로 여부는?

답 보기

O/X: 2의 보수 덧셈에서 맨 왼쪽 밖으로 자리올림이 나오면 오버플로다.

답 보기

무부호 덧셈 A + B = S에서 오버플로 조건은? (8비트 1111 0000 + 0010 0000은?)

답 보기

오버플로를 무시하는 언어와 통보를 요구하는 언어는?

답 보기

Ch3 · 2 약 10분

곱셈기 1판 vs 개선판

map 상: 곱셈 반복 표 빈칸 채우기

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 곱셈기는 손 곱셈을 「LSB 검사 → 더하기 → 시프트」로 한 비트씩 n번 반복하는 하드웨어다. 처음 설계(1판)와 레지스터·ALU 크기를 줄인 개선판이 있다.

A × B에서 곱해지는 수 A가 **피승수**(multiplicand), 곱하는 수 B가 **승수**(multiplier), 결과가 **곱**(product)이다. 2진 손 곱셈은 승수 비트를 오른쪽부터 보며, 1인 자리마다 피승수를 그 자리만큼 왼쪽으로 민 값을 더한다. 예 0010 × 0011: 승수 0번 비트 1 → 0010, 1번 비트 1 → 0100(1칸 민 것), 합 0110 = 6. 곱의 길이 = 피승수 길이 + 승수 길이라 32 × 32는 **64비트**가 필요하다. **ALU**(산술 논리 장치) = 덧셈·뺄셈 같은 계산을 하는 회로다.

1판(32비트 곱셈): 승수 레지스터 **32비트**(오른쪽 시프트), 피승수 레지스터 **64비트**(오른쪽 절반에서 시작, 왼쪽 시프트), ALU **64비트**, 곱 레지스터 64비트(처음 0). 한 반복 = **Test-Add-Shift**: ① 승수 LSB가 1이면 곱 = 곱 + 피승수 ② 피승수 왼쪽 1칸 ③ 승수 오른쪽 1칸. 피승수를 왼쪽으로 미는 이유는 손 곱셈에서 다음 자리 복사본이 한 칸 더 왼쪽에 놓이기 때문이고, 최대 32칸 밀리니 64비트가 필요하다. 승수를 오른쪽으로 미는 이유는 다음에 볼 비트가 늘 LSB에 오게 해 LSB 한 칸만 검사하려는 것이다.

1판 표 읽기(0010 × 0011, 각 반복이 끝난 뒤 값):
반복 0(초기): 승수 0011, 피승수 0000 0010, 곱 0000 0000
반복 1: LSB 1 → 곱 0000 0010, 피승수 0000 0100, 승수 0001
반복 2: LSB 1 → 곱 0000 0010 + 0000 0100 = 0000 0110, 피승수 0000 1000, 승수 0000
반복 3·4: LSB 0 → 더하지 않고 시프트만(피승수 0001 0000 → 0010 0000), 곱 0000 0110 = 6 그대로
승수 비트가 4개라 4번(n번)에서 끝난다.

개선판: 피승수를 왼쪽으로 미는 대신 곱을 **오른쪽**으로 밀어도 둘의 상대 위치가 같아 결과가 같다. 그러면 피승수가 움직이지 않으니 피승수 레지스터·ALU가 **32비트**면 된다. 승수 레지스터도 없애고 승수를 곱 레지스터(64비트)의 **오른쪽 절반**에 넣는다(왼쪽 절반은 0). 한 반복 = ① 곱 레지스터 LSB가 1이면 왼쪽 절반 = 왼쪽 절반 + 피승수 ② 곱 레지스터 전체를 오른쪽 1칸(왼쪽 빈칸 0). 다 쓴 승수 비트는 오른쪽 끝으로 빠지고 다음 승수 비트가 LSB에 온다.

개선판 표(같은 0010 × 0011, 곱 레지스터):
초기: 0000 0011(왼쪽 0000 + 오른쪽 승수 0011)
반복 1: LSB 1 → 왼쪽 0000 + 0010 = 0010 → 0010 0011 → 오른쪽 1칸 → 0001 0001

반복 2: LSB 1 → 왼쪽 0001 + 0010 = 0011 → 0011 0001 → 0001 1000

반복 3: LSB 0 → 시프트만 → 0000 1100 / 반복 4: → 0000 0110 = 6

순서 주의: 더한 값을 왼쪽 절반에 쓴 다음에 민다. 왼쪽 절반 덧셈에서 4비트를 넘는 올림이 생기면 그 비트는 시프트 때 맨 왼쪽 빈칸으로 들어간다(교재 기준).

부호 있는 곱셈: 두 수를 양수로 바꿔 크기만 곱하고, 두 부호가 다르면 마지막에 곱을 **부정**(부호 반전)한다. 32비트 중 맨 앞 부호 비트는 따로 처리하니 크기 31비트만 곱해 **31번** 반복한다. **빠른 곱셈** = 덧셈기 여러 개를 병렬 트리로 놓아 한꺼번에 더하는 방식(비용과 성능의 맞바꿈). 주의: 곱셈은 n번, 나눗셈은 n+1번 반복한다.

2 · 핵심 정리

- 곱의 길이 = 피승수 길이 + 승수 길이 (32 × 32 → 64 비트) [Ch3 p.7]
- 1판: 승수 32 비트 오른쪽 시프트, 피승수 64 비트 왼쪽 시프트, ALU 64비트, 곱 64비트(처음 0) [Ch3 p.8–9]
- 1판 한 반복 = Test-Add-Shift : 승수 LSB 1이면 곱 += 피승수 → 피승수 ← 1칸 → 승수 → 1칸
- 개선판: 피승수·ALU 32 비트, 승수 레지스터 없음(곱 레지스터 오른쪽 절반), 곱 레지스터 LSB 1이면 왼쪽 절반 += 피승수 → 곱 전체 오른쪽 시프트 [Ch3 p.10–11]
- 반복 n번(4비트 4번). 부호 있는 곱셈 = 양수로 31 번 후 부호 다르면 부정, 빠른 곱셈 = 덧셈기 병렬 트리 [Ch3 p.12]

외우는 고리

1판은 피승수가 왼쪽으로 걸어가고, 개선판은 곱이 오른쪽으로 걸어온다(상대 위치는 같다)

---

3 풀이 예

푸는 순서

① 초기값: 1판 = 승수, 피승수(오른쪽 절반), 곱 0 / 개선판 = 곱 레지스터 왼쪽 0 + 오른쪽 승수

② LSB 검사: 1판은 승수 LSB, 개선판은 곱 레지스터 LSB

③ 1이면 더하기: 1판 곱 += 피승수, 개선판 왼쪽 절반 += 피승수 (0이면 건너뜀)

④ 시프트: 1판 피승수 ← 1칸·승수 → 1칸 / 개선판 곱 레지스터 전체 → 1칸

⑤ n번 반복 후 곱을 10진으로 검산

슬라이드 예 $0010_2 \times 0011_2$(2 × 3)을 1판으로. 각 반복 뒤 승수·피승수·곱

① 초기: 승수 0011, 피승수 0000 0010, 곱 0000 0000

② 반복 1: LSB 1 → 곱 0000 0010, 피승수 0000 0100, 승수 0001

③ 반복 2: LSB 1 → 곱 0000 0010 + 0000 0100 = 0000 0110, 피승수 0000 1000, 승수 0000

④ 반복 3·4: LSB 0 → 시프트만(피승수 0001 0000 → 0010 0000), 곱 그대로

⑤ 개선판으로는 0000 0011 → 0001 0001 → 0001 1000 → 0000 1100 → 0000 0110 [Ch3 p.11]

답: **곱 0000 0110 = 6**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

1판으로 $0011_2 \times 0101_2$(피승수 3, 승수 5). 각 반복 뒤 곱

① 초기: 승수 0101, 피승수 0000 0011, 곱 0000 0000

② 반복 1: LSB 1 → 곱 0000 0011, 피승수 0000 0110, 승수 0010

③ 반복 2: LSB 0 → 곱 그대로, 피승수 0000 1100, 승수 0001

④ 반복 3: LSB 1 → 곱 0000 0011 + 0000 1100 = 0000 1111, 피승수 0001 1000, 승수 0000

⑤ 반복 4: LSB 0 → 시프트만, 곱 0000 1111

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

곱셈기 1판의 승수·피승수·ALU·곱 레지스터 비트 수는? (32비트 곱셈)

답 보기

개선판 곱셈기에서 승수는 어디에 넣고, 피승수·ALU는 몇 비트인가?

답 보기

곱셈기 1판 한 번의 반복 순서는?

답 보기

개선판으로 $0011_2 \times 0101_2$(3 × 5). 반복 1이 끝난 뒤 곱 레지스터는?

답 보기

부호 있는 32비트 곱셈은 몇 번 반복하고 마지막에 무엇을 하나?

답 보기

Ch3 · 3 약 9분

mul·mulh·mulhu·mulhsu와 upper·lower

기출 21-2: 부호 없는/있는/부호 다른 곱셈 upper·lower 빈칸

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 32 × 32 곱은 64비트라 레지스터 하나에 다 들어가지 않는다. 그래서 RISC-V는 아래 32비트(lower)와 위 32비트(upper)를 따로 주는 명령어를 두고, 시험은 작은 비트 수로 upper·lower 표를 채우게 한다.

lower = 곱의 아래(오른쪽) 절반, **upper** = 위(왼쪽) 절반이다. `mul`은 lower를 준다. lower는 두 수를 부호 있게 읽든 무부호로 읽든 같아서 명령어가 하나면 된다. upper는 해석에 따라 달라서 셋이다: `mulh`(둘 다 부호 있음), `mulhu`(둘 다 무부호), `mulhsu`(부호 있음 × 무부호, rs1이 부호 있음·rs2가 무부호, PDF 외 보충). 이름 풀이: h = high(위), u = unsigned, su = signed × unsigned. 함정: mulhu는 아래가 아니라 위 32비트다.

같은 n비트 패턴을 두 가지로 읽는다. **무부호 값**은 자릿값 합이다. **부호 값**은 맨 앞이 1이면 무부호 값 − 2^n, 맨 앞이 0이면 무부호 값과 같다. 이유: 2의 보수에서 −a의 패턴을 무부호로 읽으면 2^n − a이기 때문이다. 3비트 예: 101_2 = 무부호 5, 부호 5 − 8 = −3. 110_2 = 무부호 6, 부호 6 − 8 = −2.

푸는 순서: ① 모드별 10진 곱을 구한다. Unsigned = 무부호 × 무부호, Signed = 부호 × 부호, Signed-Unsigned = A(부호) × B(무부호). ② 곱을 2n비트로 쓴다(곱의 길이 = 두 길이의 합이라 다 들어간다). 양수는 자릿값으로 분해하고 왼쪽을 0으로 채운다. 음수는 2n비트 2의 보수라서 **2^{2n} + 곱**을 2진으로 쓴다(6비트면 64 + 곱). 예 −15 → 64 − 15 = 49 = 32 + 16 + 1 = 110001_2. ③ 왼쪽 n비트가 Upper, 오른쪽 n비트가 Lower.

슬라이드 예 101_2 × 101_2(3비트, 곱 6비트):
Unsigned 5 × 5 = 25 = 011001_2 → Upper 011, Lower 001
Signed (−3) × (−3) = 9 = 001001_2 → Upper 001, Lower 001
Signed-Unsigned (−3) × 5 = −15 → 110001_2 → Upper 110, Lower 001
Upper는 셋이 다르고 Lower는 셋 다 001이다. 32비트라면 Upper는 차례로 mulhu, mulh, mulhsu, Lower는 mul의 결과다.

Lower가 같은 이유: 부호 값과 무부호 값은 2^n(3비트면 8)만큼 차이 나서, 모드별 곱은 8의 배수만큼만 다르다. 8의 배수를 더하거나 빼도 2진 아래 3자리는 그대로다(10진에서 1000의 배수를 더해도 아래 세 자리가 그대로인 것과 같음). 예 25 − 9 = 16 = 2 × 8. 32비트에서는 2^{32}의 배수만큼 달라 아래 32비트가 같다. 그래서 풀고 나서 Lower가 세 줄 모두 같은지로 검산한다.

2 · 핵심 정리

- mul = 아래 32비트(부호 해석과 상관없이 같음) [Ch3 p.13]
- mulh = 위 32비트, 둘 다 부호 / mulhu = 위, 둘 다 무부호 / mulhsu = 위, 부호 × 무부호(rs1 부호, rs2 무부호)
- n비트 읽기: 무부호 값, 부호 값 = 무부호 값 − 2^n(맨 앞 1일 때). 101_2 = 5 / −3
- 음수 곱은 2n비트에서 2^{2n} + 곱을 2진으로(6비트면 64 + 곱)
- 슬라이드 $101_2 \times 101_2$: Unsigned 25 = 011|001, Signed 9 = 001|001, Signed-Unsigned −15 = 110|001 → Lower는 셋 다 같음

외우는 고리

h = high(위), u = unsigned, su = signed × unsigned. 아래는 mul 하나

---

3 풀이 예

푸는 순서

① 두 수를 무부호 값과 부호 값(맨 앞 1이면 − 2^n)으로 읽는다

② 모드별 곱: Unsigned = 무 × 무, Signed = 부 × 부, Signed-Unsigned = A 부 × B 무

③ 2n비트로 쓴다: 양수는 자릿값 분해, 음수는 2^{2n} + 곱

④ 왼쪽 n비트 = Upper, 오른쪽 n비트 = Lower

⑤ 검산: Lower가 세 줄 모두 같아야 한다

3비트 A = 110_2, B = 101_2. 세 모드의 A × B(6비트)와 Upper, Lower

① 110_2 = 무부호 6, 부호 6 − 8 = −2. 101_2 = 무부호 5, 부호 −3

② Unsigned 6 × 5 = 30, Signed (−2) × (−3) = 6, Signed-Unsigned (−2) × 5 = −10

③ 30 = 011110_2, 6 = 000110_2, −10 → 64 − 10 = 54 = 110110_2

④ 011|110, 000|110, 110|110

⑤ Lower 110_2 공통. 32비트였다면 Upper는 mulhu, mulh, mulhsu, Lower는 mul

답: **Unsigned 30: 011 / 110, Signed 6: 000 / 110, Signed-Unsigned −10: 110 / 110**

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

3비트 A = 100_2, B = 110_2. 세 모드의 A × B(6비트)와 Upper, Lower

① 100_2 = 무부호 4, 부호 −4. 110_2 = 무부호 6, 부호 −2

② Unsigned 24, Signed 8, Signed-Unsigned (−4) × 6 = −24

③ 24 = 011000_2, 8 = 001000_2, −24 → 64 − 24 = 40 = 101000_2

④ 011|000, 001|000, 101|000

⑤ Lower 000_2 공통

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

32 × 32 곱의 아래 32비트를 주는 명령어는? 부호에 따라 다른가?

답 보기

둘 다 부호 있는 수일 때 위 32비트 명령어는?

답 보기

둘 다 무부호일 때 위 32비트 명령어는?

답 보기

하나는 부호 있음·하나는 무부호일 때 위 32비트 명령어는?

답 보기

3비트 A = 110_2, B = 101_2. Signed-Unsigned(A 부호, B 무부호)의 A × B와 Upper, Lower는?

답 보기

Ch3 · 4 약 10분

나눗셈기와 부호 있는 나눗셈

map 상: 나눗셈 반복 표 빈칸, 부호 있는 나눗셈 몫·나머지 함정

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 나눗셈기는 손 나눗셈을 「일단 빼 보고, 음수면 되돌린다」로 한 자리씩 n+1번 반복하는 하드웨어다. 부호 있는 나눗셈은 절댓값으로 나눈 뒤 부호만 붙인다.

A ÷ B에서 나눠지는 수가 **피제수**(dividend), 나누는 수가 **제수**(divisor), 결과가 **몫**(quotient)과 **나머지**(remainder)다. 항상 피제수 = 몫 × 제수 + 나머지, 제수 ≠ 0. 손 나눗셈은 제수를 높은 자리에 대 보고, 들어가면 몫 자리에 1을 쓰고 빼고, 안 들어가면 0을 쓴 뒤 한 자리 내려간다. 나눗셈기 1판은 곱셈기 1판과 같은 하드웨어를 쓰되 시프트 방향이 반대이고 더하기 대신 뺄셈을 한다.

초기값: 제수를 레지스터 **왼쪽 절반**에 넣는다(4비트 0010 → 0010 0000 = 2×2^4, 가장 높은 자리부터 대 보려고). 나머지 레지스터에는 피제수, 몫은 0. 한 반복 = **Subtract-Test-Shift**: ① 나머지 = 나머지 − 제수 ② 결과가 음수(MSB 1)면 제수를 다시 더해 **복원**하고 몫을 왼쪽 1칸·끝 비트 0, 0 이상이면 복원 없이 몫을 왼쪽 1칸·끝 비트 1 ③ 제수를 오른쪽 1칸(÷ 2, 한 자리 아래로). 손으로는 10진으로 「나머지 < 제수」면 음수라고 바로 안다.

왜 그렇게 하나: 빼 본 결과가 음수면 제수가 안 들어간 것이라 뺀 것을 취소한다(복원). 몫 비트는 높은 자리부터 나오므로 새 비트를 오른쪽 끝에 붙이며 앞 비트를 왼쪽으로 민다. 반복 횟수는 **n+1**번: 제수가 $\times 2^4$, $\times 2^3$, $\times 2^2$, $\times 2^1$, $\times 2^0$ 자리 5개를 하나씩 시험하므로 4비트면 5번, 32비트면 33번이다.

예 7 ÷ 2 (0000 0111 ÷ 0010, 각 반복이 끝난 뒤 몫 / 제수 / 나머지):
0 초기: 0000 / 0010 0000 / 0000 0111
1: 7 − 32 음수 → 복원, 끝 0: 0000 / 0001 0000 / 0000 0111
2: 7 − 16 음수 → 복원, 끝 0: 0000 / 0000 1000 / 0000 0111
3: 7 − 8 음수 → 복원, 끝 0: 0000 / 0000 0100 / 0000 0111
4: 7 − 4 = 3 ≥ 0 → 끝 1: 0001 / 0000 0010 / 0000 0011
5: 3 − 2 = 1 ≥ 0 → 끝 1: 0011 / 0000 0001 / 0000 0001
몫 0011 = 3, 나머지 1. 검산 3 × 2 + 1 = 7.

부호 있는 나눗셈: 절댓값으로 나눈 뒤, 피제수와 제수 부호가 다르면 **몫을 부정**하고, **나머지 부호는 피제수 부호**를 따른다. 7 ÷ 2 = 3 … 1이므로 −7 ÷ 2 = −3 … −1(−4 … 1이 아님), 7 ÷ −2 = −3 … 1, −7 ÷ −2 = 3 … −1. 검산 −3 × 2 − 1 = −7. 이 규칙이면 부호가 어떻게 섞여도 몫·나머지의 크기는 같고 부호만 바뀐다.

빠르게 하기: 곱셈은 덧셈기를 병렬로 놓을 수 있지만, 나눗셈은 다음 뺄셈이 앞 나머지의 부호에 달린 조건부라 **병렬화할 수 없다**. 대신 **SRT** 나눗셈은 표를 보고 한 단계에 몫 비트 여러 개를 예측한다. RISC-V 명령어: `div`(몫), `rem`(나머지), 끝에 u가 붙은 `divu`·`remu`는 무부호판이다.

2 · 핵심 정리

- 피제수 = 몫 × 제수 + 나머지, 제수 ≠ 0. 1판 = 곱셈기 1판과 같은 HW, 시프트 방향 반대, 뺄셈 [Ch3 p.14–15]
- 한 반복 = Subtract-Test-Shift: ① 나머지 − 제수 ② 음수면 제수를 다시 더해 복원·몫 ← 1칸 끝 0 / 0 이상이면 몫 ← 1칸 끝 1 ③ 제수 → 1칸
- 초기: 제수는 왼쪽 절반, 나머지 레지스터 = 피제수. 반복 n+1번(4비트 5번, 32비트 33번) [Ch3 p.16]
- 부호 규칙: 절댓값으로 나눔 → 부호가 다르면 몫 부정 → 나머지 부호 = 피제수 부호 (−7 ÷ 2 = −3 … −1) [Ch3 p.19]
- 병렬화 불가(뺄셈이 앞 나머지 부호에 달린 조건부) → SRT(표로 몫 여러 비트 예측). 명령어 div·rem, 무부호 divu·remu [Ch3 p.20]

외우는 고리

빼 보고(Subtract) → 음수면 되돌리고(Test) → 제수를 오른쪽으로(Shift). 몫은 부호 곱, 나머지는 피제수 따라

---

3 풀이 예

푸는 순서

① 나머지 − 제수 (10진으로: 나머지 < 제수면 음수)

② 음수면 복원(제수 다시 더함), 몫 왼쪽 시프트·끝 0 / 0 이상이면 나머지 갱신, 몫 왼쪽 시프트·끝 1

③ 제수 오른쪽 1칸

④ n+1번 반복 후 검산: 몫 × 제수 + 나머지 = 피제수

슬라이드 $0000\ 0111_2 \div 0010_2$(7 ÷ 2). 반복 4 시작 때 몫 0000, 제수 0000 0100, 나머지 0000 0111. 반복 4 한 줄과 최종값

① 7 − 4 = 3 = 0000 0011, MSB 0 → 0 이상

② 복원 안 함, 나머지 0000 0011, 몫 0000 → 0001

③ 제수 0000 0100 → 0000 0010

④ 반복 5: 3 − 2 = 1 ≥ 0 → 몫 0011, 나머지 0000 0001, 제수 0000 0001. 검산 3 × 2 + 1 = 7 [Ch3 p.16]

답: **반복 4 뒤 몫 0001, 제수 0000 0010, 나머지 0000 0011 / 최종 몫 0011_2 = 3, 나머지 1**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

$0000\ 1011_2 \div 0011_2$(11 ÷ 3). 반복 4 시작 때 몫 0000, 제수 0000 0110, 나머지 0000 1011. 반복 4 한 줄과 최종값

① 11 − 6 = 5 = 0000 0101, MSB 0 → 0 이상

② 복원 안 함, 나머지 0000 0101, 몫 0000 → 0001

③ 제수 0000 0110 → 0000 0011

④ 반복 5: 5 − 3 = 2 ≥ 0 → 몫 0011, 나머지 0000 0010, 제수 0000 0001. 검산 3 × 3 + 2 = 11

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

나눗셈기 1판 한 번의 반복 이름과 순서는?

답 보기

나눗셈기 1판으로 4비트 나눗셈을 하면 몇 번 반복하나? 32비트는?

답 보기

빼기 결과가 음수(MSB 1)이면 무엇을 하고 몫 끝 비트는?

답 보기

슬라이드 규칙으로 −13 ÷ 4, 13 ÷ −4, −13 ÷ −4의 몫·나머지는?

답 보기

O/X: 나눗셈도 곱셈처럼 여러 뺄셈기를 병렬로 써서 빠르게 할 수 있다.

답 보기

RISC-V에서 몫과 나머지를 구하는 명령어는? 무부호판은?

답 보기

Ch3 · 5 약 11분

IEEE 754 형식과 10진 → 16진 변환

map 상: 변환 계산 1순위. 기출 26-1: 10진 → 2진, 16비트(1-5-10) 표현

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: IEEE 754는 실수를 $1.xxx_2 \times 2^n$ 꼴로 바꿔 부호·지수·가수 세 칸에 저장하는 표준이다. 시험은 10진수를 이 비트로 바꿔 16진수로 쓰게 한다.

정규화(normalized) = 소수점 왼쪽에 0이 아닌 숫자가 딱 한 자리인 과학적 표기(1.0×10^{-9}). 2진에서는 그 숫자가 늘 1이라 $1.xxx_2 \times 2^n$ 꼴이 된다. **부동소수점**(floating point) = 소수점 위치를 고정하지 않고 지수 n으로 나타내는 수 표현이다. xxx 부분이 **가수**(fraction), n이 **지수**(exponent)다. **IEEE 754**는 이것을 비트로 저장하는 표준으로, 32비트 **단정도**(single)와 64비트 **배정도**(double)가 있다.

칸 나누기(부호 / 지수 / 가수, bias):
단정도 1 / 8 / 23, bias 127 (합 32)
배정도 1 / 11 / 52, bias 1023 (합 64)
16비트 1 / 5 / 10, bias 15 (합 16, 슬라이드에 없고 26-1에 나옴, PDF 외 보충)
bias = 지수를 저장할 때 더해 두는 고정값이다. 저장된 값을 **지수 칸**, 원래 n을 **실제 지수**라 부른다: 지수 칸 = 실제 지수 + bias. 더해 두면 지수 칸에 음수가 없어진다. bias = $2^{k-1} - 1$(k = 지수 칸 비트 수: 8 → 127, 5 → 15).

숨은 비트(hidden bit): 정규화된 2진수는 소수점 앞이 항상 1이라 저장하지 않고, 가수 칸에는 소수점 뒤 비트만 넣는다. 그래서 단정도 정밀도는 1 + 23 = 24비트다. 저장된 비트의 값 = $(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{지수 칸} - \text{bias})}$. 지수 칸 전부 0과 전부 1(단정도 0·255, 16비트 0·31)은 NaN(0/0 같은 잘못된 연산) 같은 특수 기호용 **예약값**이다. 그래서 정규화 수의 지수 칸은 1 ~ 254, 실제 지수는 −126 ~ 127이다.

10진 → IEEE 754 순서: ① 음수면 S = 1, 크기만 다룬다 ② 크기를 2진으로: 정수 부분은 자릿값 분해(9 = 8 + 1 = 1001_2), 소수 부분은 ×2를 반복해 나온 정수 부분(0 또는 1)을 위 → 아래로 읽는다 ③ 소수점을 맨 앞 1 바로 뒤로 옮겨 $1.xxx_2 \times 2^n$: 왼쪽으로 k칸이면 n = +k, 오른쪽으로 k칸이면 n = −k(왼쪽 1칸 = 값 ÷ 2라서 × 2를 붙여 보상) ④ 지수 칸 = n + bias를 8비트(16비트는 5비트) 2진으로, 자리가 모자라면 왼쪽에 0 ⑤ 가수 칸 = 소수점 뒤 비트를 왼쪽부터 넣고 남은 칸은 오른쪽에 0 ⑥ S | 지수 | 가수를 전부 이어 붙인 뒤 왼쪽부터 4비트씩 끊어 16진. 칸별로 끊으면 안 된다(S 1 + 지수 8 = 9비트라 4비트 경계와 안 맞음).

예(슬라이드) −0.75: ① S = 1 ② 0.75 × 2 = 1.5 → 1(남은 0.5), 0.5 × 2 = 1.0 → 1(끝) → 0.11_2 ③ 오른쪽 1칸 → $1.1_2 \times 2^{-1}$ ④ 지수 칸 −1 + 127 = 126 = 0111 1110 ⑤ 가수 칸 1 뒤에 0 22개 ⑥ 1 01111110 100…0 → 1011 1111 0100 0000 0…0 = **0xBF400000**. 16비트라면 지수 칸 −1 + 15 = 14 = 01110 → 1 01110 1000000000 → 1011 1010 0000 0000 = 0xBA00.

주의: 가수 칸에 숨은 1을 넣지 않는다(−0.75는 1000…, 1100…이 아님). 지수 칸에 실제 지수를 2의 보수로 넣지 않는다(−1 → 1111 1111은 틀림). 16비트에 bias 127을 쓰지 않는다.

2 · 핵심 정리

- 단정도 1 / 8 / 23, bias 127 / 배정도 1 / 11 / 52, bias 1023 [Ch3 p.27–28]
- 16비트 1 / 5 / 10, bias 15 (PDF 외 보충, bias = $2^{k-1} - 1$)
- 값 = $(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{지수 칸} - \text{bias})}$, 맨 앞 1은 숨은 비트(정밀도 24비트) [Ch3 p.28]
- 지수 칸 0과 255(16비트 0과 31)은 예약 → 정규화 지수 칸 1 ~ 254, 실제 −126 ~ 127 [Ch3 p.30]
- $-0.75 = -1.1_2 \times 2^{-1}$ → S 1, 지수 칸 126, 가수 100…0 → 0xBF400000 [Ch3 p.31]

외우는 고리

단정도 1·8·23(127), 배정도 1·11·52(1023), 16비트 1·5·10(15). 칸 합 32·64·16

---

3 풀이 예

푸는 순서

① 부호를 뗀다: 음수면 S = 1

② 크기를 2진으로: 정수 부분 ÷2(또는 자릿값 분해), 소수 부분 ×2 반복해 정수 부분을 위 → 아래로

③ 정규화 $1.xxx_2 \times 2^n$: 소수점을 왼쪽 k칸이면 n = +k, 오른쪽 k칸이면 n = −k

④ 지수 칸 = n + bias(127, 16비트 15)를 8비트(16비트 5비트) 2진으로

⑤ 가수 칸 = 소수점 뒤 비트를 왼쪽부터, 남은 칸은 0 (숨은 1 빼고)

⑥ S | 지수 | 가수를 전부 이어 붙인 뒤 4비트씩 끊어 16진

−9.625를 단정도와 16비트(1-5-10) 16진수로

① 음수 → S = 1, 크기 9.625

② 9 = 1001_2, 0.625 → 1, 0, 1 → 0.101_2 → 1001.101_2

③ 왼쪽 3칸 → $1.001101_2 \times 2^3$

④ 32비트 지수 칸 3 + 127 = 130 = 1000 0010 / 16비트 3 + 15 = 18 = 10010

⑤ 가수 001101 뒤에 0 (23비트 / 10비트)

⑥ 1 10000010 0011010...0 → 1100 0001 0001 1010 0...0 = 0xC11A0000 / 1 10010 0011010000 → 1100 1000 1101 0000 = 0xC8D0

답: **0xC11A0000, 16비트 0xC8D0**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

−5.75를 단정도와 16비트(1-5-10) 16진수로

① 음수 → S = 1, 크기 5.75

② 5 = 101_2, 0.75 → 1, 1 → 0.11_2 → 101.11_2

③ 왼쪽 2칸 → $1.0111_2 \times 2^2$

④ 32비트 지수 칸 2 + 127 = 129 = 1000 0001 / 16비트 2 + 15 = 17 = 10001

⑤ 가수 0111 뒤에 0

⑥ 1 10000001 0111...0 → 1100 0000 1011 1000 0...0 = 0xC0B80000 / 1 10001 0111000000 → 1100 0101 1100 0000 = 0xC5C0

답 보기

떠올리기 카드 7장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

IEEE 754 단정도의 부호·지수·가수 비트 수와 bias는?

답 보기

IEEE 754 배정도의 부호·지수·가수 비트 수와 bias는?

답 보기

16비트 부동소수점(26-1 기출) 칸 수와 bias는?

답 보기

−0.75를 IEEE 754 단정도 16진수로?

답 보기

0.375를 IEEE 754 단정도 16진수로?

답 보기

0.375를 16비트(1-5-10) 16진수로?

답 보기

O/X: 단정도 가수 칸 23비트에는 정규화된 수의 맨 앞 1도 들어간다.

답 보기

Ch3 · 6 약 6분

IEEE 754 → 10진 역변환

map 상: 16진수 → 10진 변환, 지수 칸 값 계산

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 16진수로 주어진 IEEE 754 비트를 거꾸로 풀어 10진수를 구한다. 앞 덩어리 변환을 반대 방향으로 하는 것이다.

순서: ① 16진 한 자리를 4비트로 바꾼다(C = 1100, 4 = 0100) ② 앞에서부터 1 | 8 | 23(16비트는 1 | 5 | 10)으로 자른다 ③ 실제 지수 = 지수 칸 − bias(127, 16비트 15) ④ 값 = $(-1)^S \times 1.\text{가수}_2 \times 2^{\text{실제 지수}}$. S = 0이면 양수, 1이면 음수. 가수 앞에 숨은 1을 다시 붙이는 것을 잊지 않는다.

2^e를 곱하는 계산은 소수점 옮기기다: 실제 지수가 +e면 소수점을 **오른쪽**으로 e칸, −e면 **왼쪽**으로 e칸 옮기고(빈자리는 0) 자릿값을 더한다. 예 $1.01_2 \times 2^2 = 101_2 = 5$, $1.1_2 \times 2^{-1} = 0.11_2 = 0.5 + 0.25 = 0.75$.

예 단정도 0x3F400000: ① 0011 1111 0100 0000 0…0 ② S = 0, 지수 칸 0111 1110 = 126, 가수 100…0 ③ 126 − 127 = −1 ④ $1.1_2 \times 2^{-1} = 0.11_2 = 0.75$. 16비트 0x3A00도 같은 수다: 0011 1010 0000 0000 → S 0, 지수 칸 01110 = 14 → 14 − 15 = −1, 가수 1000000000 → 0.75.

주의: 지수 칸이 bias와 같으면(단정도 0111 1111 = 127, 16비트 01111 = 15) 실제 지수가 0이라 값은 $1.\text{가수}_2$ 그대로다(가수가 전부 0이면 ±1.0). 지수 칸은 무부호 자릿값으로 읽는다: 1000 0011 = 128 + 2 + 1 = 131.

2 · 핵심 정리

- ① 16진 한 자리 → 4비트 ② 1 | 8 | 23(16비트 1 | 5 | 10)으로 자르기
- ③ 실제 지수 = 지수 칸 − bias ④ $(-1)^S \times 1.\text{가수}_2 \times 2^{\text{실제 지수}}$
- +e면 소수점을 오른쪽 e칸, −e면 왼쪽 e칸 옮긴 뒤 자릿값을 더함 ($1.01_2 \times 2^2 = 101_2 = 5$)

3 풀이 예

푸는 순서

① 16진 각 자리를 4비트로

② S, 지수 칸, 가수로 자르기

③ 실제 지수 = 지수 칸 − bias

④ 1.가수$_2$의 소수점을 실제 지수만큼 옮기고 자릿값 합, S = 1이면 음수

단정도 0xC1C80000을 10진수로

① 1100 0001 1100 1000 0000 …

② S = 1, 지수 칸 1000 0011 = 131, 가수 1001 0…0

③ 131 − 127 = 4

④ 1.1001$_2$ → 오른쪽 4칸 → 11001$_2$ = 25 → −25.0

답: **−25.0**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

단정도 0xC0A00000을 10진수로

① 1100 0000 1010 0000 0000 …

② S = 1, 지수 칸 1000 0001 = 129, 가수 0100 0…0

③ 129 − 127 = 2

④ 1.01$_2$ → 오른쪽 2칸 → 101$_2$ = 5 → −5.0

답 보기

떠올리기 카드 3장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

단정도 0x41100000을 10진수로?

답 보기

16비트(1-5-10) 0xBC00을 10진수로?

답 보기

16비트(1-5-10) 0x4500을 10진수로?

답 보기

Ch3 · 7 약 9분

FP 덧셈 4단계 (26-1 A + B)

기출 26-1: A = −0.5, B = 2.625로 A + B. map 상: 단계 순서·계산

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 부동소수점 두 수를 더할 때는 지수를 맞추고, 유효숫자를 더하고, 다시 정규화한 뒤 반올림한다. 4단계 = 정렬 → 유효숫자 더하기 → 정규화·검사 → 반올림.

유효숫자(significand) = 1.xxx 전체(숨은 1 + 가수)다. 지수가 같아야 자리를 맞춰 더할 수 있다. ① **정렬**: 지수가 **작은** 쪽 유효숫자를 **오른쪽**으로 두 지수의 차이만큼 밀고, 지수를 큰 쪽에 맞춘다. 오른쪽 1칸 = 값 ÷ 2라서 지수를 1 올려 값을 유지한다. 큰 쪽을 왼쪽으로 밀면 맨 앞 1(앞자리)을 잃지만, 작은 쪽을 오른쪽으로 밀면 끝의 작은 비트만 잃는다.

② **유효숫자 더하기**: 부동소수점은 **부호-크기 표현**(부호 비트 1개 + 크기, 음수라고 크기를 2의 보수로 바꾸지 않음)이다. 부호가 같으면 크기를 더하고 그 부호를 쓴다. 부호가 다르면 **큰 크기 − 작은 크기**를 하고 **크기가 큰 쪽의 부호**를 쓴다. 2진 소수 계산 요령: 소수 자리 수를 맞춘 뒤 소수점을 떼고 정수로 계산하고 다시 찍는다. $1.000_2 - 0.111_2$ → 1000 − 0111 = 8 − 7 = 1 → 0.001_2.

③ **정규화**: 결과를 다시 1.xxx 꼴로 만든다. 10.xxx처럼 정수 자리가 둘이면 소수점을 왼쪽 1칸·지수 +1, 0.001처럼 앞이 0이면 소수점을 맨 앞 1 뒤까지 오른쪽으로 옮기고 그 칸 수만큼 지수 −. 그다음 지수가 범위(단정도 −126 ~ 127) 안인지 본다(벗어나면 오버플로·언더플로). ④ **반올림**: 유효숫자를 정해진 비트 수에 맞춘다. 반올림 올림으로 정규화가 다시 깨지면(10진 9.9999×10^2 → 10.000×10^2, 2진 1.111… → 10.000…) **③ 정규화(3번째 단계)로 돌아간다**.

슬라이드 예 0.5 + (−0.4375), 유효숫자 4비트(1 + 소수 3비트): $0.5 = 1.000_2 \times 2^{-1}$, $-0.4375 = -0.0111_2 = -1.110_2 \times 2^{-2}$

① 정렬: 지수 차이 1 → $-0.111_2 \times 2^{-1}$

② 부호 다름 → $1.000 - 0.111 = 0.001_2$, 큰 쪽 0.5의 부호 + → $0.001_2 \times 2^{-1}$

③ 맨 앞 1이 소수 셋째 자리 → 오른쪽 3칸, 지수 −1 − 3 = −4 → $1.000_2 \times 2^{-4}$, $-126 \le -4 \le 127$

④ 반올림 필요 없음 → $2^{-4} = 0.0625$

26-1 기출 A + B(A = −0.5, B = 2.625)도 같은 틀이다: $0.5 = 0.1_2$라 A = $-1.0_2 \times 2^{-1}$, $2.625 = 10.101_2$라 B = $1.0101_2 \times 2^1$. 지수 차이 2 → A를 오른쪽 2칸($-0.0100_2 \times 2^1$). 부호가 달라 $1.0101 - 0.0100 = 1.0001_2$(21 − 4 =

17), 큰 쪽 B의 부호 + → $1.0001_2 \times 2^1 = 10.001_2 = 2.125$. 비트로는 S 0, 지수 칸 1 + 127 = 128 = 1000 0000 → 0x40080000, 16비트 1 + 15 = 16 = 10000 → 0x4040.

2 · 핵심 정리

- ① 정렬: 지수가 작은 쪽 유효숫자를 오른쪽 시프트(지수 차이만큼) [Ch3 p.32–33]
- ② 유효숫자 더하기: 부호가 다르면 큰 크기 − 작은 크기, 큰 쪽 부호 (부호-크기 표현)
- ③ 정규화, 오버플로·언더플로 검사 (10.x면 소수점을 왼쪽 1칸·지수 +1, 0.0x면 소수점을 오른쪽·지수 −)
- ④ 반올림 → 정규화가 깨지면 ③으로 ($9.9999 \times 10^2 \rightarrow 10.000 \times 10^2$)
- 슬라이드: $0.5 + (-0.4375) = 1.000_2 \times 2^{-4} = 0.0625$ [Ch3 p.33]

외우는 고리

정·더·정·반: 정렬 → 더하기 → 정규화(검사) → 반올림. 반올림이 깨면 셋째로 되돌이

---

3 풀이 예

푸는 순서

① 두 수를 $1.xxx_2 \times 2^n$으로

② 정렬: 작은 지수 쪽을 지수 차이만큼 오른쪽 시프트, 지수를 큰 쪽에 맞춤

③ 유효숫자 더하기(부호 다르면 큰 크기 − 작은 크기, 큰 쪽 부호)

④ 정규화·범위 검사

⑤ 반올림(유효숫자 비트 수에 맞춤), 필요하면 ④로

⑥ 비트로: S | 지수 칸(n + bias) | 가수

26-1: A = −0.5, B = 2.625일 때 A + B를 단계대로 계산하고 단정도·16비트 16진수로

① $A = -0.1_2 = -1.0_2 \times 2^{-1}$, $B = 10.101_2 = 1.0101_2 \times 2^1$

② 지수 차이 2 → A를 오른쪽 2칸: $-0.0100_2 \times 2^1$

③ 부호 다름 → $1.0101_2 - 0.0100_2 = 1.0001_2$(21 − 4 = 17), B의 부호 + → $1.0001_2 \times 2^1$

④ 이미 정규화, 지수 1 범위 안

⑤ 반올림 필요 없음 → $10.001_2 = 2.125$

⑥ 0 10000000 0001…0 → 0x40080000 / 0 10000 0001000000 → 0100 0000 0100 0000 = 0x4040

답: **$1.0001_2 \times 2^1 = 2.125$, 0x40080000, 16비트 0x4040**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

0.75 + (−0.1875)를 덧셈 단계대로 계산하고 단정도·16비트 16진수로

① $0.75 = 0.11_2 =$ $1.1_2 \times 2^{-1}$, $0.1875 = 0.0011_2 =$ $1.1_2 \times 2^{-3}$

② 지수 차이 2 → $-0.011_2 \times 2^{-1}$

③ $1.100_2 - 0.011_2 =$ 1.001_2 (12 − 3 = 9), 부호 +

④ 이미 정규화 → $1.001_2 \times 2^{-1}$, 범위 안

⑤ 반올림 필요 없음 → 0.5625

⑥ 지수 칸 126 → 0 01111110 001…0 = 0x3F100000 / 지수 칸 14 = 01110 → 0 01110 0010000000 = 0x3880

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

FP 덧셈 4단계를 순서대로 쓰시오.

답 보기

정렬에서 어느 쪽을 어느 방향으로 미는가?

답 보기

빈칸: FP 덧셈에서 반올림 뒤 정규화가 깨지면 ( )번째 단계로 돌아간다.

답 보기

슬라이드 예 0.5 + (−0.4375)의 2진 정규화 결과와 10진 값은?

답 보기

26-1 A = −0.5, B = 2.625일 때 A + B의 단정도·16비트 16진수는?

답 보기

Ch3 · 8 약 9분

FP 곱셈 5단계 (26-1 A × B)

기출 26-1: A = −0.5, B = 2.625로 A × B. map 상: 단계 순서·bias 빼기

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 부동소수점 곱셈은 지수를 더하고 유효숫자를 곱한 뒤 정규화·반올림하고, 마지막에 부호를 정한다. 5단계 = 지수 더하기 → 유효숫자 곱하기 → 정규화·검사 → 반올림 → 부호 결정.

① **지수 더하기**: $2^a \times 2^b = 2^{a+b}$이라 지수는 더한다. 실제 지수끼리 더하면 그대로 끝이다. 지수 칸끼리 더할 때는 bias를 **한 번 뺀다**: 지수 칸 = 실제 지수 + 127이라 (a + 127) + (b + 127) = (a + b) + **254**, 즉 bias가 두 번 들어간다. 127을 한 번 빼야 (a + b) + 127, 곱의 올바른 지수 칸이 된다. 예 지수 칸 131과 120 → 131 + 120 − 127 = 124. 실제 지수로는 4 + (−7) = −3 = 124 − 127.

곱셈은 지수를 더하므로 자리를 맞출 필요가 없다. 그래서 덧셈의 **정렬 단계가 없다**. ② **유효숫자 곱하기** 요령: 소수점을 떼고 정수로 곱한 뒤 두 수의 소수 자리 수를 더한 만큼 소수점을 찍는다(10진 1.5 × 1.5 → 15 × 15 = 225 → 소수 2자리 → 2.25). $1.000_2 \times 1.110_2$ → 8 × 14 = 112 = 1110000_2, 소수 3 + 3 = 6자리 → $1.110000_2 = 1.110_2$. 한쪽이 1.0_2이면 곱은 다른 쪽 그대로다.

③ **정규화·검사**: 곱이 $10.xxx_2$가 되면 소수점을 왼쪽 1칸·지수 +1(예 $1.1_2 \times 1.1_2 = 11 \times 11 = 1001$, 소수 2자리 → $10.01_2 \rightarrow 1.001_2$, 지수 +1). 그리고 지수가 범위 안인지 본다. ④ **반올림**: 정해진 비트 수에 맞춘다. ⑤ **부호 결정**: 부호-크기 표현이라 크기를 다 구한 뒤 부호를 따로 정한다. 두 부호가 같으면 +, 다르면 −.

슬라이드 예 0.5 × (−0.4375) ($1.000_2 \times 2^{-1}$, $-1.110_2 \times 2^{-2}$):
① −1 + (−2) = −3, 지수 칸으로 126 + 125 − 127 = 124
② $1.000_2 \times 1.110_2 = 1.110_2$
③ 이미 정규화 → $1.110_2 \times 2^{-3}$, 범위 안
④ 반올림 필요 없음
⑤ + × − = − → $-1.110_2 \times 2^{-3} = -0.001110_2 = -(1/8 + 1/16 + 1/32) = -0.21875$

26-1 기출 A × B(A = $-1.0_2 \times 2^{-1}$, B = $1.0101_2 \times 2^1$): ① −1 + 1 = 0(지수 칸으로 126 + 128 − 127 = 127, 16비트 14 + 16 − 15 = 15) ② $1.0_2 \times 1.0101_2 = 1.0101_2$ ③ 이미 정규화 ④ 반올림 필요 없음 ⑤ (−) × (+) = − → $-1.0101_2 \times 2^0 = -1.3125$. 비트로는 S 1, 지수 칸 127 = 0111 1111 → 0xBFA80000, 16비트 15 = 01111 → 0xBD40.

2 · 핵심 정리

- ① 지수 더하기 (지수 칸끼리면 bias 한 번 빼기) [Ch3 p.36]
- ② 유효숫자 곱하기 ③ 정규화·오버플로/언더플로 검사 ④ 반올림 ⑤ 부호 결정
- bias 빼는 이유: (a + 127) + (b + 127) = (a + b) + 254 → 127 한 번 빼야 (a + b) + 127
- 곱셈은 정렬하지 않는다. 부호는 크기를 다 구한 뒤 같으면 +, 다르면 −
- 슬라이드: $0.5 \times (-0.4375) = -1.110_2 \times 2^{-3} = -0.21875$, 지수 칸 126 + 125 − 127 = 124 [Ch3 p.36]

외우는 고리

지·곱·정·반·부: 지수 더하기 → 곱하기 → 정규화 → 반올림 → 부호

---

3 풀이 예

푸는 순서

① 지수 더하기(실제 지수끼리, 또는 지수 칸끼리 더하고 bias 한 번 빼기)

② 유효숫자 곱: 소수점 떼고 정수로 곱한 뒤 소수 자리 수 합만큼 소수점

③ 정규화(10.x면 소수점을 왼쪽 1칸·지수 +1)·범위 검사

④ 반올림

⑤ 부호: 같으면 +, 다르면 −

⑥ 비트로: S | 지수 칸 | 가수

26-1: A = −0.5, B = 2.625일 때 A × B를 5단계로 계산하고 단정도·16비트 16진수로

① −1 + 1 = 0. 지수 칸으로 126 + 128 − 127 = 127 (16비트 14 + 16 − 15 = 15)

② $1.0_2 \times 1.0101_2 = 1.0101_2$

③ 이미 정규화, 지수 0 범위 안

④ 반올림 필요 없음

⑤ (−) × (+) = − → $-1.0101_2 \times 2^0 = -1.3125$

⑥ 1 01111111 0101…0 → 1011 1111 1010 1000 0…0 = 0xBFA80000 / 1 01111 0101000000 → 1011 1101 0100 0000 = 0xBD40

답: **$-1.0101_2 \times 2^0 = -1.3125$, 0xBFA80000, 16비트 0xBD40**

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

0.75 × (−0.1875)를 5단계로 계산하고 단정도·16비트 16진수로 ($0.75 = 1.1_2 \times 2^{-1}$, $0.1875 = 1.1_2 \times 2^{-3}$)

① −1 + (−3) = −4. 지수 칸으로 126 + 124 − 127 = 123

② $1.1_2 \times 1.1_2 = 10.01_2$ (11 × 11 = 1001, 소수 2자리)

③ $10.01_2 \rightarrow 1.001_2 \times 2^{-3}$ (왼쪽 1칸, 지수 +1, 지수 칸 124)

④ 반올림 필요 없음

⑤ (+) × (−) = − → $-1.001_2 \times 2^{-3} = -0.140625$

⑥ 1 01111100 001…0 → 1011 1110 0001 0000 0…0 = 0xBE100000 / 지수 칸 12 = 01100 → 1 01100 0010000000 → 1011 0000 1000 0000 = 0xB080

답 보기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

FP 곱셈 5단계를 순서대로 쓰시오.

답 보기

단정도 곱셈에서 두 수의 지수 칸이 131과 120이다. 곱의 지수 칸(정규화 전)과 실제 지수는?

답 보기

지수 칸끼리 더한 뒤 bias를 빼야 하는 이유는?

답 보기

덧셈에는 있고 곱셈에는 없는 단계는?

답 보기

26-1 A = −0.5, B = 2.625일 때 A × B의 단정도·16비트 16진수는?

답 보기

Ch3 · 9 약 7분

부동소수점 외울 문장

map 중: 정규화 장점·bias 이유·오버/언더플로·GRS 판정 T/F·빈칸, FP 명령어(하)

1 읽기 2 핵심 3 떠올리기

---

1 읽기

한 줄 요약: 부동소수점 단원의 T/F·빈칸용 문장 묶음이다. 정규화 장점, 정밀도와 범위, 오버플로·언더플로, bias를 쓰는 이유, 반올림(GRS), RISC-V FP 명령어.

정규화 = 소수점 왼쪽에 0이 아닌 숫자가 **한 자리**. 1.0×10^{-9}은 정규화, 10.0×10^{-10}(두 자리)과 0.1×10^{-8}(0)은 아니다. 장점 3가지: 부동소수점 **데이터 교환** 단순화, 부동소수점 **알고리즘** 단순화, 한 word에 저장되는 수의 **정확도** 증가(쓸모없는 앞자리 0 대신 진짜 숫자를 담으니까).

칸 수는 한정되어 있어 맞바꿈이 있다: 가수(fraction) 칸을 늘리면 **정밀도**(precision, 얼마나 촘촘한가), 지수 칸을 늘리면 **범위**(range, 얼마나 크고 작은 수까지)가 좋아진다. FP **오버플로** = 지수가 너무 커서 지수 칸에 담을 수 없는 것, **언더플로** = 음의 지수가 너무 커서(0에 너무 가까워서) 담을 수 없는 것이다. 가능성을 줄이려면 지수 칸이 넓은 64비트 double을 쓴다.

bias를 쓰는 이유: IEEE 754는 부동소수점 수를 **정수 비교**만으로 크기순 정렬할 수 있게 설계했다. ① 부호를 MSB에 둬 빠르게 비교 ② 지수를 가수 **앞**에 둬 정렬을 단순하게 ③ 음수 지수를 2의 보수로 쓰면 맨 앞이 1이라 큰 수로 보이는 문제를 bias로 해결(실제 지수 −1 → 지수 칸 126 = 0111 1110). 슬라이드 p.25에 지수가 「2's complement」라고 적혀 있어도 시험에서는 bias로 답한다.

반올림 4모드: 0 쪽(버림), +∞ 쪽, −∞ 쪽, **가장 가까운 짝수**(기본). 예 −2.75 → −2, −2, −3, −3 / 3.8 → 3, 4, 3, 4. 중간 계산 때는 오른쪽에 여분 비트 guard·round 2개를 유지해 정확도를 높인다($2.56 \times 10^{0} + 2.34 \times 10^{2}$ → 여분이 없으면 2.36×10^{2}, 있으면 2.37×10^{2}).

GRS: G(guard) = 남길 LSB 바로 다음 비트, R(round) = 그다음 비트, S(sticky) = 그 뒤 나머지 비트 전체의 **OR**(1이 하나라도 있었는지 1비트로 기억). G의 자릿값은 LSB의 정확히 절반이다. GRS 000 ~ 011 = 절반 미만 → 내림(버림), 101 ~ 111 = 절반 초과 → 올림(LSB에 1 더하기, 1011 + 1 = 1100처럼 올림 전파), **100** = 딱 절반 → LSB가 **0**(짝수)이 되는 쪽. 예 가수 4비트: 1.1011 100 → LSB 1이라 올림 → 1.1100 / 1.0100 100 → LSB 이미 0이라 그대로 / 1.0010 101 → 올림 → 1.0011.

RISC-V FP: 레지스터 **f0 ~ f31**(x0과 달리 f0은 0으로 고정 **아님**). 적재·저장 `flw` / `fsw`(단정도), `fld` / `fsd`(배정도). 비교 `feq` · `flt` · `fle`(.s/.d)는 결과를 FP 레지스터가 아니라 **정수 레지스터**에 0(거짓)·1(참)로 쓰고, 그

값으로 `beq` · `bne` 분기한다.

2 · 핵심 정리

- 정규화 = 소수점 왼쪽에 0이 아닌 숫자 한 자리. 장점: 데이터 교환 단순화, 알고리즘 단순화, word 안 정확도 증가 [Ch3 p.23–24]
- 가수 칸↑ = 정밀도, 지수 칸↑ = 범위 (맞바꿈). 오버플로 = 지수가 너무 커서, 언더플로 = 음의 지수가 너무 커서 지수 칸에 못 담음 → double [Ch3 p.25–26]
- bias 이유(정수 비교로 정렬): 부호가 MSB → 빠른 비교, 지수가 가수 앞 → 정렬 단순, 음수 지수가 크게 보이는 문제 해결 [Ch3 p.29]
- 반올림 4모드: 0 쪽, +∞ 쪽, −∞ 쪽, 가장 가까운 짝수(기본). G = LSB 다음, R = 그다음, S = 나머지 OR. GRS 000~011 내림, 101~111 올림, 100이면 LSB 0인 쪽 [Ch3 p.38–40]
- f0~f31(f0은 0 고정 아님), flw/fsw·fld/fsd. feq·flt·fle 결과는 정수 레지스터에 0·1 → beq·bne [Ch3 p.37]

외우는 고리

정규화 장점 교·알·정(교환, 알고리즘, 정확도). bias 이유 부·앞·음(부호 MSB, 지수 앞, 음수 지수)

---

4 떠올리기

떠올리기 카드 7장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

정규화 표기의 장점 3가지는?

답 보기

1.0×10^{-9}, 10.0×10^{-10}, 0.1×10^{-8} 중 정규화된 것은?

답 보기

O/X: 지수 칸 비트 수를 늘리면 정밀도가 좋아진다.

답 보기

FP 오버플로와 언더플로의 정의는?

답 보기

IEEE 754가 지수에 bias를 쓰는 이유는? (목표와 이유 셋)

답 보기

가수 4비트만 남길 때 가장 가까운 짝수로: 1.0111 100 (뒤 3비트가 GRS)

답 보기

O/X: (1) RISC-V의 f0은 항상 0.0이다. (2) flt.s의 결과는 FP 레지스터에 저장된다.

답 보기

Ch4 · 1 약 5분

구현 대상과 공통 두 단계

기출 21-2: 프로세서를 구성하는 두 가지. 구현 명령어 목록은 단답 단골

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: Ch4는 명령어 7개만 실행하는 가장 단순한 프로세서를 부품부터 조립해 보는 장이고, 어떤 명령어든 「명령어 가져오기 → 레지스터 읽기」 두 단계로 시작한다.

프로세서 = 데이터패스 + 제어. Ch1의 5대 구성 요소(입력, 출력, 메모리, 데이터패스, 제어) 중 뒤의 둘이 프로세서다. **데이터패스(datapath)**는 데이터를 옮기고 계산하고 저장하는 부품과 그 연결선 전체이고, **제어**는 그 부품마다 「지금 이 일을 해라」 하는 0/1 신호를 보내는 쪽이다. 그 신호를 만드는 회로를 **제어 유닛(control unit)**이라 부른다.

구현할 명령어는 7개뿐이다: 메모리 참조 lw·sw, 산술·논리 add·sub·and·or, 분기 beq. 종류로 묶으면 2 · 4 · 1개다. 이 7개를 실행할 부품만 모은 것이 이 장의 **단일 사이클(single-cycle)** 구현이고, 명령어 하나를 클럭 한 사이클에 끝까지 실행한다는 뜻이다.

어떤 명령어든 처음 두 단계는 같다. ① **인출(fetch)**: PC(프로그램 카운터, 지금 실행할 명령어의 주소를 담은 레지스터)의 값을 **명령어 메모리**(명령어가 든 메모리)에 보내 그 주소의 32비트 명령어를 가져온다. ② 명령어 안의 레지스터 번호 칸(rs1, rs2)으로 레지스터 1개 또는 2개를 읽는다. 그다음부터는 명령어 종류마다 다르다.

예: `add x5, x6, x7`은 인출 → x6·x7 읽기 → **ALU**(산술 논리 장치, 덧셈·뺄셈·AND·OR를 하는 회로)에서 덧셈 → 합을 x5에 쓰기. `lw x5, 40(x6)`은 인출 → x6 읽기 → ALU 덧셈으로 주소 x6 + 40 계산 → 그 주소의 메모리를 읽어 x5에 쓰기. 둘 다 ALU가 덧셈을 하지만, ALU 둘째 입력이 레지스터(x7)냐 명령어 속 숫자(40)냐가 다르고, x5에 쓰는 값이 ALU 결과냐 메모리 값이냐가 다르다.

그래서 **MUX(멀티플렉서)**가 필요하다. MUX는 입력 여러 개 중 하나를 선택 신호대로(0이면 입력 0, 1이면 입력 1) 골라 내보내는 장치다. 데이터 선 두 가닥은 그냥 한 점에 합칠 수 없으므로 갈림길마다 MUX를 두고, 데이터패스를 조종할 제어 유닛이 선택 신호를 정한다. 기본 구현 = 단순한 데이터패스에 MUX 3개(ALU 둘째 입력 앞, 레지스터에 쓸 값 앞, 다음 PC 앞)와 제어 유닛을 더한 것이다.

2 · 핵심 정리

- 구현 7개: lw, sw / add, sub, and, or / beq [Ch4 p.3]
- 공통 두 단계: ① PC로 명령어 메모리에서 인출 ② 필드로 레지스터 1~2개 읽기 [Ch4 p.3]
- 프로세서 = 데이터패스 + 제어 [Ch4 p.8]
- 선을 그냥 합칠 수 없다 → MUX, 데이터패스 조종 → 제어 유닛. 기본 구현 = MUX 3개 + 제어 유닛 [Ch4 p.7–8]
- add x5, x6, x7: 인출 → x6·x7 읽기 → 덧셈 → x5 쓰기 / lw x5, 40(x6): 인출 → x6 읽기 → 주소 계산(x6 + 40) → 메모리 읽어 x5 [Ch4 p.5–6]

외우는 고리

7개 = 메모리 둘(lw sw) + 산술 넷(add sub and or) + 분기 하나(beq): 2 · 4 · 1

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

단일 사이클 구현 대상 명령어 7개는?

답 보기

빠진 것 하나: lw, sw, add, sub, and, ( ), beq

답 보기

모든 명령어에 공통인 두 단계는?

답 보기

빈칸: 프로세서는 ( )와 ( )로 이루어진다.

답 보기

데이터패스에 MUX가 필요한 이유는?

답 보기

Ch4 · 2 약 7분

데이터패스 구성 요소와 그림 빈칸

기출 21-2: 그림 빈칸(ALU, adder 등), 레지스터가 있는 하드웨어 이름

1 읽기 2 핵심 3 떠올리기

---

1 읽기

한 줄 요약: 데이터패스는 PC, 명령어 메모리, 가산기 둘, 레지스터 파일, ImmGen, ALU, 데이터 메모리, MUX 셋으로 이루어지고, 시험은 그림에서 상자 이름을 지우고 묻는다.

그림을 왼쪽부터 글로 따라가 보자. 맨 왼쪽 작은 상자 **PC**가 현재 명령어 주소를 내보내고, 바로 오른쪽 **명령어 메모리**가 그 주소를 받아 32비트 명령어를 내보낸다. PC 위쪽의 작은 **가산기**(덧셈만 하는 회로, 그림에 Add)는 PC와 상수 4를 받아 PC + 4를 만든다. 명령어 하나가 4바이트라 다음 명령어 주소가 +4이기 때문이다.

명령어 비트는 가운데 큰 사각형 **레지스터 파일(register file)**로 간다. 레지스터 파일 = 범용 레지스터 32개를 모아 번호로 읽고 쓰는 부품이고, **포트**(값이 들어가고 나오는 입구·출구)는 읽기 2개, 쓰기 1개다. 입구 읽기 레지스터 1·2와 쓰기 레지스터에는 번호(5비트)가 들어오고, 출구 읽기 데이터 1·2로 그 레지스터의 값(32비트)이 나오며, 입구 쓰기 데이터로 쓸 값이 들어온다. 실제로 쓸지는 제어 신호 RegWrite가 정한다.

레지스터 파일 아래 작은 타원 **ImmGen(즉시값 생성기)**은 명령어 전체 [31:0]을 받아 그 안의 12비트 즉시값을 32비트로 **부호 확장**(부호 비트를 왼쪽에 복사해 값 유지)해 내보낸다. ALU가 32비트끼리 계산하기 때문이다. 그 오른쪽 **ALU**는 왼쪽 변 가운데가 V자로 파인 도형으로, 32비트 입력 2개와 4비트 제어 신호를 받아 결과와 **Zero**(결과가 0이면 1인 출력)를 낸다. ALU 둘째 입력 앞의 MUX(ALUSrc)가 읽기 데이터 2와 ImmGen 출력 중 하나를 고른다.

오른쪽 큰 상자 **데이터 메모리**는 주소 입구로 ALU 결과(베이스 + 오프셋)를 받고, 쓰기 데이터 입구로 읽기 데이터 2(sw가 저장할 rs2 값, ALU를 거치지 않고 바로 옴)를 받는다. 출구 읽기 데이터와 ALU 결과 중 하나를 MUX(MemtoReg)가 골라 레지스터 파일의 쓰기 데이터로 돌려보낸다. 오른쪽 위 둘째 가산기는 **분기 가산기**로, PC + (오프셋 << 1) = 분기 목표 주소를 만들고, 맨 오른쪽 MUX(PCSrc)가 이것과 PC + 4 중 하나를 골라 다음 PC로 돌려보낸다.

예 `beq x1, x2, L` : 분기 목표 주소는 ALU가 아니라 분기 가산기가 계산한다. ALU는 x1 − x2를 계산해 Zero로 두 값이 같은지만 알려 준다. `<< 1` (왼쪽 1비트 시프트 = × 2)은 Ch2에서 분기 오프셋의 0번 비트를 저장하지 않았던 것을 되돌리는 일이고, 이 그림에서는 ImmGen이 함께 한다(따로 Shift left 1 상자가 그려진 그림도 있다).

그림 빈칸 푸는 법: 모양으로 후보를 좁히고, 모양이 같으면 들어오는 선으로 가른다. Add 둘: 입력이 PC와 4면 PC + 4 가산기, PC와 ImmGen 출력이면 분기 가산기. 타원 셋: [6:0](opcode)을 받으면 제어 유닛, [31:0]을 받으면

ImmGen, ALUOp를 받아 4비트를 ALU로 보내면 ALU 제어. 주의: 슬라이드 그림은 MemtoReg MUX만 위가 1·아래가 0이니 위치가 아니라 적힌 숫자로 읽는다.

2 · 핵심 정리

- PC = 현재 명령어 주소 / 명령어 메모리 = PC 받아 32비트 명령어 출력 / 가산기 = PC + 4 [Ch4 p.11]
- 레지스터 파일 = 레지스터 32개, 읽기 포트 2, 쓰기 포트 1 [Ch4 p.12]
- ALU = 입력 2개, 4비트 제어, 결과 0이면 Zero = 1 / ImmGen = 12비트 → 32비트 부호 확장 [Ch4 p.12–13]
- 분기 가산기 = PC + (오프셋 << 1), ALU와 별개 / 데이터 메모리 주소 = ALU 결과 [Ch4 p.14–15]
- MUX 3개 자리: ALUSrc = ALU 둘째 입력 앞, MemtoReg = 레지스터 쓰기 데이터 앞, PCSrc = PC 입력 앞 [Ch4 p.16–17]

외우는 고리

모양이 같은 상자는 선으로 가른다: Add 둘(PC + 4 / PC + ImmGen), 타원 셋([6:0] 제어 유닛 / [31:0] ImmGen / ALUOp ALU 제어)

---

4 떠올리기

떠올리기 카드 7장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

그림 빈칸: PC와 ImmGen 출력을 더하는 상자는?

답 보기

그림 빈칸: 결과가 0이면 1을 내는 출력(Zero)을 가진 상자는?

답 보기

그림 빈칸: 명령어 [31:0]을 받아 32비트를 내는 작은 타원은?

답 보기

범용 레지스터 32개를 모아 번호로 읽고 쓰는 하드웨어 이름은?

답 보기

데이터 메모리의 주소 입력과 쓰기 데이터 입력은 각각 어디서 오나?

답 보기

MUX 3개의 선택 신호와 자리는?

답 보기

분기 목표 주소를 ALU가 계산한다. O/X

답 보기

Ch4 · 3 약 6분

조합·상태 소자, 엣지 트리거, 메모리 둘

map 상: 조합 vs 상태 분류, 클로킹 정의, 「왜 메모리가 둘?」

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 데이터패스 부품은 「값을 기억하는가」로 조합 소자와 상태 소자로 나뉘고, 상태 소자는 클럭 엣지에서만 값을 바꾼다.

조합 소자(combinational element) = 출력이 지금 들어온 입력만의 함수인 소자다. 같은 입력이면 언제나 같은 출력이고 기억하는 것이 없다. 예: 가산기에 3과 4가 들어오면 언제나 7이다. AND 게이트, ALU, 가산기, MUX(입력 중 하나를 고를 뿐), ImmGen(지금 명령어 비트를 늘릴 뿐)이 여기에 든다.

상태 소자(state element) = 값을 저장하는 소자다. PC, 레지스터 파일, 메모리가 여기에 든다. 입력은 최소 2개로, 써 넣을 데이터 값과 언제 쓸지 정하는 **클럭**(일정 간격으로 0과 1을 오가는 신호)이고, 출력은 이전 클럭 사이클에 써 둔 값이다. 예: 레지스터 파일에 같은 번호 5를 넣어도 전에 x5에 무엇을 써 두었느냐에 따라 다른 값이 나온다.

클로킹 방식(clocking methodology) = 신호를 언제 읽고 언제 쓸 수 있는지 정한 규칙이고, 하드웨어 동작을 예측 가능하게 하려고 정한다. 이 장은 **엣지 트리거(edge-triggered)** 방식을 쓴다: 상태 소자의 값은 **클럭 엣지**(클럭이 0에서 1로, 또는 1에서 0으로 바뀌는 순간)에서만 바뀐다. 클럭 사이클 하나는 엣지 하나에서 다음 엣지까지다.

한 사이클의 흐름: ① 엣지에서 상태 소자(PC, 레지스터 파일)가 값을 내보낸다. ② 엣지 사이에 조합 소자(가산기, ALU, MUX)가 그 값으로 계산한다. ③ 다음 엣지에서 결과가 상태 소자에 저장된다. 예: PC는 사이클 내내 현재 주소를 내보내고, 가산기가 만든 PC + 4는 다음 엣지에야 PC에 들어간다. 조합 소자도 신호가 지나는 데 시간(지연)이 걸리고 가장 느린 계산까지 다음 엣지 전에 끝나야 하므로, 가장 긴 지연이 클럭 주기를 정한다.

메모리가 왜 둘인가: 단일 사이클에서는 한 명령어가 한 자원을 한 번만 쓸 수 있다. 그런데 lw·sw는 명령어를 가져올 때(인출) 한 번, 데이터를 읽거나 쓸 때 한 번, 메모리를 두 번 쓴다. 그래서 명령어 메모리와 데이터 메모리를 따로 둔다. 레지스터 파일은 한 명령어에서 읽고 쓰지만, 읽기 포트 2개와 쓰기 포트 1개가 따로 달려 있고 쓰기는 사이클 끝 엣지에 일어나서 괜찮다.

비유: 조합 소자는 계산기(넣으면 답, 기억 없음), 상태 소자는 메모장(적어 둔 값을 보여 주다가 종이 울릴 때만 고쳐 씀), 클럭 엣지는 그 종이다. 안 맞는 점: 실제 회로는 답이 즉시 나오지 않고 지연이 있으며, 그 지연이 클럭 주기를 정한다. 주의: 메모리가 둘인 이유를 「속도 때문」이라고 쓰면 틀린다.

2 · 핵심 정리

- 조합 소자 = 출력이 입력만의 함수(AND 게이트, ALU, 가산기, MUX, ImmGen) [Ch4 p.9]
- 상태 소자 = 저장(PC, 레지스터 파일, 메모리). 입력 = 쓸 데이터·클럭, 출력 = 이전 사이클에 쓴 값 [Ch4 p.9]
- 클로킹 방식 = 언제 읽고 쓰는지 정한 규칙(하드웨어를 예측 가능하게) [Ch4 p.10]
- 엣지 트리거 = 상태 소자는 클럭 엣지에서만 갱신, 가장 긴 지연이 클럭 주기 결정 [Ch4 p.10]
- 메모리 둘 = 단일 사이클은 자원을 명령어당 한 번만 → lw·sw는 인출 + 데이터 접근으로 메모리 두 번 [Ch4 p.16]

외우는 고리

분류 질문 하나: 「값을 기억하는가?」 기억하면 상태, 아니면 조합

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

분류(조합 C / 상태 S): PC, MUX, 레지스터 파일, ImmGen

답 보기

상태 소자의 입력 두 가지와 출력은?

답 보기

엣지 트리거 방식의 정의는?

답 보기

O/X: 엣지 트리거에서 레지스터 파일 값은 사이클 내내 바뀔 수 있다.

답 보기

단일 사이클에서 명령어 메모리와 데이터 메모리를 나누는 이유는?

답 보기

Ch4 · 4 약 11분

명령어 비트 → 포트, 기계어 → 선 번호

기출 26-1: datapath·control line, 라인별 참조 레지스터 번호

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 명령어 32비트의 칸은 전선으로 데이터패스 포트에 고정 연결되어 있어서, 기계어만 보면 각 포트 선에 실리는 레지스터 번호를 바로 읽을 수 있다.

Ch2에서 본 칸 자리: opcode = [6:0](맨 오른쪽 7비트), rd = [11:7], funct3 = [14:12], rs1 = [19:15], rs2 = [24:20], funct7 = [31:25]. [a:b]는 a번 비트부터 b번 비트까지라는 뜻이다(0번이 맨 오른쪽). rs1·rs2·rd는 모든 형식에서 같은 자리라서, 명령어를 해독하기 전에 이 비트들을 포트에 바로 전선으로 이을 수 있다.

연결: [6:0] → 제어 유닛, [19:15] → 읽기 레지스터 1, [24:20] → 읽기 레지스터 2, [11:7] → 쓰기 레지스터, [31:0] 전체 → ImmGen, [30]과 [14:12] → ALU 제어. [14:12]는 funct3 전체이고 [30]은 funct7 중 한 비트다. 구현하는 R형 넷 중 funct7이 다른 것은 sub(0100000)뿐이고 그 차이가 30번 비트라서, 이 한 비트만 보내도 add와 sub를 가를 수 있다.

칸의 역할: rs1 = R형·분기의 첫째 피연산자, load·store의 **베이스**(주소의 기준 레지스터). rs2 = R형·분기의 둘째 피연산자, store에서 메모리로 복사될(저장할) 값. rd = R형·load의 결과를 받는 목적지. 형식은 opcode로 가른다: 0110011 R형, 0000011 lw, 0100011 sw, 1100011 beq. R형은 funct7·funct3으로 고른다: add 0000000·000, sub 0100000·000, and 0000000·111, or 0000000·110.

예 `0x01DE73B3`: 16진 한 자리를 4비트로 바꾸면 0000 0001 1101 1110 0111 0011 1011 0011. 맨 오른쪽 7비트 0110011 → R형이니 7 · 5 · 5 · 3 · 5 · 7로 끊는다: funct7 0000000 · rs2 11101 · rs1 11100 · funct3 111 · rd 00111. 읽기 레지스터 1 = 11100_2 = 28, 읽기 레지스터 2 = 11101_2 = 29, 쓰기 레지스터 = 00111_2 = 7. funct 0000000·111 → and이고 어셈블리는 rd, rs1, rs2 순이라 `and x7, x28, x29`.

선에는 값이 아니라 번호가 실리고, 형식과 상관없이 그 자리 비트가 그대로 실린다. lw(I형)의 칸은 imm[11:0] 12 · rs1 5 · funct3 3 · rd 5 · opcode 7이라 [24:20]은 rs2가 아니라 즉시값의 아래 5비트다. 그래도 전선은 고정이라 그 조각이 읽기 레지스터 2에 번호로 실리지만, lw는 ALUSrc = 1(ALU 둘째 입력으로 즉시값을 고름)이라 결과에 쓰이지 않는다. lw의 주소 = 읽기 데이터 1(베이스 값) + ImmGen 출력([31:20]을 부호 확장한 값)이다.

sw(S형)의 칸은 imm[11:5] 7 · rs2 · rs1 · funct3 · imm[4:0] 5 · opcode라 [11:7]은 rd가 아니라 즉시값 조각이다 (beq의 [11:7]도 즉시값 조각). 예 `sw x8, 12(x9)` = 0x0084A623: [19:15] = 01001 = 9(베이스), [24:20] =

01000 = 8(저장할 값), [11:7] = 01100 = 12. 쓰기 레지스터 선에 12가 실리지만 sw는 레지스터에 쓰지 않아 RegWrite = 0이므로 x12는 바뀌지 않는다. 쓸지 말지는 제어 신호(ALUSrc, RegWrite)가 정한다.

2 · 핵심 정리

- opcode 6:0 → 제어 유닛 / rs1 19:15 → 읽기 레지스터 1 [Ch4 p.21, 24]
- rs2 24:20 → 읽기 레지스터 2 / rd 11:7 → 쓰기 레지스터 [Ch4 p.21, 24]
- [31:0] → ImmGen / [30]·[14:12] → ALU 제어 [Ch4 p.24]
- rs1 = R형·분기 첫 피연산자, load·store 베이스 / rs2 = store에서 메모리로 복사될 값 / rd = R형·load의 목적지 [Ch4 p.21]
- 세 선은 비트에 고정 연결 → lw의 [24:20], sw·beq의 [11:7]은 즉시값 조각이 실리지만 ALUSrc·RegWrite 때문에 안 쓰임

외우는 고리

19·24·11 = 「일구 이사 십일」: rs1 [19:15], rs2 [24:20], rd [11:7]. opcode는 맨 오른쪽 7비트

---

3 풀이 예

푸는 순서

① 16진 한 자리를 4비트로 → 32비트

② 오른쪽 7비트 opcode로 형식: 0110011 R형, 0000011 lw, 0100011 sw, 1100011 beq

③ [19:15] → 읽기 레지스터 1, [24:20] → 읽기 레지스터 2, [11:7] → 쓰기 레지스터. 5비트씩 10진

④ 선에는 값이 아닌 번호가 실린다. 쓸지 말지는 제어 신호가 정함

lw x14, 20(x10) = 0x01452703. 읽기 레지스터 1·2, 쓰기 레지스터 선의 번호, ImmGen 출력, x10 = 0x2000일 때 데이터 메모리 주소는?

① 0000 0001 0100 0101 0010 0111 0000 0011

② 오른쪽 7비트 0000011 → lw(I형): imm 000000010100 · rs1 01010 · funct3 010 · rd 01110

③ [19:15] = 01010_2 = 10 → 읽기 레지스터 1 = 10

④ [24:20] = 10100_2 = 20(imm 아래 5비트) → 읽기 레지스터 2 = 20, ALUSrc = 1이라 안 쓰임

⑤ [11:7] = 01110_2 = 14 → 쓰기 레지스터 = 14

⑥ ImmGen = 000000010100_2 = 20, 주소 = 0x2000 + 0x14 = 0x2014

답: **10, 20, 14, ImmGen 20, 주소 0x2014**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

lw x9, 44(x21) = 0x02CAA483. 읽기 레지스터 1·2, 쓰기 레지스터 선의 번호, x21 = 0x1000일 때 데이터 메모리 주소는?

① 0000 0010 1100 1010 1010 0100 1000 0011

② 오른쪽 7비트 0000011 → lw(I형)

③ [19:15] = 10101_2 = 21 → 읽기 레지스터 1

④ [24:20] = 01100_2 = 12 → 읽기 레지스터 2(imm 조각, 안 쓰임)

⑤ [11:7] = 01001_2 = 9 → 쓰기 레지스터

⑥ ImmGen = 000000101100_2 = 44 = 0x2C, 주소 = 0x1000 + 0x2C = 0x102C

답 보기

떠올리기 카드 6장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

rs1, rs2, rd, opcode의 비트 범위는?

답 보기

ALU 제어 유닛으로 가는 명령어 비트는?

답 보기

0x41498933 실행 시 읽기 레지스터 1, 읽기 레지스터 2, 쓰기 레지스터 번호와 명령어는?

답 보기

0x005261B3의 세 선 번호와 명령어는?

답 보기

---

sw x7, 8(x3) = 0x0071A423. 쓰기 레지스터 선의 번호와 그 값이 문제없는 이유는?

답 보기

---

store에서 rs2의 역할은?

답 보기

Ch4 · 5 약 6분

제어 신호 7개의 뜻

map 상: 제어 신호. 기출 21-2·26-1 control line

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 주 제어 유닛은 opcode를 보고 0/1 신호 7개를 내보내, 데이터패스의 스위치(쓰기·읽기 켜고 끄기)와 MUX 갈림길을 정한다.

일을 시키는 스위치 셋: RegWrite = 1이면 쓰기 레지스터(rd)에 쓰기 데이터를 기록한다. MemRead = 1이면 데이터 메모리가 주소의 내용을 읽기 데이터로 내보내고, MemWrite = 1이면 주소의 내용을 쓰기 데이터로 바꾼다. 셋 다 0이면 아무 일도 하지 않는다.

MUX 선택 신호 둘: ALUSrc는 ALU 둘째 입력을 고른다(0 = 읽기 데이터 2, 즉 레지스터 값 / 1 = 부호 확장 즉시값). MemtoReg는 레지스터에 쓸 값을 고른다(0 = ALU 결과 / 1 = 메모리 읽기 데이터). 이름을 풀면 ALUSrc = ALU의 source(출처), MemtoReg = Memory to Register라서 1이면 메모리 값이 레지스터로 간다.

다음 PC: Branch = 1이면 지금 명령어가 분기(beq)라는 뜻이다. PCSrc는 다음 PC를 고르는 MUX 신호다(0 = PC + 4 / 1 = 분기 목표 주소). PCSrc는 주 제어 유닛이 직접 내지 않고, 바깥 AND 게이트가 Branch와 ALU의 Zero를 받아 만든다(PCSrc 덩어리에서 자세히).

ALUOp(2비트)는 주 제어 유닛이 ALU 제어 유닛에 주는 힌트다(00 덧셈, 01 뺄셈, 10 funct를 보고 정함). ALU에 실제로 들어가는 4비트 ALU 제어 값은 따로 있는 작은 **ALU 제어 유닛**이 ALUOp와 funct를 보고 만든다. 이렇게 주 제어 → ALUOp → ALU 제어로 나누는 것을 **다단 디코딩**이라 한다. 그래서 ALUOp(2비트, 주 제어 유닛 출력)와 ALU 제어(4비트, ALU 제어 유닛 출력)는 다른 것이다.

주 제어 유닛 출력은 그림 위에서부터 Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite, 모두 7개다. 묶어 외우기: 레지스터 쪽 2(RegWrite, ALUSrc), 메모리 쪽 3(MemRead, MemWrite, MemtoReg), 분기·ALU 2(Branch, ALUOp). PCSrc는 이 7개에 들지 않는다.

예 `lw x5, 40(x6)`에 대입: 메모리에서 읽은 값을 x5에 쓰니 RegWrite = 1, MemRead = 1, MemtoReg = 1(쓸 값 = 메모리 값). 주소 = x6 + 40이라 ALU 둘째 입력은 즉시값 → ALUSrc = 1. 분기가 아니니 Branch = 0이고 다음 PC = PC + 4. 신호마다 「이 명령어가 그 일을 하나?」를 물으면 값이 나온다.

2 · 핵심 정리

- RegWrite = 1 → rd에 쓰기 데이터 기록 / MemRead·MemWrite = 1 → 데이터 메모리 읽기·쓰기 [Ch4 p.24]
- ALUSrc: 0 = 읽기 데이터 2, 1 = 부호 확장 즉시값 [Ch4 p.24]
- MemtoReg: 0 = ALU 결과, 1 = 메모리 읽기 데이터 [Ch4 p.24]
- Branch = 1 → 분기 명령어 / PCSrc: 0 = PC + 4, 1 = 분기 목표 주소 [Ch4 p.24–25]
- ALUOp(2비트) = 주 제어 → ALU 제어 유닛에 주는 힌트 [Ch4 p.18]
- 주 제어 출력 순서(그림 위부터): Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite. PCSrc는 바깥 AND 게이트 [Ch4 p.26–28]

외우는 고리

7개 = 레지스터 쪽 2(RegWrite, ALUSrc) + 메모리 쪽 3(MemRead, MemWrite, MemtoReg) + 분기·ALU 2(Branch, ALUOp). PCSrc는 따로 AND 게이트가 만든다

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

주 제어 유닛 출력 7개 중 빠진 것: RegWrite, ALUSrc, MemRead, MemWrite, Branch, ALUOp

답 보기

ALUSrc = 1의 뜻은?

답 보기

MemtoReg = 1의 뜻은?

답 보기

PCSrc = 0, 1일 때 다음 PC는?

답 보기

ALUOp(2비트)와 ALU 제어(4비트)는 각각 누가 내나?

답 보기

Ch4 · 6 약 9분

명령어별 제어 신호 0/1/X 표

기출 21-2: ld, sd, beq, add의 0/1/X 표 (ld = lw, sd = sw)

1 읽기 2 핵심 3 떠올리기

1 읽기

한 줄 요약: 네 종류 명령어(R형, lw, sw, beq)마다 신호 7개의 값이 정해져 있고, 값은 「그 명령어가 하는 일」에서 하나씩 끌어낼 수 있다.

먼저 하는 일을 식으로 쓴다: R형 rd ← rs1 연산 rs2, lw rd ← 메모리[rs1 + imm], sw 메모리[rs1 + imm] ← rs2, beq rs1 = rs2면 분기. 그다음 차례로 묻는다. ① 식의 왼쪽이 rd인가(레지스터에 쓰나)? 쓰면 RegWrite = 1. ② 쓴다면 그 값이 메모리 값인가? 메모리면 MemtoReg = 1, ALU 결과면 0. ③ 메모리를 읽나(MemRead), 쓰나(MemWrite)? ④ ALU 둘째 입력이 즉시값인가(ALUSrc)? ⑤ 분기인가(Branch)? ⑥ ALU가 할 일: 주소 덧셈 00, 비교 뺄셈 01, funct로 정함 10(ALUOp).

예 sw(메모리[rs1 + imm] ← rs2): ① 왼쪽이 메모리라 RegWrite = 0. ② 레지스터에 안 쓰니 쓸 값을 고르는 MemtoReg는 무엇을 골라도 상관없다 → X. ③ 메모리에 쓰기만 → MemRead = 0, MemWrite = 1. ④ 주소 = 베이스 + 즉시값 → ALUSrc = 1. ⑤ Branch = 0. ⑥ 주소 덧셈 → ALUOp = 00. 열 순서 ALUSrc · MemtoReg · RegWrite · MemRead · MemWrite · Branch · ALUOp로 쓰면 1 X 0 0 1 0 00.

같은 방법으로: R형(레지스터끼리 계산해 ALU 결과를 rd에) = 0 0 1 0 0 0 10. lw(주소 계산 후 메모리 값을 rd에) = 1 1 1 1 0 0 00. beq(두 레지스터를 빼서 비교, 아무것도 쓰지 않음) = 0 X 0 0 0 1 01. beq에도 오프셋이 있지만 오프셋은 분기 가산기로 가고 ALU는 두 레지스터를 빼므로 ALUSrc = 0이다. 거꾸로 신호에서 명령어를 찾을 때: RegWrite = 1은 R형·lw뿐이고, 그중 ALUSrc = 1은 lw다.

X(don't care) = 0이든 1이든 실행 결과가 같다는 뜻이다(0과 같은 뜻이 아니다). X가 될 수 있는 것은 MUX 선택 신호뿐이고, 그 MUX의 출력을 아무도 쓰지 않을 때뿐이다. MUX는 고르기만 하고 아무것도 바꾸지 않기 때문이다. MemtoReg MUX의 출력은 레지스터 쓰기 데이터로만 가므로 RegWrite = 0인 sw·beq에서 X다. ALUSrc MUX의 출력은 ALU로 가는데 ALU 결과는 네 명령어 모두 쓰므로(R형 결과, lw·sw 주소, beq 비교) ALUSrc는 X가 될 수 없다.

반대로 RegWrite·MemWrite·MemRead·Branch는 레지스터·메모리·PC에 일을 시키는 스위치라 X로 두면 안 된다. 예: sw의 RegWrite가 1이 되면 [11:7](즉시값 조각)을 번호로 하는 레지스터가 덮어써진다. MemWrite가 잘못 1이면 메모리 값이 망가지고, Branch가 잘못 1이면 엉뚱하게 분기한다. MemRead는 불필요한 메모리 읽기를 막으려고 0으로 둔다(PDF 외 보충).

주의: 기출의 **ld·sd**는 64비트판 이름(load·store doubleword)이고 신호 값은 lw·sw와 같다(PDF 외 보충). 문제마다 열 순서가 다를 수 있으니(예: 제어 유닛 출력 순서 Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite) 값은 외운 줄을 그대로 베끼지 말고 신호 이름별로 옮겨 적는다.

2 · 핵심 정리

- 열 순서: ALUSrc · MemtoReg · RegWrite · MemRead · MemWrite · Branch · ALUOp [Ch4 p.25]
- R형: 0 0 1 0 0 0 10
  lw(ld): 1 1 1 1 0 0 00
- sw(sd): 1 X 0 0 1 0 00
  beq: 0 X 0 0 0 1 01
- X = don't care: RegWrite = 0인 sw·beq의 MemtoReg만 X [Ch4 p.25]
- X가 될 수 있는 것 = MUX 선택 신호뿐, 그 MUX 출력을 아무도 안 쓸 때.
  RegWrite·MemWrite·MemRead·Branch는 X 불가

외우는 고리

푸는 순서: 레지스터에 쓰나(RegWrite) → 메모리 값인가(MemtoReg) → 메모리 읽나·쓰나 → 즉시값인가(ALUSrc) → 분기인가 → ALU 할 일(00 덧셈, 01 뺄셈, 10 funct)

---

4 떠올리기

떠올리기 카드 9장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

lw의 MemtoReg는?

답 보기

---

sw의 MemtoReg는?

답 보기

---

beq의 ALUSrc는?

답 보기

---

sw의 ALUSrc와 MemWrite는?

답 보기

R형(add)의 RegWrite, MemtoReg, ALUSrc는?

답 보기

---

ld의 MemRead와 RegWrite는?

답 보기

---

beq의 RegWrite, MemtoReg, Branch, ALUOp는?

답 보기

Branch = 0, MemWrite = 0, RegWrite = 1, ALUSrc = 1인 명령어와 그 MemtoReg는?

답 보기

O/X: sw의 RegWrite는 결과에 영향이 없으므로 X로 둬도 된다.

답 보기

Ch4 · 7 약 4분

PCSrc = Branch AND Zero

map 상: PCSrc만 opcode로 정할 수 없음. 빈칸 단골

1 읽기 2 핵심 3 떠올리기

---

1 읽기

한 줄 요약: 다음 PC를 분기 목표로 바꾸는 신호 PCSrc는 Branch AND Zero로 만들고, 제어 신호 중 유일하게 opcode만으로 정할 수 없다.

Zero는 ALU 출력으로, ALU 결과가 0이면 1이다. ALU는 모든 명령어에서 계산하므로 Zero도 언제나 나온다. add의 결과가 우연히 0이어도 Zero = 1이다. beq에서는 ALU가 rs1 − rs2를 계산하므로 Zero = 1 ⇔ 두 레지스터 값이 같음이다.

그래서 Zero만 보고 분기하면 안 된다. 제어 유닛이 opcode를 보고 beq일 때만 Branch = 1을 내고, **AND 게이트**(두 입력이 모두 1일 때만 1을 내는 회로)가 Branch와 Zero를 받아 PCSrc를 만든다. PCSrc = 1이면 다음 PC = 분기 목표 주소, 0이면 PC + 4다.

PCSrc가 특별한 이유: 다른 신호는 명령어 종류(opcode)만 알면 미리 정해진다. 하지만 Zero는 실행 중에 레지스터에 든 값을 실제로 빼 봐야 알 수 있어서, opcode만 보는 제어 유닛은 PCSrc를 미리 정할 수 없다. beq라도 Zero가 0이면 PCSrc = 0이다.

예 1 `beq x1, x2, L`, x1 = 5, x2 = 7: ALU 5 − 7 = −2 ≠ 0 → Zero = 0, PCSrc = 1 AND 0 = 0 → 다음 PC = PC + 4(분기 안 함). 예 2 `add x5, x6, x7`, x6 = 4, x7 = −4: ALU 결과 0 → Zero = 1이지만 add라 Branch = 0, PCSrc = 0 AND 1 = 0 → PC + 4.

비유: 제어 유닛은 opcode를 보고 스위치를 한 번에 정하는 조작판이다. 안 맞는 점: PCSrc 스위치 하나만은 조작판만으로 못 정하고 계산 결과(Zero)를 봐야 한다.

2 · 핵심 정리

- PCSrc = Branch AND Zero [Ch4 p.25]
- Zero = ALU 결과가 0이면 1. 모든 명령어에서 나온다 [Ch4 p.12]
- beq: ALU가 rs1 − rs2 → Zero = 1 ⇔ 두 값이 같음 [Ch4 p.15]
- PCSrc는 opcode만으로 정할 수 없는 유일한 신호(Zero는 실행 중에 정해짐) [Ch4 p.25]

4 떠올리기

떠올리기 카드 4장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

빈칸: PCSrc = ( ) AND ( )

답 보기

PCSrc가 opcode만으로 정해지지 않는 이유는?

답 보기

---

add x5, x6, x7에서 x6 = 9, x7 = −9. Zero와 PCSrc는?

답 보기

---

beq x1, x2, L에서 x1 = 5, x2 = 7. Zero와 다음 PC는?

답 보기

Ch4 · 8 약 6분

ALUOp + funct → ALU 제어 4비트

map 상: ALU 제어 진리표 채우기

1 읽기 2 핵심 3 떠올리기

---

1 읽기

한 줄 요약: ALU에 들어가는 4비트 ALU 제어 값은 ALU 제어 유닛이 ALUOp(2비트)와 funct7·funct3을 보고 만든다.

ALU 제어 4비트 값은 정해진 약속이라 그대로 외운다: 0000 = AND, 0001 = OR, 0010 = add(덧셈), 0110 = sub(뺄셈). 외우는 고리: 끝 두 자리가 00 AND, 01 OR, 10 덧셈이고, 뺄셈만 덧셈 0010 앞에 1이 붙은 0110이다.

주 제어 유닛은 명령어 종류만 보고 ALUOp를 낸다. lw·sw는 주소를 더해야 하니 00(무조건 덧셈), beq는 두 레지스터를 빼서 비교하니 01(무조건 뺄셈), R형은 add·sub·and·or 중 무엇인지 funct를 봐야 하니 10. ALUOp 두 비트는 왼쪽이 ALUOp1, 오른쪽이 ALUOp0이다.

ALU 제어 유닛이 값을 정하는 법: ALUOp가 00이면 funct는 보지 않고 0010, 01이면 0110. ALUOp가 10일 때만 funct7·funct3이 둘 다 맞는 줄을 찾는다: add 0000000·000 → 0010, sub 0100000·000 → 0110, and 0000000·111 → 0000, or 0000000·110 → 0001. funct7이 0100000인 것은 sub뿐이다. 예: lw는 ALUOp 00 → funct와 상관없이 0010. 예: sub는 ALUOp 10 → funct7 0100000·funct3 000 줄 → 0110.

다단 디코딩(multi-level decoding) = 제어를 주 제어 → ALUOp → ALU 제어의 두 단계로 나누는 것이다. 주 제어 유닛은 opcode 7비트만 보면 되고 funct 10비트까지 볼 필요가 없어서 **주 제어 유닛이 작아진다**. 작은 제어 유닛 여러 개로 나누면 제어 유닛의 지연도 줄 수 있다.

ALU 제어 유닛의 입력은 ALUOp 2 + funct7 7 + funct3 3 = 12비트다. 입력 조합마다 출력을 적은 표(진리표)를 다 쓰면 2^{12} = 4096행이다. 실제로 나오는 조합은 몇 개뿐이라, 상관없는 입력 열을 **don't care(X)**로 적어 표를 작게 줄인다. 이 표의 X는 「출력이 그 입력에 달려 있지 않다」는 뜻으로, 신호 표의 X(출력 신호가 0이든 1이든 결과가 같다)와 가리키는 쪽이 다르다.

주의: 4096은 2^4가 아니라 입력 12비트의 2^{12}다. ALUOp(2비트)와 ALU 제어(4비트)를 섞지 말 것.

2 · 핵심 정리

- ALUOp: 00 = 덧셈(lw·sw), 01 = 뺄셈(beq), 10 = funct로 정함(R형) [Ch4 p.18]
- ALU 제어: 0000 AND, 0001 OR, 0010 add, 0110 sub [Ch4 p.18]
- R형: add 0000000·000, sub 0100000·000, and 0000000·111, or 0000000·110 (funct7·funct3) [Ch4 p.20]
- 다단 디코딩: 주 제어 → ALUOp → ALU 제어 → 주 제어 유닛이 작아지고 지연 감소 [Ch4 p.19]
- 입력 2 + 7 + 3 = 12비트 → 2^{12} = 4096 조합, don't care로 압축 [Ch4 p.20]

외우는 고리

ALU 제어 끝 두 자리: 00 AND, 01 OR, 10 덧셈. 뺄셈만 앞에 1 붙은 0110

---

4 떠올리기

떠올리기 카드 5장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

lw·sw의 ALUOp와 ALU 제어는?

답 보기

---

beq의 ALUOp와 ALU 제어는?

답 보기

ALUOp 10, funct7 0000000, funct3 111이면 ALU 제어는?

답 보기

ALUOp 10, funct7 0100000, funct3 000이면 ALU 제어는?

답 보기

ALU 제어 유닛 입력은 몇 비트이고 조합은 몇 개인가?

답 보기

Ch4 · 9 약 8분

단일 사이클의 한계와 클럭 주기

map 상: 「단일 사이클을 안 쓰는 이유」 서술 단골, 최장 경로·주기 계산

1 읽기 2 핵심 3 풀이 예 4 떠올리기

1 읽기

한 줄 요약: 단일 사이클은 동작은 맞지만, 모든 명령어가 가장 긴 lw에 맞춘 긴 클럭 주기를 써야 해서 너무 비효율적이다.

단일 사이클에서는 모든 명령어가 한 클럭 사이클에 끝나므로 모든 명령어의 사이클 길이가 같다. 다음 엣지 전에 가장 느린 명령어까지 끝나야 하므로 **가장 긴 경로**가 클럭 주기를 정한다. 여기서 경로는 한 명령어가 차례로 지나는 장치들의 줄이다.

한 명령어 안의 장치는 앞 장치의 출력을 받아야 일을 시작한다(직렬). ALU는 레지스터 파일이 값을 내기 전에 계산할 수 없고, 레지스터 파일은 명령어 메모리가 명령어를 내기 전에 무엇을 읽을지 모른다. 그래서 한 명령어의 시간 = 지나는 장치 지연의 합이다. PC + 4 가산기와 분기 가산기는 ALU와 동시에 일하므로 더하지 않고, 문제에 「나머지 무시」라 하면 MUX·가산기·제어 유닛 지연은 0으로 본다.

명령어별 경로(제어 신호가 0이면 그 장치를 안 씀): lw = 명령어 메모리 → 레지스터 파일(읽기) → ALU → 데이터 메모리 → 레지스터 파일(쓰기)로 5개, 가장 길다. sw = 마지막 레지스터 쓰기가 빠진 4개(RegWrite = 0). R형 = 데이터 메모리가 빠진 4개. beq = 데이터 메모리와 레지스터 쓰기가 둘 다 빠진 3개(명령어 메모리 → 레지스터 파일 → ALU).

예: 명령어 메모리 300 ps, 레지스터 파일 읽기·쓰기 각 100 ps, ALU 200 ps, 데이터 메모리 300 ps(나머지 무시). lw = 300 + 100 + 200 + 300 + 100 = 1000 ps, sw = 900 ps, R형 = 700 ps, beq = 600 ps. 클럭 주기 = 가장 긴 1000 ps이고, beq는 600 ps면 끝나지만 1000 ps를 기다린다. 명령어 10^6개면 CPU 시간 = IC × CPI × 주기 = 10^6 × 1 × 1000 × 10^{-12} s = 10^{-3} s = 1 ms.

명령어마다 주기를 다르게 하는 것은 불가능하다(not feasible). 그래서 자주 쓰는 짧은 명령어도 lw 길이를 기다리고, 이는 Ch1 8대 아이디어의 「자주 생기는 경우를 빠르게(Make the common case fast)」를 어긴다. 이를 개선하는 방법이 파이프라이닝이다.

주의: 단일 사이클 CPU 시간 = IC × 1 × (가장 긴 경로 지연). CPI는 1로 최소지만 주기가 길어서 「CPI가 1이니 가장 빠르다」는 틀린 말이다. 가장 긴 경로는 sw가 아니라 lw다(sw는 마지막 레지스터 쓰기가 없다).

2 · 핵심 정리

- 모든 명령어가 같은 길이의 클럭 사이클, 가장 긴 경로가 주기를 정함 [Ch4 p.29]
- 최장 = lw: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일(5개 직렬) [Ch4 p.29]
- 명령어마다 주기를 다르게 = 불가능 → 자주 생기는 경우를 빠르게 위반 → 파이프라이닝 [Ch4 p.29]
- CPU 시간 = IC × 1 × (최장 경로 지연). CPI는 최소지만 주기가 길다 [Ch1 p.28]
- 빠지는 장치: sw = 마지막 레지스터 쓰기, R형 = 데이터 메모리, beq = 둘 다

외우는 고리

lw 경로 = 「명 레 알 데 레」(명령어 메모리, 레지스터 파일, ALU, 데이터 메모리, 레지스터 파일)

---

3 풀이 예

푸는 순서

① 명령어가 지나는 장치를 차례로 쓴다(lw 5개, sw 4개, R형 4개, beq 3개)

② 직렬이라 지연을 더한다. 가산기·MUX는 「나머지 무시」면 0

③ 클럭 주기 = 가장 긴 합(lw)

④ CPU 시간 = IC × 1 × 주기

명령어 메모리 200 ps, 레지스터 파일 읽기·쓰기 각 100 ps, ALU 150 ps, 데이터 메모리 250 ps(나머지 무시). 클럭 주기, R형에 실제로 필요한 시간, 명령어 10^9개의 CPU 시간은?

① lw: 200 + 100 + 150 + 250 + 100 = 800 ps

② R형(데이터 메모리 없음): 200 + 100 + 150 + 100 = 550 ps

③ 주기 = 가장 긴 lw = 800 ps. R형도 800 ps를 쓴다

④ CPU 시간 = $10^9 \times 1 \times 800 \times 10^{-12}$ s = 0.8 s

답: **주기 800 ps, R형 550 ps, CPU 시간 0.8 s**

---

4 떠올리기

빈칸 풀이 · 노란 칸을 먼저 떠올리고 눌러 확인

명령어 메모리 250 ps, 레지스터 파일 읽기·쓰기 각 150 ps, ALU 200 ps, 데이터 메모리 350 ps(나머지 무시). 클럭 주기, sw에 실제로 필요한 시간, 명령어 2×10^6개의 CPU 시간은?

① lw: 250 + 150 + 200 + 350 + 150 = 1100 ps

② sw(레지스터 쓰기 없음): 250 + 150 + 200 + 350 = 950 ps

③ 주기 = 1100 ps

④ CPU 시간 = $2 \times 10^6 \times 1 \times 1100 \times 10^{-12}$ s = 2.2×10^{-3} s = 2.2 ms

답 보기

떠올리기 카드 4장 · 답을 떠올린 뒤 펼치고 [맞음]/[틀림]

단일 사이클에서 가장 긴 경로의 명령어와 장치 순서는?

답 보기

단일 사이클이 위반하는 8대 아이디어는?

답 보기

O/X: 단일 사이클은 CPI가 1이라 가장 빠르다.

답 보기

명령어 메모리 300, 레지스터 파일 120, ALU 180, 데이터 메모리 300 ps. beq에 실제로 필요한 시간은?

답 보기

마무리 약 10분

시험 전날 한 장

노란 칸은 가려져 있습니다. 먼저 떠올리고 눌러서 확인하세요. 시험 날 아침에 이것만 한 번 더.

기초

- 16진 한 자리 = 2진 4비트, 32비트 = 16진 8자리 (0xB7 = $1011\ 0111_2$ = 183)
- 2의 보수 부정 = ~x + 1, n비트 범위 $-2^{n-1} \sim 2^{n-1} - 1$ [Ch2 p.19–20]
- 부호 확장 = 부호 비트를 왼쪽에 복제(1111 1110 → 1111…1110, 여전히 −2) [Ch2 p.20]
- 왼쪽 시프트 k = × 2^k, sra = 부호 비트로 채우며 오른쪽 시프트 [Ch2 p.25]
- 1 ps = 10^{-12} s, 4 GHz ↔ 250 ps (역수)
- KB = 10^3, KiB = 2^{10}
- 8비트 −20 = $1110\ 1100_2$ = 0xEC, 32비트 0xFFFFFFF0 = −16

Ch1

- CPU 시간 = IC × CPI × 클럭 주기 = IC × CPI / 클럭 속도 [Ch1 p.28]
- 성능 = 1 / 실행 시간, 「X가 n배 빠름」 ⇔ 시간_Y / 시간_X = n [Ch1 p.24]
- MIPS = 클럭 속도 / (CPI × 10^6). ISA가 다르면 비교 불가 [Ch1 p.34]
- 암달: $T_{\text{개선}}$ = $T_{\text{영향}}$ / n + $T_{\text{비영향}}$ (100 s 중 80 s, 2.5배 → n = 4) [Ch1 p.33]
- 전력 ∝ 용량성 부하 × 전압^2 × 주파수, 전력 장벽 → 멀티코어 [Ch1 p.30–32]
- 8대 아이디어: 무어의 법칙, 추상화, 자주 생기는 경우를 빠르게, 병렬성, 파이프라이닝, 예측, 메모리 계층, 중복을 통한 신뢰성 [Ch1 p.12]
- 5대 구성 요소: 입력, 출력, 메모리, 데이터패스, 제어 [Ch1 p.15]
- IC = ISA·컴파일러, CPI·클럭 주기 = CPU 하드웨어 [Ch4 p.2]
- 임베디드 = 다른 기기 안, 정해진 응용, 전력·성능·비용 제약 엄격 [Ch1 p.3–7]

Ch2

- R: funct7 7 · rs2 5 · rs1 5 · funct3 3 · rd 5 · opcode 7
- I: imm[11:0] 12비트 · rs1 · funct3 · rd · opcode (lw, addi, jalr) / S: imm[11:5] · rs2 · rs1 · funct3 · imm[4:0](11–7번) · opcode (sw)
- x0 = 항상 0, x1 = 복귀 주소, x2 = sp, x10~x17 = 인자(x10·x11 반환값) [Ch2 p.16, 34]
- word 배열 A[i] 오프셋 = 4 × i, 리틀 엔디언 = 최하위 바이트가 가장 낮은 주소 [Ch2 p.12–13]
- 설계 원칙 3: 단순성은 규칙성을 선호 / 작을수록 빠르다 / 좋은 설계는 좋은 타협 [Ch2 p.9–10, 23]
- 주소 지정 4: 즉시값, 레지스터, 베이스(lw·sw·jalr), PC 상대(beq·bne·jal) [Ch2 p.49]
- 분기 목표 = PC + 오프셋 필드 × 2, 범위 −4096 ~ 4094바이트 [Ch2 p.47]
- 스택은 아래로 자람, 레지스터 하나당 sp 4 (5개 저장 → addi sp, sp, -20) [Ch2 p.37–38]
- 프로시저 6단계: 인자 배치 → 제어 이동 → 저장 공간 확보 → 작업 → 결과 배치 → 복귀 [Ch2 p.33]

Ch3

- 오버플로: + + + = −, − + − = +, + − − = −, − − + = +. 부호 다른 덧셈·부호 같은 뺄셈은 없음 [Ch3 p.4–5]
- mul = 하위 32비트, mulh(부호 × 부호)·mulhu(무 × 무)·mulhsu(부호 × 무) = 상위 32비트 [Ch3 p.13]
- 부호 있는 나눗셈: 부호가 다르면 몫 음수, 나머지 부호 = 피제수 부호(−7 ÷ 2 → 몫 −3, 나머지 −1) [Ch3 p.19]
- 단정도 1 · 8 · 23, bias 127 / 배정도 1 · 11 · 52, bias 1023 [Ch3 p.28]
- 16비트 1 · 5 · 10, bias 15 (PDF 외 보충, 26-1 기출)
- 값 = $(-1)^S \times (1 + F) \times 2^{(\text{지수 칸} - \text{bias})}$, −0.75 = 0xBF400000 [Ch3 p.28, 31]
- FP 덧셈 4단계: 소수점 맞추기 → 유효숫자 더하기 → 정규화·검사 → 반올림 [Ch3 p.32]
- FP 곱셈 5단계: 지수 더하기(bias 한 번 빼기) → 유효숫자 곱 → 정규화·검사 → 반올림 → 부호 결정 [Ch3 p.35]

Ch4

- 구현 7개: lw, sw, add, sub, and, or, beq / 프로세서 = 데이터패스 + 제어 [Ch4 p.3, 8]
- R형 0 0 1 0 0 0 10 / lw 1 1 1 1 0 0 00 (ALUSrc · MemtoReg · RegWrite · MemRead · MemWrite · Branch · ALUOp) [Ch4 p.25]
- sw 1 X 0 0 1 0 00 / beq 0 X 0 0 0 1 01 (ld = lw, sd = sw)
- ALU 제어: 0000 AND, 0001 OR, 0010 add, 0110 sub [Ch4 p.18]
- PCSrc = Branch AND Zero, opcode만으로 못 정하는 유일한 신호 [Ch4 p.25]
- rs1 19:15, rs2 24:20, rd 11:7, opcode [6:0] [Ch4 p.21]
- 조합 = 출력이 입력만의 함수, 상태 = 저장, 엣지 트리거 = 클럭 엣지에서만 갱신 [Ch4 p.9–10]
- 메모리 둘 = 한 사이클에 자원 한 번만 [Ch4 p.16]
- 최장 경로 lw: 명령어 메모리 → 레지스터 파일 → ALU → 데이터 메모리 → 레지스터 파일 [Ch4 p.29]
- 단일 사이클 = 자주 생기는 경우를 빠르게 위반 → 파이프라이닝 [Ch4 p.29]

틀린 문제 모아 보기

오답 노트

회독 점검 문항 아래의 **[틀림]**을 누른 문제가 단원별로 여기에 모입니다. 다시 풀어 맞히면 **[다시 풀어 맞음]**을 눌러 목록에서 빼세요. 다음 회독을 시작하기 전에 여기부터 다시 풉니다.
